由上图不难计算出

$$ASL_{成功}=(1\times7+2\times2+3\times1+4\times1)/11=18/11$$

$$ASL_{失败}=(1+0+2+1+0+1+1+0+0+0+1+0+4)/13=11/13$$

05.【解答】

1）$H_0(2)=2$，$H_0(8)=8$，$H_0(31)=5$，$H_0(20)=7$，$H_0(19)=6$，没有冲突。$H_0(18)=5$，发生冲突，$H_1(18)=(H_0(18)+REV(18+1)\%11+1)\%13=(5+3+1)\%13=9$，没有冲突。$H_0(53)=1$，没有冲突。$H_0(27)=1$，发生冲突，$H_1(27)=(H_0(27)+REV(27+1)\%11+1)\%13=(1+5+1)\%13=7$，发生冲突，$H_2(27)=(H_1(27)+REV(27+1)\%11+1)\%13=0$，没有冲突。构造的散列表如下：

散列地址	0	1	2	3	4	5	6	7	8	9	10	11	12
关键字	27	53	2			31	19	20	8	18			
比较次数	3	1	1			1	1	1	1	2			

2）由 1）中散列表的构造过程，各个关键字查找成功的比较次数如上表所示，所以有

$$ASL_{成功}=(3+1+1+1+1+1+1+2)/8=11/8$$

06.【解答】

1）由装填因子 0.7 和数据总数 7，得一维数组大小为 7/0.7 = 10，数组下标为 0～9。所构造的散列函数值如下所示：

key	7	8	30	11	18	9	14
H(key)	0	3	6	5	5	6	0

采用线性探测再散列法处理冲突，所构造的散列表为

地址	0	1	2	3	4	5	6	7	8	9
关键字	7	14		8		11	30	18	9	

2）查找成功时，在等概率情况下，查找每个表中元素的概率是相等的。因此，根据表中元素的个数来计算平均查找长度，各关键字的比较次数如下所示：

key	7	8	30	11	18	9	14
比较次数	1	1	1	1	3	3	2

所以 $ASL_{成功}$ = 查找次数/元素个数 $=(1+2+1+1+1+3+3)/7=12/7$。

在计算查找失败时的平均查找长度时，要特别注意防止思维定式，在查找失败的情况下既不是根据表中的元素个数，也不是根据表长来计算平均查找长度的。

查找失败时，在等概率情况下，经过散列函数计算后只可能映射到表中的 0～6 位置，且映射到 0～6 中任意一个位置的概率是相等的。因此，是根据散列函数（mod 后面的数字）来计算平均查找长度的。在等概率情况下，查找失败的比较次数如下所示：

H(key)	0	1	2	3	4	5	6
比较次数	3	2	1	2	1	5	4

所以 $ASL_{不成功}$ = 查找次数/散列后的地址个数 $=(3+2+1+2+1+5+4)/7=18/7$。

07.【解答】

1）①$H(20)=5$，装入地址 5；②$H(3)=9$，装入地址 9；③$H(11)=0$，装入地址 0；④$H(18)=10$，装入地址 10；⑤$H(9)=5$，冲突，$H_1(9)=(5+1)\%11=6$，装入地址 6；⑥$H(14)=9$，冲突，$H_1(14)=(9+1)\%11=10$，冲突，$H_2(14)=(9+4)\%11=2$，装入地址 2。散列表如下。

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字	11		14	7		20	9			3	18

装填因子 $\alpha = 7/11$。

2）$H(14) = 9$，和关键字 3 比较，不命中；$H_1(14) = (9 + 1)\%11 = 10$，和 18 比较，不命中；$H_2(14) = (9 + 4)\%11 = 2$，和 14 比较，命中。因此，关键字比较序列是 3, 18, 14。

3）$H(8) = 2$，和关键字 14 比较，不命中；$H_1(8) = (2 + 1)\%11 = 3$，和 7 比较，不命中；$H_2(8) = (2 + 4)\%11 = 6$，和 9 比较，不命中；$H_3(8) = (2 + 9)\%11 = 0$，和 11 比较，不命中；$H_4(8) = (2 + 16)\%11 = 7$，是空位置，确认查找失败。因此，确认查找失败时的散列地址是 7。

归纳总结

本章的核心考查点是求平均查找长度（ASL），以度量各种查找算法的性能。查找算法本身依托于查找结构，查找结构又是由相同数据类型的记录或结点构成的，所以最终落脚于数据结构类型的区别。不管是何种查找算法，其平均查找长度的计算公式都是一样的。

查找成功的平均查找长度 $\mathrm{ASL}_{\text{成功}} = \sum_{i=1}^{n} p_i c_i$。

查找失败的平均查找长度 $\mathrm{ASL}_{\text{不成功}} = \sum_{j=0}^{n} q_j c_j$。

设一个查找集合中已有 n 个数据元素，每个元素的查找概率为 p_i，查找成功的数据比较次数为 c_i（$i = 1, 2, \cdots, n$）；不在此集合中的数据元素分布在由这 n 个元素的间隔构成的 $n + 1$ 个子集合内，每个子集合元素的查找概率为 q_j，查找不成功的数据比较次数为 c_j（$j = 0, 1, \cdots, n$）。因此，对某一特定查找算法的查找成功的 $\mathrm{ASL}_{\text{成功}}$ 和查找失败的 $\mathrm{ASL}_{\text{不成功}}$，是综合考虑还是分开考虑呢？

若综合考虑，即 $\sum_{i=1}^{n} p_i + \sum_{j=0}^{n} q_j = 1$，若所有元素查找概率相等，则有 $p_i = q_j = \frac{1}{2n+1}$；若分开考虑，即 $\sum_{i=1}^{n} p_i = 1$，$\sum_{j=0}^{n} q_j = 1$，若所有元素查找概率相等，则有 $p_i = \frac{1}{n}$，$q_j = \frac{1}{n+1}$。

虽然综合考虑更为理想，但在实际应用中多数是分开考虑的，因为对于查找不成功的情况，很多场合下没有明确给出，往往会被忽略掉。不过读者仍要注意的是，这两种考虑的计算结果是不同的，考试中一定要仔细阅读题目的要求，以免失误。

思维拓展

本章介绍了几种基本的查找算法，在实际中又会碰到怎样的查找问题呢？

题目：数组中有一个数字出现的次数超过了数组长度的一半，请找出这个数字。读者也许会想到先进行排序，位于位置$(n + 1)/2$ 的数即要找的数，这样最小时间复杂度就为 $O(n\log_2 n)$；若进行散列查找，数字的范围又未知，则应如何将时间复杂度控制在 $O(n)$内呢？

提　示

出现的次数超过数组长度的一半，表明这个数字出现的次数比其他数字出现的次数的总和还多。所以我们可以考虑每次删除两个不同的数，则在剩下的数中，待查找数字出现的次数仍然超过总数的一半。通过不断重复这个过程，不断排除其他的数字，最终剩下的都为同一个数字，即要找的数字。

第8章 排　序

【考纲内容】

扫一扫

视频讲解

（一）排序的基本概念

（二）插入排序

直接插入排序；折半插入排序；希尔排序（shell sort）

（三）交换排序

冒泡排序（bubble sort）；快速排序

（四）选择排序

简单选择排序；堆排序

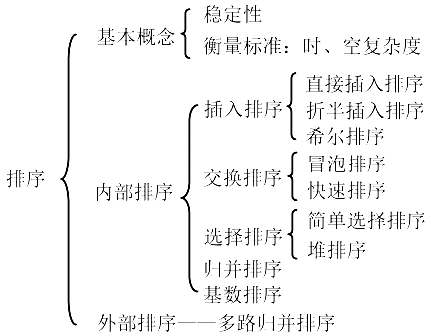
（五）二路归并排序（merge sort）

（六）基数排序

（七）外部排序

（八）排序算法的分析和应用

【知识框架】



【复习提示】

堆排序、快速排序和归并排序是本章的重难点。读者应深入掌握各种排序算法的思想、排序过程（能动手模拟）和特征（初态的影响、复杂度、稳定性、适用性等），通常以选择题的形式考查不同算法之间的对比。此外，对于一些常用排序算法的关键代码，要达到熟练编写的程度；看到某特定序列，读者应具有选择最优排序算法（根据排序算法特征）的能力。

8.1 排序的基本概念

8.1.1 排序的定义

排序，就是重新排列表中的元素，使表中的元素满足按关键字有序的过程。为了查找方便，通常希望计算机中的表是按关键字有序的。排序的确切定义如下：

输入：n 个记录 $R_1, R_2, \cdots, R_n$，对应的关键字为 $k_1, k_2, \cdots, k_n$。

输出：输入序列的一个重排 $R_1', R_2', \cdots, R_n'$，使得 $k_1' \leqslant k_2' \leqslant \cdots \leqslant k_n'$（其中“$\leqslant$”可以换成其他的比较大小的符号）。

算法的稳定性。若待排序表中有两个元素 R_i 和 R_j，其对应的关键字相同，即 $\text{key}_i = \text{key}_j$，且在排序前 R_i 在 R_j 的前面，若使用某一排序算法排序后，R_i 仍然在 R_j 的前面，则称这个排序算法是稳定的，否则称这个排序算法是不稳定的。需要注意的是，算法是否具有稳定性并不能衡量一个算法的优劣，它主要是对算法的性质进行描述。若待排序表中的关键字不允许重复，排序结果是唯一的，则对于排序算法的选择，稳定与否无关紧要。

注 意

对于不稳定的排序算法，只需举出一组关键字的实例，说明它的不稳定性即可。

在排序过程中，根据数据元素是否完全存放在内存中，可将排序算法分为两类：①内部排序，是指在排序期间元素全部存放在内存中的排序；②外部排序，是指在排序期间元素无法全部同时存放在内存中，必须在排序的过程中根据要求不断地在内、外存之间移动的排序。

一般情况下，内部排序算法在执行过程中都要进行两种操作：比较和移动。通过比较两个关键字的大小，确定对应元素的前后关系，然后通过移动元素以达到有序。当然，并非所有的内部排序算法都要基于比较操作，事实上，基数排序就不基于比较操作。

每种排序算法都有各自的优缺点，适合在不同的环境下使用，就其全面性能而言，很难提出一种被认为是最好的算法。通常可以将排序算法分为插入排序、交换排序、选择排序、归并排序和基数排序五大类，后面几节会分别进行详细介绍。内部排序算法的性能取决于算法的时间复杂度和空间复杂度，而时间复杂度一般是由比较和移动的次数决定的。

注 意

大多数的内部排序算法都更适用于顺序存储的线性表。

8.1.2 本节试题精选

单项选择题

01. 下述排序算法中，不属于内部排序算法的是（ ）。

A. 插入排序　　B. 选择排序　　C. 拓扑排序　　D. 冒泡排序

02. 排序算法的稳定性是指（ ）。

A. 经过排序后，能使关键字相同的元素保持原顺序中的相对位置不变

B. 经过排序后，能使关键字相同的元素保持原顺序中的绝对位置不变

C. 排序算法的性能与被排序元素个数关系不大

D. 排序算法的性能与被排序元素的个数关系密切

03. 下列关于排序的叙述中，正确的是（　）。

A. 稳定的排序算法优于不稳定的排序算法

B. 对同一线性表使用不同的排序算法进行排序，得到的排序结果可能不同

C. 排序算法都是在顺序表上实现的，在链表上无法实现排序算法

D. 在顺序表上实现的排序算法在链表上也可以实现

8.1.3　答案与解析

单项选择题

01. C

拓扑排序是将有向图中所有结点排成一个线性序列，虽然也是在内存中进行的，但它不属于我们这里所提到的内部排序范畴，也不满足前面排序的定义。

02. A

注意，这里的绝对位置是指若在排序前元素 R 在位置 i，则绝对位置就是 i，即排序后 R 的位置不发生变化，显然选项 B 是不对的。选项 C、D 与题目要求无关。

03. B

算法的稳定性与算法优劣无关，选项 A 排除。使用链表也可以进行排序，只是有些排序算法不再适用，因为这时定位元素只能顺序逐链查找，如折半插入排序。

8.2　插入排序

插入排序是一种简单直观的排序算法，其基本思想是每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。由插入排序的思想可以引申出三个重要的排序算法：直接插入排序、折半插入排序和希尔排序。

8.2.1　直接插入排序[①]

根据上面的插入排序思想，不难得出一种最简单也最直观的直接插入排序算法。假设在排序过程中，待排序表 `L[1…n]` 在某次排序过程中的某一时刻状态如下：

有序序列 `L[1…i-1]`	`L(i)`	无序序列 `L[i+1…n]`

要将元素 `L(i)` 插入已有序的子序列 `L[1…i-1]`，需要执行以下操作（为避免混淆，下面用 `L[]` 表示一个表，而用 `L()` 表示一个元素）：

1）查找出 `L(i)` 在 `L[1…i-1]` 中的插入位置 `k`。

2）将 `L[k…i-1]` 中的所有元素依次后移一个位置。

3）将 `L(i)` 复制到 `L(k)`。

为了实现对 `L[1…n]` 的排序，可以将 `L(2)`～`L(n)` 依次插入前面已排好序的子序列，初始 `L[1]` 可以视为一个已排好序的子序列。上述操作执行 $n-1$ 次就能得到一个有序的表。插入排序在实现上通常采用原地排序（空间复杂度为 $O(1)$），因而在从后往前的比较过程中，需要反复把已排序元素逐步向后挪位，为新元素提供插入空间。

① 在本书中，凡是没有特殊注明的，通常默认排序结果为非递减有序序列。

下面是直接插入排序的代码，其中再次用到了前面提到的“哨兵”（作用相同）。

```
void InsertSort(ElemType A[],int n){
    int i,j;
    for(i=2;i<=n;i++)                    //依次将A[2]～A[n]插入前面已排序序列
        if(A[i]<A[i-1]){                 //若A[i]关键码小于其前驱，将A[i]插入有序表
            A[0]=A[i];                   //复制为哨兵，A[0]不存放元素
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
                A[j+1]=A[j];             //向后挪位
            A[j+1]=A[0];                 //复制到插入位置
        }
}
```

假定初始序列为 49, 38, 65, 97 ,76, 13, 27, $\overline{49}$，初始时 49 可以视为一个已排好序的子序列，按照上述算法进行直接插入排序的过程如图 8.1 所示，括号内是已排好序的子序列。

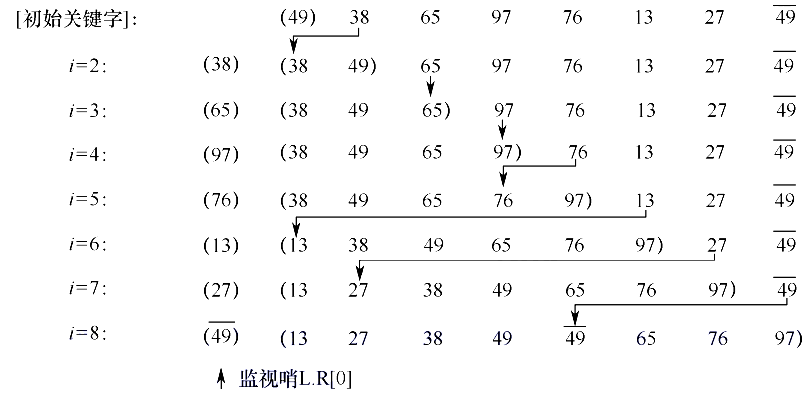


图 8.1　直接插入排序示例

直接插入排序算法的性能分析如下：

空间效率：仅使用了常数个辅助单元，因而空间复杂度为 $O(1)$。

时间效率：在排序过程中，向有序子表中逐个地插入元素的操作进行了 $n-1$ 趟，每趟操作都分为比较关键字和移动元素，而比较次数和移动次数取决于待排序表的初始状态。

在最好情况下，表中元素已经有序，此时每插入一个元素，都只需比较一次而不用移动元素，因而时间复杂度为 $O(n)$。

在最坏情况下，表中元素顺序刚好与排序结果中的元素顺序相反（逆序），总的比较次数达到最大，总的移动次数也达到最大，总的时间复杂度为 $O(n^2)$。

平均情况下，考虑待排序表中元素是随机的，此时可以取上述最好与最坏情况的平均值作为平均情况下的时间复杂度，总的比较次数与总的移动次数均约为 $n^2/4$。

因此，直接插入排序算法的时间复杂度为 $O(n^2)$。

稳定性：因为每次插入元素时总是从后往前先比较再移动，所以不会出现相同元素相对位置发生变化的情况，即直接插入排序是一个稳定的排序算法。

适用性：直接插入排序适用于顺序存储和链式存储的线性表，采用链式存储时无须移动元素。

8.2.2　折半插入排序

从直接插入排序算法中，不难看出每趟插入的过程中都进行了两项工作：①从前面的有序子表中查找出待插入元素应该被插入的位置；②给插入位置腾出空间，将待插入元素复制到表中的插入位置。注意到在该算法中，总是边比较边移动元素。下面将比较和移动操作分离，即先折半

查找出元素的待插入位置，然后统一地移动待插入位置之后的所有元素。当排序表为顺序表时，可以对直接插入排序算法做如下改进：因为是顺序存储的线性表，所以查找有序子表时可以用折半查找来实现。确定待插入位置后，就可统一地向后移动元素。

折半插入排序算法的排序过程举例见本书配套课程，其代码如下：

```
void InsertSort(ElemType A[],int n){
    int i,j,low,high,mid;
    for(i=2;i<=n;i++){                  //依次将 A[2]～A[n]插入前面的已排序序列
        A[0]=A[i];                      //将 A[i]暂存到 A[0]
        low=1;high=i-1;                 //设置折半查找的范围
        while(low<=high){               //折半查找(默认递增有序)
            mid=(low+high)/2;           //取中间点
            if(A[mid]>A[0]) high=mid-1; //查找左半子表
            else low=mid+1;             //查找右半子表
        }
        for(j=i-1;j>=high+1;--j)
            A[j+1]=A[j];                //统一后移元素，空出插入位置
        A[high+1]=A[0];                 //插入操作
    }
}
```

命题追踪 ▶ 直接插入排序和折半插入排序的比较（2012）

从上述算法中，不难看出折半插入排序仅减少了比较元素的次数，时间复杂度约为 $O(n\log_2 n)$，该比较次数与待排序表的初始状态无关，仅取决于表中的元素个数 n；而元素的移动次数并未改变，它依赖于待排序表的初始状态。因此，折半插入排序的时间复杂度仍为 $O(n^2)$，但对于数据量不算很大的排序表，折半插入排序往往能表现出很好的性能。折半插入排序是一种稳定的排序算法。

折半插入排序仅适用于顺序存储的线性表。

8.2.3　希尔排序

从前面的分析可知，直接插入排序算法的时间复杂度为 $O(n^2)$，但若待排序列为“正序”时，其时间效率可提高至 $O(n)$，由此可见它更适用于基本有序的排序表和数据量不大的排序表。希尔排序正是基于这两点分析对直接插入排序进行改进而得来的，也称缩小增量排序。

命题追踪 ▶ 希尔排序中各子序列采用的排序算法（2015）

希尔排序的基本思想是：先将待排序表分割成若干形如 $L[i, i+d, i+2d, \cdots, i+kd]$的“特殊”子表，即把相隔某个“增量”的记录组成一个子表，对各个子表分别进行直接插入排序，当整个表中的元素已呈“基本有序”时，再对全体记录进行一次直接插入排序。

命题追踪 ▶ 根据希尔排序的中间过程判断所采用的增量（2014、2018）

希尔排序的过程如下：先取一个小于 n 的增量 d_1，把表中的全部记录分成 d_1 组，所有距离为 d_1 的倍数的记录放在同一组，在各组内进行直接插入排序；然后取第二个增量 $d_2 < d_1$，重复上述过程，直到所取到的 $d_t = 1$，即所有记录已放在同一组中，再进行直接插入排序，此时已经具有较好的局部有序性，因此可以很快得到最终结果。到目前为止，尚未求得一个最好的增量序列。仍以 8.2.1 节的关键字为例，假定第一趟取增量 $d_1 = 5$，将该序列分成 5 个子序列，即图中第 2 行至第 6 行，分别对各子序列进行直接插入排序，结果如第 7 行所示；假定第二趟取增量 $d_2 = 3$，分别对三个子序列进行直接插入排序，结果如第 11 行所示；最后对整个序列进行一趟直接插入排序，整个排序过程如图 8.2 所示。

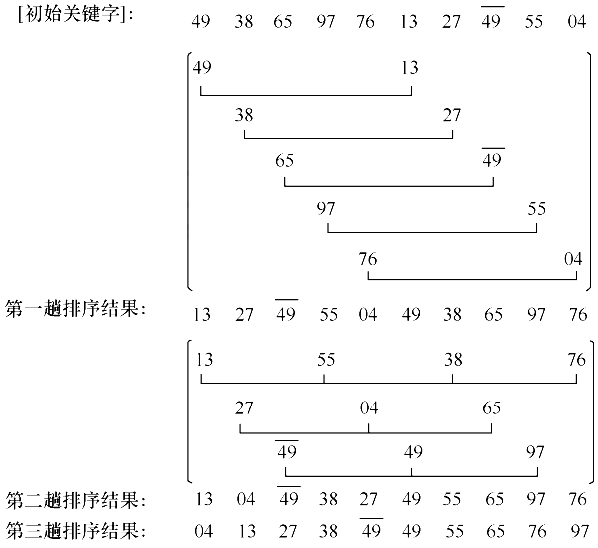


图 8.2 希尔排序示例

希尔排序算法的代码如下：

```
void ShellSort(ElemType A[],int n){
//A[0]只是暂存单元，不是哨兵，当 j<=0 时，插入位置已到
    int dk,i,j;
    for(dk=n/2;dk>=1;dk=dk/2)          //增量变化（无统一规定）
        for(i=dk+1;i<=n;++i)
            if(A[i]<A[i-dk]){          //需将 A[i]插入有序增量子表
                A[0]=A[i];             //暂存在 A[0]
                for(j=i-dk;j>0&&A[0]<A[j];j-=dk)
                    A[j+dk]=A[j];      //记录后移，查找插入的位置
                A[j+dk]=A[0];          //插入
            }
}
```

希尔排序算法的性能分析如下：

空间效率：仅使用了常数个辅助单元，因而空间复杂度为 $O(1)$。

时间效率：因为希尔排序的时间复杂度依赖于增量序列的函数，这涉及数学上尚未解决的难题，所以其时间复杂度分析比较困难。当 n 在某个特定范围时，希尔排序的时间复杂度约为 $O(n^{1.3})$。在最坏情况下希尔排序的时间复杂度为 $O(n^2)$。

稳定性：当相同关键字的记录被划分到不同的子表时，可能会改变它们之间的相对次序，因此希尔排序是一种不稳定的排序算法。例如，图 8.2 中 49 与 $\overline{49}$ 的相对次序已发生了变化。

适用性：希尔排序仅适用于顺序存储的线性表。

8.2.4 本节试题精选

一、单项选择题

01. 对 5 个不同的数据元素进行直接插入排序，最多需要进行的比较次数是（ ）。

A. 8　　B. 10　　C. 15　　D. 25

02. 在待排序的元素序列基本有序的前提下，效率最高的排序算法是（ ）。

A. 直接插入排序　B. 简单选择排序　C. 快速排序　D. 归并排序

03. 在图书馆中，计算机类书籍区共有 12 列书架，书架上的书都是按照编号排列好的，其

中有些书被读者放错了地方，但通常不超过一个书架。未来将这些书重新放回正确的位置，应该采用何种排序算法？（　）。

A. 堆排序　　B. 直接插入排序　　C. 归并排序　　D. 简单选择排序

04. 对有 n 个元素的顺序表采用直接插入排序算法进行排序，在最坏情况下所需的比较次数是（　），在最好情况下所需的比较次数是（　）。

A. $n-1$　　B. $n+1$　　C. $n/2$　　D. $n(n-1)/2$

05. 数据序列{8, 10, 13, 4, 6, 7, 22, 2, 3}只能是（　）两趟排序后的结果。

A. 简单选择排序　B. 冒泡排序　　C. 直接插入排序　D. 堆排序

06. 用直接插入排序算法对下列 4 个表进行（从小到大）排序，比较次数最少的是（　）。

A. 94, 32, 40, 90, 80, 46, 21, 69　　B. 21, 32, 46, 40, 80, 69, 90, 94

C. 32, 40, 21, 46, 69, 94, 90, 80　　D. 90, 69, 80, 46, 21, 32, 94, 40

07. 在下列算法中，（　）算法可能出现下列情况：在最后一趟开始之前，所有元素都不在最终位置上。

A. 堆排序　　B. 冒泡排序　　C. 直接插入排序　D. 快速排序

08. 希尔排序属于（　）。

A. 插入排序　　B. 交换排序　　C. 选择排序　　D. 归并排序

09. 对序列{15, 9, 7, 8, 20, −1, 4}采用希尔排序，经一趟后序列变为{15, −1, 4, 8, 20, 9, 7}，则该次采用的增量是（　）。

A. 1　　B. 4　　C. 3　　D. 2

10. 若序列{15, 9, 7, 8, 20, −1, 4}经一趟排序后变成{9, 15, 7, 8, 20, −1, 4}，则采用的是（　）方法。

A. 选择排序　　B. 快速排序　　C. 直接插入排序　D. 冒泡排序

11. 对序列{98, 36, −9, 0, 47, 23, 1, 8, 10, 7}采用希尔排序，下列序列（　）是增量为 4 的一趟排序结果。

A. {10, 7, −9, 0, 47, 23, 1, 8, 98, 36}　　B. {−9, 0, 36, 98, 1, 8, 23, 47, 7, 10}

C. {36, 98, −9, 0, 23, 47, 1, 8, 7, 10}　　D. 以上都不对

12. 对序列{E, A, S, Y, Q, U, E, S, T, I, O,N}按照字典顺序排序，采用增量 $d=6, 3, 1$ 的希尔排序算法。则前两趟排序后，关键字的总比较次数为（　）。

A. 15　　B. 17　　C. 16　　D. 18

13. 已知输入序列{13, 24, 7, 1, 8, 9, 11, 56, 34, 51, 2, 77, 5}，增量序列 $d=5, 3, 1$，采用希尔排序算法进行排序，则两趟排序后的结果为（　）。

A. 1, 7, 8, 9, 13, 24, 11, 34, 51, 2, 5, 56, 77

B. 1, 7, 5, 2, 8, 9, 24, 11, 34, 51, 13, 77, 56

C. 2, 11, 5, 1, 8, 9, 24, 7, 34, 51, 13, 77, 56

D. 2, 5, 11, 1, 8, 9, 7, 24, 34, 13, 51, 77, 56

14. 折半插入排序算法的时间复杂度为（　）。

A. $O(n)$　　B. $O(n\log_2 n)$　　C. $O(n^2)$　　D. $O(n^3)$

15. 有些排序算法在每趟排序过程中，都会有一个元素被放置到其最终位置上，（　）算法不会出现此种情况。

A. 希尔排序　　B. 堆排序　　C. 冒泡排序　　D. 快速排序

16. 以下排序算法中，不稳定的是（　）。

A. 冒泡排序　　B. 直接插入排序　　C. 希尔排序　　D. 归并排序

17. 以下排序算法中，稳定的是（　）。

A. 快速排序　　B. 堆排序　　C. 直接插入排序　　D. 简单选择排序

18. 【2012统考真题】对同一待排序序列分别进行折半插入排序和直接插入排序，两者之间可能的不同之处是（　）。

A. 排序的总趟数　　B. 元素的移动次数

C. 使用辅助空间的数量　　D. 元素之间的比较次数

19. 【2014统考真题】用希尔排序算法对一个数据序列进行排序时，若第一趟排序结果为9, 1, 4, 13, 7, 8, 20, 23, 15，则该趟排序采用的增量（间隔）可能是（　）。

A. 2　　B. 3　　C. 4　　D. 5

20. 【2015统考真题】希尔排序的组内排序采用的是（　）。

A. 直接插入排序　　B. 折半插入排序　　C. 快速排序　　D. 归并排序

21. 【2018统考真题】对初始数据序列(8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6)进行希尔排序。若第一趟排序结果为(1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8)，第二趟排序结果为(1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9)，则两趟排序采用的增量（间隔）依次是（　）。

A. 3, 1　　B. 3, 2　　C. 5, 2　　D. 5, 3

二、综合应用题

01. 给出关键字序列{4, 5, 1, 2, 6, 3}的直接插入排序过程。

02. 给出关键字序列{50, 26, 38, 80, 70, 90, 8, 30, 40, 20}的希尔排序过程（取增量序列为 $d=\{5, 3, 1\}$，排序结果为从小到大排列）。

8.2.5 答案与解析

一、单项选择题

01. B

直接插入排序在最坏的情况下要做 $n(n-1)/2$ 次关键字的比较，当 $n=5$ 时，关键字的比较次数为10。注意不考虑与哨兵的比较。

02. A

序列初始基本有序，因此使用直接插入排序算法的时间复杂度接近 $O(n)$，而使用其他算法的时间复杂度均大于 $O(n)$。

03. B

大部分图书都是有序的，因此采用直接插入排序比较合适。

04. D、A

待排序表为反序时，直接插入排序需要进行 $n(n-1)/2$ 次比较（从前往后依次需要比较1, 2, …, $n-1$ 次）；待排序表为正序时，只需进行 $n-1$ 次比较。注意本题不考虑与哨兵的比较。

05. C

冒泡排序和选择排序经过两趟排序后，应该有两个最大（或最小）元素放在其最终位置；插入排序经过两趟排序后，前三个元素应该是局部有序的。只可能是插入排序。

注　意

在排序过程中，每趟都能确定一个元素在其最终位置的有冒泡排序、简单选择排序、堆排序、快速排序，其中前三者能形成全局有序的子序列，后者能确定枢轴元素的最终位置。

06．B

越接近正序的序列，直接插入排序的比较次数就越少。选项 B 和 C 是比较接近正序的，然后分别判断两个序列的比较次数，以选项 B 为例：第一趟，插入 32，比较 1 次；第二趟，插入 46，比较 1 次；第三趟，插入 40，因为 40 比 46 小但比 32 大，所以比较 2 次；第四趟，插入 80，比较 1 次；第五趟，插入 69，比较 2 次；以此类推，共比较 9 次。同理求出选项 C 的比较次数为 11 次。所以选择选项 B。

07．C

在直接插入排序中，若待排序列中的最后一个元素应插入表中的第一个位置，则前面的有序子序列中的所有元素都不在最终位置上。

08．A

希尔排序是对直接插入排序算法改进后提出来的，本质上仍属于插入排序的范畴。

09．B

希尔排序将序列分成若干组，记录只在组内进行交换。由观察可知，经过一趟后 9 和−1 交换，7 和 4 交换，可知增量为 4。

10．C

前两个元素已经局部有序，很明显一趟直接插入排序算法有效。再排除其他算法即可。

11．A

增量为 4 意味着所有相距为 4 的记录构成一组，然后在组内进行直接插入排序，经观察，只有选项 A 满足要求。

12．B

第一趟：EE 为一组，比较；AS 为一组，比较；ST 为一组，比较；YI 为一组，比较后交换；QO 为一组，比较后交换；UN 为一组，比较后交换，结果为 EASIONESTYQU。第二趟：EIEY 为一组，用直接插入排序需要依次比较 I 和 E、E 和 I、E 和 E、Y 和 I；AOSQ 为一组，依次比较 O 和 A、S 和 O、Q 和 S、Q 和 O；SNTU 为一组，依次比较 N 和 S、T 和 S、U 和 T。第一趟比较次数为 6，第二趟比较次数为 11，总比较次数为 17。

13．B

第一趟增量 $d=5$，第一趟排序后，结果为 2, 11, 5, 1, 8, 9, 24, 7, 34, 51, 13, 77, 56。第二趟增量 $d=3$，第二趟排序后，结果为 1, 7, 5, 2, 8, 9, 24, 11, 34, 51, 13, 77, 56。

14．C

虽然折半插入排序是对直接插入排序的改进，但它改进的只是比较的次数，而移动次数未发生变化，时间复杂度仍为 $O(n^2)$。

15．A

因为希尔排序是基于插入排序算法提出的，所以它不一定在每趟排序过程后将某一元素放置到最终位置上。

16．C

希尔排序是一种复杂的插入排序算法，它是一种不稳定的排序算法。

17．C

基于插入、交换、选择的三类排序算法中，通常简单方法是稳定的（直接插入、折半插入、冒泡），但有一个例外就是简单选择，复杂方法都是不稳定的（希尔排序、快速排序、堆排序）。

18．D

折半插入排序与直接插入排序都将待插入元素插入前面的有序子表，区别是：确定当前记录在前面有序子表中的位置时，直接插入排序采用顺序查找法，而折半插入排序采用折半查找法。

排序的总趟数取决于元素个数 n，两者都是 $n-1$ 趟。元素的移动次数都取决于初始序列，两者相同。使用辅助空间的数量也都是 $O(1)$。折半插入排序的比较次数与序列初态无关，时间复杂度为 $O(n\log_2 n)$；而直接插入排序的比较次数与序列初态有关，时间复杂度为 $O(n)\sim O(n^2)$。

19．B

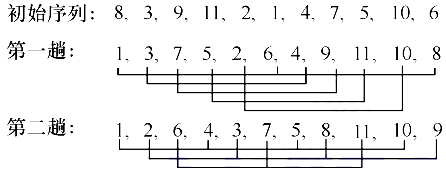
首先，第二个元素为 1，是整个序列中的最小元素，可知该希尔排序为从小到大排序。然后考虑增量问题，若增量为 2，则第 1 + 2 个元素 4 明显比第 1 个元素 9 要小，排除选项 A。若增量为 3，则第 $i, i+3, i+6$（$i=1, 2, 3$）个元素都为有序序列，符合希尔排序的特点。若增量为 4，则第 1 个元素 9 比第 1 + 4 个元素 7 要大，排除选项 C。若增量为 5，则第 1 个元素 9 比第 1 + 5 个元素 8 要大，排除选项 D。

20．A

希尔排序的思想是：先将待排元素序列分割成若干子序列（由相隔某个“增量”的元素组成），分别进行直接插入排序，然后依次缩减增量再进行排序，待整个序列中的元素基本有序（增量足够小）时，再对全体元素进行一次直接插入排序。

21．D

如下图所示。



第一趟分组：8, 1, 6；3, 4；9, 7；11, 5；2, 10；间隔为 5，排序后组内递增。

第二趟分组：1, 5, 4, 10；3, 2, 9, 8；7, 6, 11；间隔为 3，排序后组内递增。

因此，选择选项 D。

二、综合应用题

01.【解答】

直接插入排序过程如下。

初始序列：　4, 5, 1, 2, 6, 3

第一趟：　4, 5, 1, 2, 6, 3　（将 5 插入{4}）

第二趟：　1, 4, 5, 2, 6, 3　（将 1 插入{4, 5}）

第三趟：　1, 2, 4, 5, 6, 3　（将 2 插入{1, 4, 5}）

第四趟：　1, 2, 4, 5, 6, 3　（将 6 插入{1, 2, 4, 5}）

第五趟：　1, 2, 3, 4, 5, 6　（将 3 插入{1, 2, 4, 5, 6}）

02.【解答】

原始序列：　50, 26, 38, 80, 70, 90, 8, 30, 40, 20

第一趟（增量 5）：　50, 8, 30, 40, 20, 90, 26, 38, 80, 70

第二趟（增量 3）：　26, 8, 30, 40, 20, 80, 50, 38, 90, 70

第三趟（增量 1）：　8, 20, 26, 30, 38, 40, 50, 70, 80, 90

8.3　交换排序

所谓交换，是指根据序列中两个元素关键字的比较结果来对换这两个记录在序列中的位置。

基于交换的排序算法很多，本书主要介绍冒泡排序和快速排序，其中冒泡排序算法比较简单，一般很少直接考查，通常会重点考查快速排序算法的相关内容。

8.3.1　冒泡排序

冒泡排序的基本思想是：从后往前（或从前往后）两两比较相邻元素的值，若为逆序（A[i-1]>A[i]），则交换它们，直到序列比较完。我们称它为第一趟冒泡，结果是将最小的元素交换到待排序列的第一个位置（或将最大的元素交换到待排序列的最后一个位置），关键字最小的元素如气泡一般逐渐往上“漂浮”至“水面”（或关键字最大的元素如石头一般下沉至水底）。下一趟冒泡时，前一趟确定的最小元素不再参与比较，每趟冒泡的结果是把序列中的最小元素（或最大元素）放到了序列的最终位置……这样最多做 $n-1$ 趟冒泡就能把所有元素排好序。

图 8.3 所示为冒泡排序的过程，第一趟冒泡时：27 < 49，不交换；13 < 27，不交换；76 > 13，交换；97 > 13，交换；65 > 13，交换；38 > 13，交换；49 > 13，交换。通过第一趟冒泡后，最小元素已交换到第一个位置，也是它的最终位置。第二趟冒泡时对剩余子序列采用同样方法进行排序，如此重复，到第五趟结束后没有发生交换，说明表已有序，冒泡排序结束。

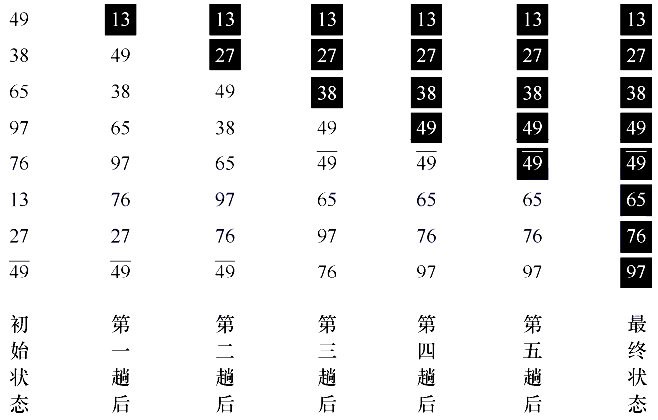


图 8.3　冒泡排序示例

冒泡排序算法的代码如下：

```
void BubbleSort(ElemType A[],int n){
    for(int i=0;i<n-1;i++){
        bool flag=false;            //表示本趟冒泡是否发生交换的标志
        for(int j=n-1;j>i;j--)      //一趟冒泡过程
            if(A[j-1]>A[j]){        //若为逆序
                swap(A[j-1],A[j]);  //使用封装的 swap 函数交换①
                flag=true;
            }
        if(flag==false)
            return;                 //本趟遍历后没有发生交换，说明表已经有序
    }
}
```

冒泡排序的性能分析如下：

空间效率：仅使用了常数个辅助单元，因而空间复杂度为 $O(1)$。

时间效率：当初始序列有序时，显然第一趟冒泡后 flag 依然为 false（本趟没有元素交换），

① 本章有多处要两两交换元素，为使代码更简洁，使用函数 swap(int &a, int &b)，其代码如下：
int temp=a; a=b; b=temp;

从而直接跳出循环，比较次数为 $n-1$，移动次数为 0，从而最好情况下的时间复杂度为 $O(n)$；当初始序列为逆序时，需要进行 $n-1$ 趟排序，第 i 趟排序要进行 $n-i$ 次关键字的比较，而且每次比较后都必须移动元素 3 次来交换元素位置。这种情况下，

$$\text{比较次数}=\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2},\qquad \text{移动次数}=\sum_{i=1}^{n-1}3(n-i)=\frac{3n(n-1)}{2}$$

从而，最坏情况下的时间复杂度为 $O(n^2)$，平均时间复杂度为 $O(n^2)$。

稳定性：$i>j$ 且 A[i]=A[j]时，不会发生交换，因此冒泡排序是一种稳定的排序算法。

适用性：冒泡排序适用于顺序存储和链式存储的线性表。

注　意

冒泡排序中所产生的有序子序列一定是全局有序的（不同于直接插入排序），也就是说，有序子序列中的所有元素的关键字一定小于（或大于）无序子序列中所有元素的关键字，这样每趟排序都会将一个元素放置到其最终的位置上。

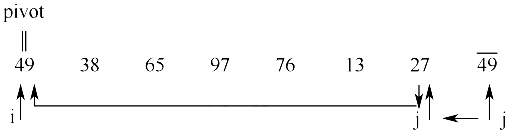
8.3.2 快速排序

命题追踪 ▶ 快速排序的思想（2024）

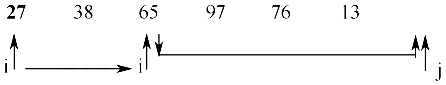
快速排序的基本思想：在待排序表 L[1…n]中任取一个元素 pivot 作为枢轴（或称基准，通常取首元素），通过一趟排序将待排序表划分为独立的两部分 L[1…k-1]和 L[k+1…n]，使得 L[1…k-1]中的所有元素小于 pivot，L[k+1…n]中的所有元素大于或等于 pivot，则 pivot 放在了其最终位置 L(k)上，这个过程称为一次划分。然后分别递归地对两个子表重复上述过程，直至每部分内只有一个元素或为空为止，即所有元素放在了其最终位置上。

一趟快速排序的过程是一个交替搜索和交换的过程，下面通过实例来介绍，附设两个指针 i 和 j，初值分别为 low 和 high，取第一个元素 49 为枢轴赋值到变量 pivot。

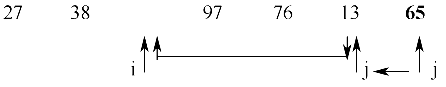
指针 j 从 high 往前搜索找到第一个小于枢轴的元素 27，将 27 交换到 i 所指位置。



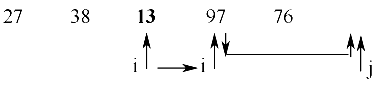
指针 i 从 low 往后搜索找到第一个大于枢轴的元素 65，将 65 交换到 j 所指位置。



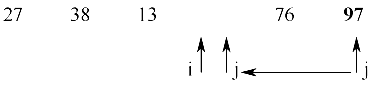
指针 j 继续往前搜索找到小于枢轴的元素 13，将 13 交换到 i 所指位置。



指针 i 继续往后搜索找到大于枢轴的元素 97，将 97 交换到 j 所指位置。



指针 j 继续往前搜索小于枢轴的元素，直至 i==j。



命题追踪 ▶ 快速排序的中间过程的分析（2014、2019、2023）

此时，指针 i（==j）之前的元素均小于 49，指针 i 之后的元素均大于或等于 49，将 49 放在 *i* 所指位置即其最终位置，经过第一趟排序后，将原序列分割成了前后两个子序列。

第一趟后：　{27　38　13}　**49**　{76　97　65　$\overline{49}$}

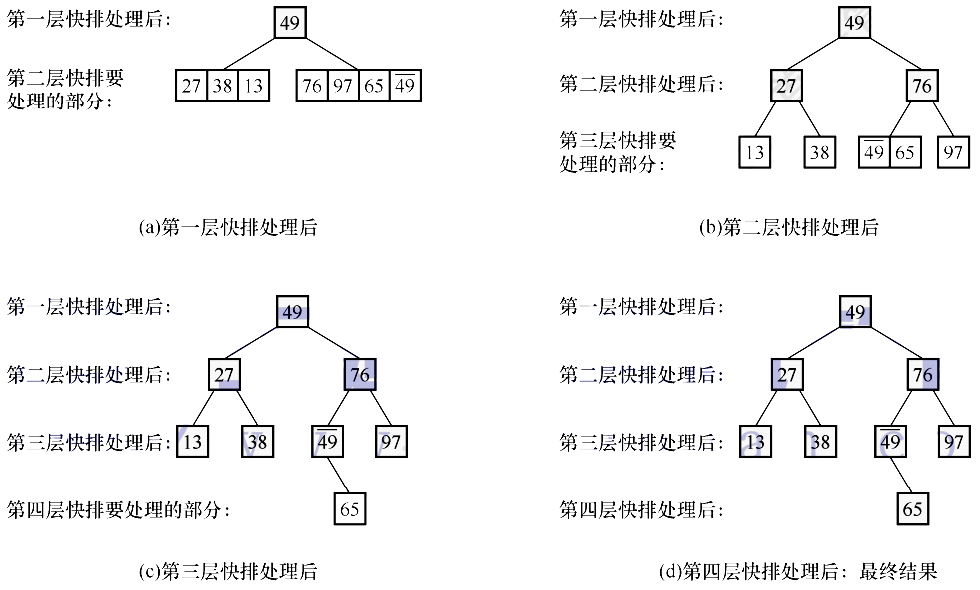
按照同样的方法对各子序列进行快速排序，若待排序列中只有一个元素，显然已有序。

第二趟后：　{13}　**27**　{38}　**49**　{$\overline{49}$　65}　**76**　{97}

第三趟后：　**13**　**27**　**38**　**49**　**$\overline{49}$**　{65}　**76**　**97**

第四趟后：　**13**　**27**　**38**　**49**　**$\overline{49}$**　**65**　**76**　**97**

用二叉树的形式描述这个快速排序示例的递归调用过程，如图 8.4 所示。



(a)第一层快排处理后　(b)第二层快排处理后

(c)第三层快排处理后　(d)第四层快排处理后：最终结果

图 8.4　快速排序的递归执行过程

假设划分算法已知，记为 Partition()，返回的是上述的 k，则 L(k) 已放在其最终位置。因此可以先对表进行划分，然后对两个子表递归地调用快速排序算法进行排序。代码如下：

```
void QuickSort(ElemType A[],int low,int high){
    if(low<high){                                    //递归跳出的条件
     //Partition()就是划分操作，将表A[low…high]划分为满足上述条件的两个子表
        int pivotpos=Partition(A,low,high); //划分
        QuickSort(A,low,pivotpos-1);//依次对两个子表进行递归排序
        QuickSort(A,pivotpos+1,high);
    }
}
```

命题追踪 ▶ （算法题）快速排序中划分操作的应用（2016）

快速排序算法的性能主要取决于划分操作的好坏。考研所考查的快速排序的划分操作通常总以表中第一个元素作为枢轴来对表进行划分，则将表中比枢轴大的元素向右移动，将比枢轴小的元素向左移动，使得一趟 Partition() 操作后，表中的元素被枢轴一分为二。代码如下：

```
int Partition(ElemType A[],int low,int high){  //一趟划分
    ElemType pivot=A[low];  //将当前表中第一个元素设为枢轴，对表进行划分
```

```
    while(low<high){      //循环跳出条件
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];  //将比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];  //将比枢轴大的元素移动到右端
    }
    A[low]=pivot;              //枢轴元素存放到最终位置
    return  low;               //返回存放枢轴的最终位置
}
```

快速排序算法的性能分析如下：

命题追踪 ▶ 快速排序中递归次数的影响因素分析（2010）

空间效率：快速排序是递归的，因此需要借助一个递归工作栈来保存每层递归调用的必要信息，其容量与递归调用的最大层数一致。最好情况下为 $O(\log_2 n)$；最坏情况下，要进行 $n-1$ 次递归调用，因此栈的深度为 $O(n)$；平均情况下，栈的深度为 $O(\log_2 n)$。

时间效率：快速排序的运行时间与划分是否对称有关，快速排序的最坏情况发生在两个区域分别包含 $n-1$ 个元素和 0 个元素时，这种最大限度的不对称性若发生在每层递归上，即对应于初始排序表基本有序或基本逆序时，就得到最坏情况下的时间复杂度为 $O(n^2)$。

有很多方法可以提高算法的效率：一种方法是尽量选取一个可以将数据中分的枢轴元素，如从序列的头尾及中间选取三个元素，再取这三个元素的中间值作为最终的枢轴元素；或者随机地从当前表中选取枢轴元素，这样做可使得最坏情况在实际排序中几乎不会发生。

在最理想的状态下，即 Partition() 能做到最平衡的划分，得到的两个子问题的大小都不可能大于 $n/2$，在这种情况下，快速排序的运行速度将大大提升，此时，时间复杂度为 $O(n\log_2 n)$。好在快速排序平均情况下的运行时间与其最佳情况下的运行时间很接近，而不是接近其最坏情况下的运行时间。快速排序是所有内部排序算法中平均性能最优的排序算法。

稳定性：在划分算法中，若右端区间有两个关键字相同，且均小于基准值的记录，则在交换到左端区间后，它们的相对位置会发生变化，即快速排序是一种不稳定的排序算法。例如，表 $L=\{3, \mathbf{2}, 2\}$，经过一趟排序后 $L=\{2, \mathbf{2}, 3\}$，最终排序序列也是 $L=\{2, \mathbf{2}, 3\}$，显然，2 与 **2** 的相对次序已发生了变化。

命题追踪 ▶ 快速排序适合采用的存储方式（2011）

适用性：快速排序仅适用于顺序存储的线性表。

注 意

在快速排序算法中，并不产生有序子序列，但每一趟排序后会将上一趟划分的各个无序子表的枢轴（基准）元素放到其最终的位置上。

8.3.3 本节试题精选

一、单项选择题

01. 对 n 个不同的元素利用冒泡法从小到大排序，在（ ）情况下元素交换的次数最多。

A. 从大到小排列好的　　B. 从小到大排列好的

C. 元素无序　　D. 元素基本有序

02. 若用冒泡排序算法对序列{10, 14, 26, 29, 41, 52}从大到小排序，则需进行（ ）次比较。

A. 3　　B. 10　　C. 15　　D. 25

03. 用某种排序算法对线性表{25, 84, 21, 47, 15, 27, 68, 35, 20}进行排序时，元素序列的变化情况如下：

1）25, 84, 21, 47, 15, 27, 68, 35, 20

2）20, 15, 21, 25, 47, 27, 68, 35, 84

3）15, 20, 21, 25, 35, 27, 47, 68, 84

4）15, 20, 21, 25, 27, 35, 47, 68, 84

则所采用的排序算法是（　）。

A. 选择排序　　B. 插入排序　　C. 二路归并排序　　D. 快速排序

04. 一组记录的关键码为(46, 79, 56, 38, 40, 84)，则利用快速排序算法，以第一个记录为基准，从小到大得到的一次划分结果为（　）。

A. (38, 40, 46, 56, 79, 84)　　B. (40, 38, 46, 79, 56, 84)

C. (40, 38, 46, 56, 79, 84)　　D. (40, 38, 46, 84, 56, 79)

05. 快速排序算法在（　）情况下最不利于发挥其长处。

A. 要排序的数据量太大　　B. 要排序的数据中含有多个相同值

C. 要排序的数据个数为奇数　　D. 要排序的数据已基本有序

06. 就平均性能而言，目前最好的内部排序算法是（　）。

A. 冒泡排序　　B. 直接插入排序　　C. 希尔排序　　D. 快速排序

07. 数据序列 F = {2, 1, 4, 9, 8, 10, 6, 20}只能是下列排序算法中的（　）两趟排序后的结果。

A. 快速排序　　B. 冒泡排序　　C. 选择排序　　D. 插入排序

08. 对元素序列{8, 9, 10, 4, 5, 6, 20, 1, 2}采用冒泡排序（从后往前次序进行，要求升序），需要进行元素交换的趟数至少是（　）（不考虑无元素交换的最后一趟）。

A. 3　　B. 4　　C. 5　　D. 8

09. 双向冒泡排序是指对一个序列在正反两个方向交替进行扫描，第一趟把最大值放在序列的最右端，第二趟把最小值放在序列的最左端，之后在缩小的范围内进行同样的扫描，放在次右端、次左端，直至序列有序。对数组{4, 7, 8, 3, 5, 6, 10, 9, 1, 2}进行双向冒泡排序，则排序趟数是（　）。（第一趟从左往右开始，从左往右或从右往左都称为一趟。）

A. 7　　B. 6　　C. 8　　D. 9

10. 对下列关键字序列用快速排序进行排序时，每次选取的基准元素都为待处理序列的第一个元素，速度最快的情形是（　），速度最慢的情形是（　）。

A. {21, 25, 5, 17, 9, 23, 30}　　B. {25, 23, 30, 17, 21, 5, 9}

C. {21, 9, 17, 30, 25, 23, 5}　　D. {5, 9, 17, 21, 23, 25, 30}

11. 对下列 4 个序列，以第一个关键字为基准用快速排序算法进行排序，在第一趟过程中移动记录次数最多的是（　）。

A. 92, 96, 88, 42, 30, 35, 110, 100　　B. 92, 96, 100, 110, 42, 35, 30, 88

C. 100, 96, 92, 35, 30, 110, 88, 42　　D. 42, 30, 35, 92, 100, 96, 88, 110

12. 下列序列中，（　）可能是执行第一趟快速排序后所得到的序列（按从大到小排序和从小到大排序来分别讨论）。

I. {68, 11, 18, 69, 23, 93, 73}　　II. {68, 11, 69, 23, 18, 93, 73}

III. {93, 73, 68, 11, 69, 23, 18}　　IV. {68, 11, 69, 23, 18, 73, 93}

A. I、IV　　B. II、III　　C. III、IV　　D. 只有 IV

13. 对 n 个关键字进行快速排序，最大递归深度为（　），最小递归深度为（　）。

A. 1　　B. n　　C. $\log_2 n$　　D. $n\log_2 n$

14. 对8个元素的序列进行快速排序，在最好情况下的关键字比较次数是（　）。

A. 7　　B. 8　　C. 12　　D. 13

15. 【2010统考真题】采用递归方式对顺序表进行快速排序。下列关于递归次数的叙述中，正确的是（　）。

A. 递归次数与初始数据的排列次序无关

B. 每次划分后，先处理较长的分区可以减少递归次数

C. 每次划分后，先处理较短的分区可以减少递归次数

D. 递归次数与每次划分后得到的分区的处理顺序无关

16. 【2011统考真题】为实现快速排序算法，待排序序列宜采用的存储方式是（　）。

A. 顺序存储　　B. 散列存储　　C. 链式存储　　D. 索引存储

17. 【2014统考真题】下列选项中，不可能是快速排序第二趟排序结果的是（　）。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9

C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

18. 【2019统考真题】排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一趟。下列序列中，不可能是快速排序第二趟结果的是（　）。

A. 5, 2, 16, 12, 28, 60, 32, 72　　B. 2, 16, 5, 28, 12, 60, 32, 72

C. 2, 12, 16, 5, 28, 32, 72, 60　　D. 5, 2, 12, 28, 16, 32, 72, 60

19. 【2023统考真题】使用快速排序算法对数据进行升序排序，若经过一次划分后得到的数据序列是68, 11, 70, 23, 80, 77, 48, 81, 93, 88，则该次划分的枢轴是（　）。

A. 11　　B. 70　　C. 80　　D. 81

20. 【2024统考真题】使用快速排序算法对含n（$n\geqslant 3$）个元素的数组M进行排序，若第一趟排序将M中除枢轴外的$n-1$个元素划分为均不为空的P和Q两块，则下列叙述中，正确的是（　）。

A. P与Q块间有序　　B. P与Q均块内有序

C. P和Q的元素个数大致相等　　D. P中和Q中均不存在相等的元素

二、综合应用题

01. 已知线性表按顺序存储，且每个元素都是不相同的整数型元素，设计把所有奇数移动到所有偶数前边的算法（要求时间最短，辅助空间最小）。

02. 试编写一个算法，使之能够在数组L[1…n]中找出第k小的元素（从小到大排序后处于第k个位置的元素）。

03. 荷兰国旗问题：设有一个仅由红、白、蓝三种颜色的条块组成的条块序列，存储在一个顺序表中，请编写一个时间复杂度为$O(n)$的算法，使得这些条块按红、白、蓝的顺序排好，即排成荷兰国旗图案。请完成算法实现：

```
typedef  enum{RED, WHITE, BLUE} color;        //设置枚举数组
void Flag_Arrange(color a[],int n){ … }
```

04. 【2016统考真题】已知由n（$n\geqslant 2$）个正整数构成的集合$A=\{a_k|0\leqslant k<n\}$，将其划分为两个不相交的子集A_1和A_2，元素个数分别是n_1和n_2，A_1和A_2中的元素之和分别为S_1和S_2。设计一个尽可能高效的划分算法，满足$|n_1-n_2|$最小且$|S_1-S_2|$最大。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用C或C++语言描述算法，关键之处给出注释。

3）说明所设计算法的平均时间复杂度和空间复杂度。

8.3.4　答案与解析

一、单项选择题

01．A

冒泡排序最少进行 1 趟冒泡，最多进行 $n-1$ 趟冒泡。初始序列为逆序时，需进行 $n-1$ 趟冒泡，并且元素交换的次数最多。初始序列为正序时，进行 1 趟冒泡（无交换）就可结束算法。

02．C

冒泡排序始终在调整“逆序”，因此交换次数为排列中逆序的个数。对逆序序列进行冒泡排序，每个元素向后调整时都需要进行比较，因此共需要比较 $5+4+3+2+1=15$ 次。

03．D

选择排序在每趟结束后可以确定一个元素的最终位置，不对。插入排序，第 i 趟后前 $i+1$ 个元素应该是有序的，不对。第二趟{20, 15}和{21, 25}是反序的，因此不是归并排序。快速排序每趟都将基准元素放在其最终位置，然后以它为基准将序列划分为两个子序列。观察题中的排序过程，可知是快速排序。

04．C

以 46 为基准元素，首先从后往前扫描比 46 小的元素，并与之进行交换，而后从前往后扫描比 46 大的元素并将 46 与该元素交换，得到(40, 46, 56, 38, 79, 84)。此后，继续重复从后往前扫描与从前往后扫描的操作，直到 46 处于最终位置。

05．D

当待排序数据为基本有序时，每次选取第 n 个元素为基准，会导致划分区间分配不均匀，不利于发挥快速排序算法的优势。相反，当待排序数据分布较为随机时，基准元素能将序列划分为两个长度大致相等的序列，这时才能发挥快速排序的优势。

06．D

这里问的是平均性能，选项 A、B 的平均性能都会达到 $O(n^2)$，而希尔排序虽然大大降低了直接插入排序的时间复杂度，但其平均性能不如快速排序。另外，虽然众多排序算法的平均时间复杂度也是 $O(n\log_2 n)$，但快速排序算法的常数因子是最小的。

07．A

若为插入排序，则前三个元素应该是有序的，显然不对。而冒泡排序和选择排序经过两趟排序后应该有两个元素处于最终位置（最左/右端），无论是按从小到大还是从大到小排序，数据序列中都没有两个满足这样的条件的元素，因此只可能选择选项 A。

【另解】先写出排好序的序列，并和题中的序列做对比。

题中序列：2　1　4　9　8　10　6　20

已排好序序列：1　2　4　6　8　9　10　20

在已排好序的序列中，与题中序列相同的元素有 4、8 和 20，最左和最右两个元素与题中的序列不同，所以不可能是冒泡排序、选择排序或插入排序。

08．C

从后往前冒泡的过程为，第一趟{1, 8, 9, 10, 4, 5, 6, 20, 2}，第二趟{1, 2, 8, 9, 10, 4, 5, 6, 20}，第三趟{1, 2, 4, 8, 9, 10, 5, 6, 20}，第四趟{1, 2, 4, 5, 8, 9, 10, 6, 20}，第五趟{1, 2, 4, 5, 6, 8, 9, 10, 20}，经过第五趟冒泡后，序列已经全局有序，因此选择选项 C。实际每趟冒泡发生交换后可以判断是否会产生新的逆序对，若不会产生，则本趟冒泡之后序列全局有序，所以最少 5 趟即可。

09．B

第一趟从左往右的排序结果为4, 7, 3, 5, 6, 8, 9, 1, 2, 10；第二趟从右往左的排序结果为1, 4,7, 3, 5, 6, 8, 9, 2, 10；第三趟从左往右的排序结果为1, 4, 3, 5, 6, 7, 8, 2, 9, 10；第四趟从右往左的排序结果为1, 2, 4, 3, 5, 6, 7, 8, 9, 10；第五趟从左往右的排序结果为1, 2, 3, 4, 5, 6, 7, 8, 9, 10，此时序列已有序，但仍需进行一趟无交换的排序才能确定序列已有序，因此共需6趟排序。

10．A、D

当每趟的枢轴值都把表等分为长度相近的两个子表时，速度是最快的；当表本身已经有序或逆序时，速度最慢。选项D中的序列已按关键字排好序，因此它是最慢的，而选项A中第一趟枢轴值21将表划分为两个子表{9, 17, 5}和{25, 23, 30}，而后对两个子表划分时，枢轴值再次将它们等分，所以该序列是快速排序最优的情况，速度最快。针对其他选项，可以进行类似的分析。

11．B

对各序列分别执行一趟快速排序，可做如下分析（以选项A为例）：枢轴值为92，因此35移动到第一个位置，96移动到第六个位置，30移动到第二个位置，再将枢轴值移动到30所在的单元，即第五个位置，所以选项A中序列移动的次数为4。同样，可以分析出选项B中序列的移动次数为8，选项C中序列的移动次数为4，选项D中序列的移动次数为2。

12．C

显然，若按从小到大排序，则最终有序的序列是{11, 18, 23, 68, 69, 73, 93}；若按从大到小排序，则最终有序的序列是{93, 73, 69, 68, 23, 18, 11}。对比可知选项I、II中没有处于最终位置的元素，所以选项I、II都不可能。选项III中73和93处于从大到小排序后的最终位置，而且73将序列分割成大于73和小于73的两部分，所以选项III是有可能的。选项IV中73和93处于从小到大排列后的最终位置，73也将序列分割成大于73和小于73的两部分。

13．B、C

快速排序过程构成一个递归树，递归深度即递归树的高度。枢轴值每次都将子表等分时，递归树的高为$\log_2 n$；枢轴值每次都是子表的最大值或最小值时，递归树退化为单链表，树高为n。

14．D

快速排序的最好情况是每次划分将待排序列划分为等长的两部分。因此，第一趟将第1个元素与后面的7个元素进行比较，将原序列划分为长度为3和4的两个子表，比较7次；第二趟对两个子表进行划分，将长度为3的子表划分为长度为1的两个子表（不用继续划分），比较2次，将长度为4的子表划分为长度为1和2的两个子表，比较3次；第三趟将长度为2的子表划分为长度为1的子表，比较1次。至此，排序结束，共进行的比较次数是7 + 2 + 3 + 1 = 13。

15．D

快速排序的递归次数与元素的初始排列有关。若每次划分后分区比较平衡，则递归次数少；若划分后分区不平衡，则递归次数多。递归次数与分区处理顺序无关。

16．A

对于绝大部分内部排序而言，只适用于顺序存储结构。快速排序在排序的过程中，既要从后往前查找，也要从前往后查找，因此宜采用顺序存储。

17．C

对n个元素进行第一趟快速排序后，会确定一个基准元素，根据这个基准元素在数组中的位置，有两种情况：①基准元素在数组的首端或尾端，接下来对剩下的$n-1$个元素构成的子序列进行第二趟快速排序，再确定一个基准元素。这样，在两趟排序后就至少能确定两个元素的最终位

置，其中至少有一个元素是在数组的首端或尾端。②基准元素不在数组的首端或尾端，第二趟快速排序对基准元素划分开的两个子序列分别进行一次划分，两个子序列各确定一个基准元素。这样，两趟排序后就至少能确定三个元素的最终位置。基于上述结论，观察题中的四个选项，选项 A 的 2, 3, 6, 7, 9 符合第一种或第二种情况；选项 B 中 2, 9 符合第一种情况；选项 D 中 5, 9 符合第一种情况；最后看选项 C，只有 9 处于最终位置，因此不可能是快速排序第二趟的结果。

18．D

基于上题中分析得出的结论，观察题中的四个选项，选项 A 的 28, 72 符合第一种情况；选项 B 的 2, 72 符合第一种情况；选项 C 的 2, 28, 32 符合第一种或第二种情况；最后看选项 D，只有 12 和 32 处于最终位置，既不符合第一种情况，又不符合第二种情况。

19．D

第一趟划分后得到的序列中只有一个枢轴，因此可将当前序列和最终排好序的序列进行比较，如下表所示。枢轴会出现在两个序列的相同位置，可以看出枢轴只可能是 77、81，选项只有 81。在当前序列中，77 左边有比它大的元素 80，因此 77 不是枢轴；而 81 左边都是比它小的元素，右边都是比它大的元素，因此 81 是枢轴。

当前序列	68	11	70	23	80	77	48	81	93	88
最终序列	11	23	48	68	70	77	80	81	88	93

20．A

依题意，对数组 M 进行第一趟快速排序后，会将 M 划分为三部分：小于枢轴值的块 P，枢轴元素，大于枢轴值的块 Q。P 和 Q 块内都是无序的，但是 P 块内的数据≤枢轴值，Q 块内的数据≥枢轴值，因此 P 块内的数据≤Q 块内的数据，即 P 与 Q 块间有序。

二、综合应用题

01.【解答】

本题可采用基于快速排序的划分思想来设计算法，只需遍历一次即可，其时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。假设表为 `L[1…n]`，基本思想是：先从前往后找到一个偶数元素 `L(i)`，再从后往前找到一个奇数元素 `L(j)`，将二者交换；重复上述过程直到 `i` 大于 `j`。

算法的实现如下：

```
void move(ElemType A[],int len){
//对表 A 按奇偶进行一趟划分
    int i=0,j=len-1;     //i 表示左端偶数元素的下标；j 表示右端奇数元素的下标
    while(i<j){
        while(i<j&&A[i]%2!=0) i++;  //从前往后找到一个偶数元素
        while(i<j&&A[j]%2!=1) j--;  //从后往前找到一个奇数元素
        if (i<j){
            Swap(A[i],A[j]);        //交换这两个元素
        i++; j--;
        }
    }
}
```

02.【解答】

显然，本题最直接的做法是用排序算法对数组先进行从小到大的排序，然后直接提取 `L(k)` 便得到了第 k 小的元素，但其平均时间复杂度将达到 $O(n\log_2 n)$以上。此外，还可采用小顶堆的方法，每次堆顶元素都是最小值元素，时间复杂度为 $O(n + k\log_2 n)$。下面介绍一个更精彩的算法，它基于快速排序的划分操作。

这个算法的主要思想如下：从数组 `L[1…n]`中选择枢轴 `pivot`（随机或直接取第一个）进行和

快速排序一样的划分操作后，表L[1…n]被划分为L[1…m-1]和L[m+1…n]，其中L(m)=pivot。

讨论m与k的大小关系：

1）当$m=k$时，显然pivot就是所要寻找的元素，直接返回pivot即可。

2）当$m<k$时，所要寻找的元素一定落在L[m+1…n]中，因此可对L[m+1…n]递归地查找第$k-m$小的元素。

3）当$m>k$时，所要寻找的元素一定落在L[1…m-1]中，因此可对L[1…m-1]递归地查找第k小的元素。

该算法的时间复杂度在平均情况下可以达到$O(n)$，而所占空间的复杂度则取决于划分的方法。算法的实现如下：

```
int kth_elem(int a[],int low,int high,int k){
    int pivot=a[low];
    int low_temp=low;          //由于下面会修改low与high，在递归时又要用到它们
    int high_temp=high;
    while(low<high){
        while(low<high&&a[high]>=pivot)
            --high;
        a[low]=a[high];
        while(low<high&&a[low]<=pivot)
            ++low;
        a[high]=a[low];
    }
    a[low]=pivot;
    //上面为快速排序中的划分算法
    //以下是本算法思想中所述的内容
    if(low==k)                 //由于与k相同，直接返回pivot元素
        return  a[low];
    else if(low>k)             //在前一部分表中递归寻找
        return  kth_elem(a,low_temp,low-1,k);
    else                       //在后一部分表中递归寻找
        return  kth_elem(a,low+1,high_temp,k);
}
```

03.【解答】

算法思想：顺序扫描线性表，将红色条块交换到线性表的最前面，蓝色条块交换到线性表的最后面。为此，设立三个指针，其中，j为工作指针，表示当前扫描的元素，i以前的元素全部为红色，k以后的元素全部为蓝色。根据j所指示元素的颜色，决定将其交换到序列的前部或尾部。初始时$i=0$，$k=n-1$，算法的实现如下：

```
typedef  enum{RED,WHITE,BLUE} color; //设置枚举数组
void Flag_Arrange(color a[],int n){
    int i=0,j=0,k=n-1;
    while(j<=k)
    switch(a[j]){//判断条块的颜色
        case  RED: Swap(a[i],a[j]);i++;j++;break;
        //红色，则和i交换
        case  WHITE: j++;break;
        case  BLUE: Swap(a[j],a[k]);k--;
        //蓝色，则和k交换
        //这里没有j++语句，以防止交换后a[j]仍为蓝色
    }
}
```

例如，将元素值正数、负数和零排序为前面都是负数，接着是0，最后是正数，也用同样的

方法。思考：为什么 case RED 语句不用考虑交换后 a[j]仍为红色，而 case BLUE 语句中却需要考虑交换后 a[j]仍为蓝色？

04.【解答】

1）算法的基本设计思想

由题意可知，将最小的$\lfloor n/2 \rfloor$个元素放在 A_1 中，其余的元素放在 A_2 中，分组结果即可满足题目要求。仿照快速排序的思想，基于枢轴将 n 个整数划分为两个子集。根据划分后枢轴所处的位置 i 分别处理：

① 若 $i=\lfloor n/2 \rfloor$，则分组完成，算法结束。

② 若 $i<\lfloor n/2 \rfloor$，则枢轴及之前的所有元素均属于 A_1，继续对 i 之后的元素进行划分。

③ 若 $i>\lfloor n/2 \rfloor$，则枢轴及之后的所有元素均属于 A_2，继续对 i 之前的元素进行划分。

基于该设计思想实现的算法，无须对全部元素进行全排序，其平均时间复杂度是 $O(n)$，空间复杂度是 $O(1)$。

2）算法实现

```
int setPartition(int a[], int n){
    int pivotkey, low=0,low0=0,high=n-1,high0=n-1,flag=1,k=n/2,i;
    int s1=0,s2=0;
    while(flag) {
        pivotkey=a[low];                    //选择枢轴
        while(low<high) {                   //基于枢轴对数据进行划分
            while(low<high && a[high]>=pivotkey) --high;
            if(low!=high) a[low]=a[high];
            while(low<high && a[low]<=pivotkey) ++low;
            if(low!=high) a[high]=a[low];
        }                                           //end of while(low<high)
        a[low]=pivotkey;
        if(low==k-1)                        //若枢轴是第 n/2 小的元素，划分成功
            flag=0;
        else{                               //是否继续划分
            if(low<k-1){
                low0=++low;
                high=high0;
            }
            else{
                high0=--high;
                low=low0;
            }
        }
    }
    for(i=0;i<k;i++) s1+=a[i];
    for(i=k;i<n;i++) s2+=a[i];
    return s2-s1;
}
```

3）本算法的平均时间复杂度是 $O(n)$，空间复杂度是 $O(1)$。

8.4　选择排序

选择排序的基本思想是：每一趟（如第 i 趟）在后面 $n-i+1$（$i=1,2,\cdots,n-1$）个待排序元素中选取关键字最小的元素，作为有序子序列的第 i 个元素，直到第 $n-1$ 趟做完，待排序元素只

剩下 1 个，就不用再选。选择排序中的堆排序是历年统考考查的重点。

8.4.1　简单选择排序

根据上面选择排序的思想，可以很直观地得出简单选择排序算法的思想：假设排序表为 L[1…n]，第 i 趟排序即从 L[i…n]中选择关键字最小的元素与 L(i)交换，每一趟排序可以确定一个元素的最终位置，这样经过 $n-1$ 趟排序就可使得整个排序表有序。

简单选择排序算法的过程较为简单，相关举例见本书配套课程，其代码如下：

```
void SelectSort(ElemType A[],int n){
    for(int i=0;i<n-1;i++){                  //一共进行 n-1 趟
        int min=i;                           //记录最小元素位置
        for(int j=i+1;j<n;j++)               //在 A[i…n-1]中选择最小的元素
            if(A[j]<A[min]) min=j;           //更新最小元素位置
        if(min!=i) swap(A[i],A[min]);        //封装的 swap()函数共移动元素 3 次
    }
}
```

简单选择排序算法的性能分析如下：

空间效率：仅使用常数个辅助单元，所以空间效率为 $O(1)$。

时间效率：从上述伪码中不难看出，在简单选择排序过程中，元素移动的操作次数很少，不会超过 $3(n-1)$次，最好的情况是移动 0 次，此时对应的表已经有序；但元素间比较的次数与序列的初始状态无关，始终是 $n(n-1)/2$ 次，因此时间复杂度始终是 $O(n^2)$。

稳定性：在第 i 趟找到最小元素后，和第 i 个元素交换，可能会导致第 i 个元素与含有相同关键字的元素的相对位置发生改变。例如，表 $L=\{2, \mathbf{2}, 1\}$，经过一趟排序后 $L=\{1, \mathbf{2}, 2\}$，最终排序序列也是 $L=\{1, \mathbf{2}, 2\}$，显然，2 与 **2** 的相对次序已发生变化。因此，简单选择排序是一种不稳定的排序算法。

适用性：简单选择排序适用于顺序存储和链式存储的线性表，以及关键字较少的情况。

8.4.2　堆排序

堆的定义如下，n 个关键字序列 L[1…n]称为堆，当且仅当该序列满足：

① L(i)≥L(2i)且 L(i)≥L(2i+1)　或

② L(i)≤L(2i)且 L(i)≤L(2i+1)　（$1 \leqslant i \leqslant \lfloor n/2 \rfloor$）

命题追踪　堆的性质与特点（2020）

可以将堆视为一棵完全二叉树，满足条件①的堆称为大根堆（大顶堆），大根堆的最大元素存放在根结点，且其任意一个非根结点的值小于或等于其双亲结点值。满足条件②的堆称为小根堆（小顶堆），小根堆的定义刚好相反，根结点是最小元素。图 8.5 所示为一个大根堆。

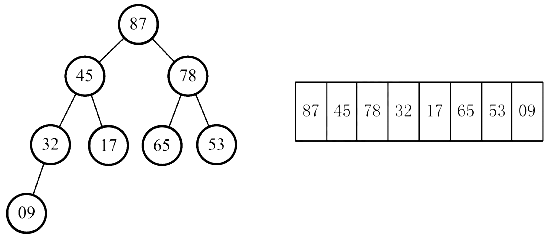


图 8.5　一个大根堆示意图

最新 408 统考大纲增加了考点“堆的应用”，堆的应用只有两个：堆排序和优先队列（见本节综合题 7），优先队列也利用了堆排序的思想，因此本节仅介绍堆排序。

堆排序的思路很简单：首先将存放在 L[1…n]中的 n 个元素建成初始堆，因为堆本身的特点（以大顶堆为例），所以堆顶元素就是最大值。输出堆顶元素后，通常将堆底元素送入堆顶，此时根结点已不满足大顶堆的性质，堆被破坏，将堆顶元素向下调整使其继续保持大顶堆的性质，再输出堆顶元素。如此重复，直到堆中仅剩一个元素为止。可见，堆排序需要解决两个问题：①如何将无序序列构造成初始堆？②输出堆顶元素后，如何将剩余元素调整成新的堆？

命题追踪 ▶ 初始建堆的操作（2018、2021）

堆排序的关键是构造初始堆。建堆思路是从后往前检查所有分支结点，看是否满足堆的要求，若不满□，则对以该分支结点为根的子树进行调整。n 个结点的完全二叉树，最后一个结点是第 $\lfloor n/2 \rfloor$ 个结点的孩子。对以第 $\lfloor n/2 \rfloor$ 个结点为根的子树筛选（对于大根堆，若根结点的关键字小于左右孩子中关键字较大者，则交换），使该子树成为堆。之后向前依次对以各结点（$\lfloor n/2 \rfloor - 1 \sim 1$）为根的子树进行筛选，看该结点值是否大于其左右子结点的值，若不大于，则将左右子结点中的较大值与之交换，交换后可能会破坏下一级的堆，于是继续采用上述方法构造下一级的堆，直到以该结点为根的子树构成堆为止。反复利用上述调整堆的方法建堆，直到根结点。

如图 8.6 所示，初始时调整 $L(4)$子树，09 < 32，交换，交换后满足堆的定义；向前继续调整 $L(3)$子树，78 < 左右孩子的较大者 87，交换，交换后满足堆的定义；向前调整 $L(2)$子树，17 < 左右孩子的较大者 45，交换后满足堆的定义；向前调整至根结点 $L(1)$，53 < 左右孩子的较大者 87，交换，交换后破坏了 $L(3)$子树的堆，采用上述方法对 $L(3)$进行调整，53 < 左右孩子的较大者 78，交换，至此该完全二叉树满足堆的定义。

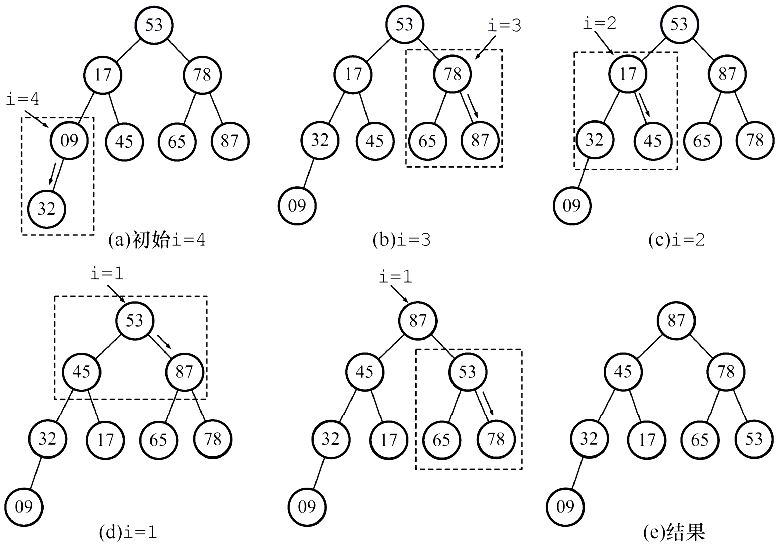


图 8.6　自下往上逐步调整为大根堆

命题追踪 ▶ 堆的删除操作及调整操作分析（2015、2024）

输出堆顶元素后，将堆的最后一个元素与堆顶元素交换，此时堆的性质被破坏，需要向下进行筛选。将 09 和左右孩子的较大者 78 交换，交换后破坏了 $L(3)$子树的堆，继续对 $L(3)$子树向下筛选，将 09 和左右孩子的较大者 65 交换，交换后得到了新堆，调整过程如图 8.7 所示。

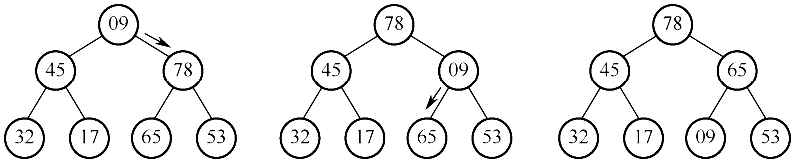


图 8.7 输出堆顶元素后再将剩余元素调整成新堆

下面是建立大根堆的算法：

```
void BuildMaxHeap(ElemType A[],int len){
    for(int i=len/2;i>0;i--)      //从 i=[n/2]～1，反复调整堆
        HeadAdjust(A,i,len);
}
void HeadAdjust(ElemType A[],int k,int len){
//函数 HeadAdjust 对以元素 k 为根的子树进行调整
    A[0]=A[k];                    //A[0]暂存子树的根结点
    for(int i=2*k;i<=len;i*=2){ //沿 key 较大的子结点向下筛选
        if(i<len&&A[i]<A[i+1])
            i++;                  //取 key 较大的子结点的下标
        if(A[0]>=A[i])  break;    //筛选结束
        else{
            A[k]=A[i];            //将 A[i]调整到双亲结点上
            k=i;                  //修改 k 值，以便继续向下筛选
        }
    }
    A[k]=A[0];                    //被筛选结点的值放入最终位置
}
```

调整的时间与树高有关，为 $O(h)$。在建含 n 个元素的堆时，关键字的比较总次数不超过 $4n$，时间复杂度为 $O(n)$，这说明可以在线性时间内将一个无序数组建成一个堆。

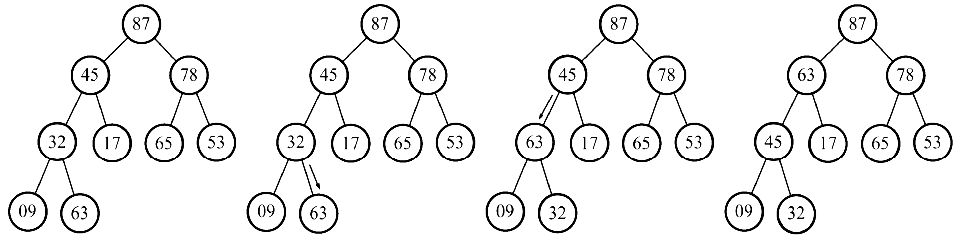
下面是堆排序算法：

```
void HeapSort(ElemType A[],int len){
    BuildMaxHeap(A,len);          //初始建堆
    for(int i=len;i>1;i--){       //n-1 趟的交换和建堆过程
        Swap(A[i],A[1]);          //输出堆顶元素（和堆底元素交换）
        HeadAdjust(A,1,i-1);      //调整，把剩余的 i-1 个元素整理成堆
    }
}
```

公众号：小兔网盘 免费分享无水印PDF

命题追踪 ▶ 堆的插入操作及调整操作分析（2009、2011）

同时，堆也支持插入操作。对堆进行插入操作时，先将新结点放在堆的末端，再对这个新结点向上执行调整操作。大根堆的插入操作示例如图 8.8 所示。



(a)初始，尾部加 63　(b)父结点关键字 32 下降　(c)父结点关键字 45 下降　(d)调整完成

图 8.8 大根堆的插入操作示例

命题追踪 ▶ 堆在海量数据中选出最小 k 个数的应用及效率分析（2022）

堆排序适合关键字较多的情况。例如，在 1 亿个数中选出前 100 个最大值。首先使用一个大小为 100 的数组，读入前 100 个数，建立小顶堆，而后依次读入余下的数，若小于堆顶则舍弃，否则用该数取代堆顶并重新调整堆，待数据读取完毕，堆中 100 个数为所求。

堆排序算法的性能分析如下：

空间效率：仅使用了常数个辅助单元，所以空间复杂度为 $O(1)$。

时间效率：建堆时间为 $O(n)$，之后有 $n-1$ 次向下调整操作，每次调整的时间复杂度为 $O(h)$，所以在最好、最坏和平均情况下，堆排序的时间复杂度为 $O(n\log_2 n)$。

稳定性：进行筛选时，有可能把后面相同关键字的元素调整到前面，所以堆排序算法是一种不稳定的排序算法。例如，表 $L=\{1, \mathbf{2}, 2\}$，构造初始堆时可能将 **2** 交换到堆顶，此时 $L=\{\mathbf{2}, 1, 2\}$，最终排序序列为 $L=\{1, 2, \mathbf{2}\}$，显然，2 与 **2** 的相对次序已发生变化。

适用性：堆排序仅适用于顺序存储的线性表。

8.4.3　本节试题精选

一、单项选择题

01. 在以下排序算法中，每次从未排序的记录中选取最小关键字的记录，加入已排序记录的末尾，该排序算法是（　）。

A. 简单选择排序　B. 冒泡排序　C. 堆排序　D. 直接插入排序

02. 简单选择排序算法的比较次数和移动次数分别为（　）。

A. $O(n)$，$O(\log_2 n)$　B. $O(\log_2 n)$，$O(n^2)$

C. $O(n^2)$，$O(n)$　D. $O(n\log_2 n)$，$O(n)$

03. 若只想得到 100000 个元素组成的序列中第 10 个最小元素之前的部分排序的序列，用（　）方法最快。

A. 冒泡排序　B. 快速排序　C. 归并排序　D. 堆排序

04. 下列（　）是一个堆。

A. 19, 75, 34, 26, 97, 56　B. 97, 26, 34, 75, 19, 56

C. 19, 56, 26, 97, 34, 75　D. 19, 34, 26, 97, 56, 75

05. 在含有 n 个元素的小根堆中（下标从 1 开始），关键字最大的元素可能存储在（　）位置。

A. $n/2$　B. $n/2+2$　C. 1　D. $n/2-1$

06. 向具有 n 个结点的堆中插入一个新元素的时间复杂度为（　），删除一个元素的时间复杂度为（　）。

A. $O(1)$　B. $O(n)$　C. $O(\log_2 n)$　D. $O(n\log_2 n)$

07. 构建 n 个记录的初始堆，其时间复杂度为（　）；对 n 个记录进行堆排序，最坏情况下其时间复杂度为（　）。

A. $O(n)$　B. $O(n^2)$　C. $O(\log_2 n)$　D. $O(n\log_2 n)$

08. 下列 4 种排序算法中，排序过程中的比较次数与序列初始状态无关的是（　）。

A. 简单选择排序　B. 直接插入排序　C. 快速排序　D. 冒泡排序

09. 对由相同的 n 个整数构成的二叉排序树和小根堆，下列说法中不正确的是（　）。

A. 二叉排序树的高度大于或等于小根堆的高度

B. 对二叉排序树进行中序遍历可以得到从小到大的序列

C. 从小根堆的根结点到任意叶结点的路径构成从小到大的序列

D. 对小根堆进行层序遍历可以得到从小到大的序列

10. 有一组数据(15, 9, 7, 8, 20, -1, 7, 4)，用堆排序的筛选方法建立的初始小根堆为（ ）。

A. -1, 4, 8, 9, 20, 7, 15, 7　　B. -1, 7, 15, 7, 4, 8, 20, 9

C. -1, 4, 7, 8, 20, 15, 7, 9　　D. A、B、C均不对

11. 对关键字序列{23, 17, 72, 60, 25, 8, 68, 71, 52}进行堆排序，输出两个最小关键字后的剩余堆是（ ）。

A. {23, 72, 60, 25, 68, 71, 52}　　B. {23, 25, 52, 60, 71, 72, 68}

C. {71, 25, 23, 52, 60, 72, 68}　　D. {23, 25, 68, 52, 60, 72, 71}

12. 堆排序分为两个阶段：第一阶段将给定的序列构造成一个初始堆，第二阶段逐次输出堆顶元素，并调整使其保持堆的性质。设有给定序列{48, 62, 35, 77, 55, 14, 35, 98}，若在堆排序的第一阶段将该序列构造成一个大根堆，则交换元素的次数为（ ）。

A. 5　　B. 6　　C. 7　　D. 8

13. 已知大根堆{62, 34, 53, 12, 8, 46, 22}，删除堆顶元素后需要重新调整堆，则在此过程中关键字的比较次数为（ ）。

A. 2　　B. 3　　C. 4　　D. 5

14. 从根结点到任意叶结点的路径都是有序的数据结构是（ ）。

A. 红黑树　　B. 二叉查找树　　C. 哈夫曼树　　D. 堆

15. 【2009统考真题】已知关键字序列{5, 8, 12, 19, 28, 20, 15, 22}是小根堆，插入关键字3，调整好后得到的小根堆是（ ）。

A. 3, 5, 12, 8, 28, 20, 15, 22, 19　　B. 3, 5, 12, 19, 20, 15, 22, 8, 28

C. 3, 8, 12, 5, 20, 15, 22, 28, 19　　D. 3, 12, 5, 8, 28, 20, 15, 22, 19

16. 【2011统考真题】已知序列{25, 13, 10, 12, 9}是大根堆，在序列尾部插入新元素18，再将其调整为大根堆，调整过程中元素之间进行的比较次数是（ ）。

A. 1　　B. 2　　C. 4　　D. 5

17. 【2015统考真题】已知小根堆为8, 15, 10, 21, 34, 16, 12，删除关键字8之后需重建堆，在此过程中，关键字之间的比较次数是（ ）。

A. 1　　B. 2　　C. 3　　D. 4

18. 【2018统考真题】在将序列(6, 1, 5, 9, 8, 4, 7)建成大根堆时，正确的序列变化过程是（ ）。

A. 6, 1, 7, 9, 8, 4, 5 → 6, 9, 7, 1, 8, 4, 5 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5

B. 6, 9, 5, 1, 8, 4, 7 → 6, 9, 7, 1, 8, 4, 5 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5

C. 6, 9, 5, 1, 8, 4, 7 → 9, 6, 5, 1, 8, 4, 7 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5

D. 6, 1, 7, 9, 8, 4, 5 → 7, 1, 6, 9, 8, 4, 5 → 7, 9, 6, 1, 8, 4, 5 → 9, 7, 6, 1, 8, 4, 5 → 9, 8, 6, 1, 7, 4, 5

19. 【2020统考真题】下列关于大根堆（至少含2个元素）的叙述中，正确的是（ ）。

I. 可以将堆视为一棵完全二叉树

II. 可以采用顺序存储方式保存堆

III. 可以将堆视为一棵二叉排序树

IV. 堆中的次大值一定在根的下一层

A. 仅I、II　　B. 仅II、III　　C. 仅I、II和IV　　D. I、III和IV

20. 【2021统考真题】将关键字6,9,1,5,8,4,7依次插入初始为空的大根堆H，得到的H是（ ）。

A. 9,8,7,6,5,4,1　　B. 9,8,7,5,6,1,4　　C. 9,8,7,5,6,4,1　　D. 9,6,7,5,8,4,1

21. 【2024统考真题】已知关键字序列28, 22, 20, 19, 8, 12, 15, 5是大根堆（最大堆），对该堆

进行两次删除操作后，得到的新堆是（　）。

A. 20, 19, 15, 12, 8, 5　　B. 20, 19, 15, 5, 8, 12

C. 20, 19, 12, 15, 8, 5　　D. 20, 19, 8, 12, 15, 5

二、综合应用题

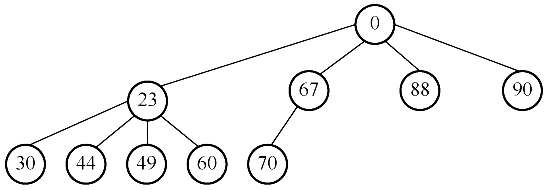
01. 指出堆和二叉排序树的区别？

02. 画出一棵二叉树，使得它既满足大根堆的要求又满足二叉排序树的要求。

03. 若只想得到一个序列中第 k（$k \geqslant 5$）个最小元素之前的部分的排序序列，则最好采用什么排序算法？

04. 通常使用的堆也称二叉堆，因为它是用完全二叉树来实现的，树中结点最多只有两个孩子。同理可以有 m 叉堆，即用完全 m 叉树来实现的堆。

1）下图是一个 m 叉小根堆，问 m 值是多少？向这个堆插入一个元素 65 后，堆中的元素如何变化？再删除堆顶元素呢？请画出变化后的树形。



2）从 0 开始对完全 4 叉树中的结点从左到右、从上到下进行编号。若给定一个结点 k，其父结点的编号是多少？（若存在），其第 i（$i=1, 2, 3, 4$）个孩子的编号是多少？

3）在 m 叉堆中进行插入和删除操作的时间复杂度是多少？

05. 编写一个算法，在基于单链表表示的待排序关键字序列上进行简单选择排序。

06. 试设计一个算法，判断一个数据序列是否构成一个小根堆。

07. 优先队列（Priority Queue）是一种数据结构，它类似于普通队列，但每个元素都有一个优先级。元素在入队时会根据其优先级来排序，而不按照先入先出的顺序来排序。每次从优先队列中出队时，出队的是优先级最高的元素，而不是最早进入队列的元素。

队列中的元素的数据结构的定义如下：

```
typedef struct{
int value;          //元素的值
int priority;       //元素的优先级，priority 越大，优先级越高
}PriorityQueueElement;
```

请设计一个优先队列，要求满足：①初始时队列为空；②入队时，不允许增加队列的占用空间；③出队后，出队元素所占用的空间可重复使用，即整个队列所占用的空间不变；④入队操作和出队操作的时间复杂度始终保持为 $O(\log_2 n)$。请回答：

1）该队列是应选择链式存储结构，还是选择顺序存储结构？

2）给出优先队列的数据结构的定义。

3）用伪代码给出入队操作和出队操作的基本过程（关键之处可用文字描述）。

08. 【2022 统考真题】现有 n（$n > 100000$）个数保存在一维数组 M 中，需要查找 M 中最小的 10 个数。请回答下列问题。

1）设计一个完成上述查找任务的算法，要求平均情况下的比较次数尽可能少，简述其算法思想（不需要编程实现）。

2）说明你所设计的算法平均情况下的时间复杂度和空间复杂度。

8.4.4 答案与解析

一、单项选择题

01. A

02. C

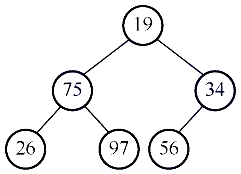
注意，读者应熟练掌握各种排序算法的思想、过程和特点。

03. D

采用堆排序时，读入前 10 个元素，建立含 10 个元素的大根堆，而后依次扫描剩余元素，若大于堆顶，则舍弃，否则用该元素取代堆顶并重新调整堆，当元素全部扫描完毕，堆中保存的即是最小的 10 个元素。冒泡排序需要从后往前执行 10 趟冒泡才能得到 10 个最小的元素。两者的时间复杂度都和数据规模 n 线性相关，但显然堆排序的常系数更小。而快速排序、归并排序的每一趟都不能保证得到当前序列的最小值，也无法达到线性时间复杂度。

04. D

可将每个选项中的序列表示成完全二叉树，再看父结点与子结点的关系是否全部满足堆的定义。例如，选项 A 中序列对应的完全二叉树如下图所示。显然，最小元素 19 在根结点，因此可能是小根堆，但 75 与 26 的关系却不满足小根堆的定义，所以选项 A 中的序列不是一个堆。其他选项采用类似的过程分析。



05. B

在小根堆中，关键字最大的元素一定存储在该堆所对应的完全二叉树的叶结点中（位于最底层或次底层）。又因为二叉树中的最后一个非叶结点存储在$\lfloor n/2 \rfloor$中，所以关键字最大的元素的存储范围为$\lfloor n/2 \rfloor+1 \sim n$。注意，读者可类比在大根堆中求关键字最小的元素的存储范围。

06. C、C

在向有 n 个元素的堆中插入一个新元素时，需要调用一个向上调整的算法，比较次数最多等于树的高度减 1，因为树的高度为$\lfloor \log_2 n \rfloor+1$，所以堆的向上调整算法的比较次数最多等于$\lfloor \log_2 n \rfloor$。此处需要注意，调整堆和建初始堆的时间复杂度是不一样的，读者可以仔细分析两个算法的具体执行过程。

07. A、D

建堆过程中，向下调整的时间与树高 h 有关，为 $O(h)$。每次向下调整时，大部分结点的高度都较小。因此，可以证明在元素个数为 n 的序列上建堆，其时间复杂度为 $O(n)$。无论是在最好情况下还是在最坏情况下，堆排序的时间复杂度均为 $O(n\log_2 n)$。

08. A

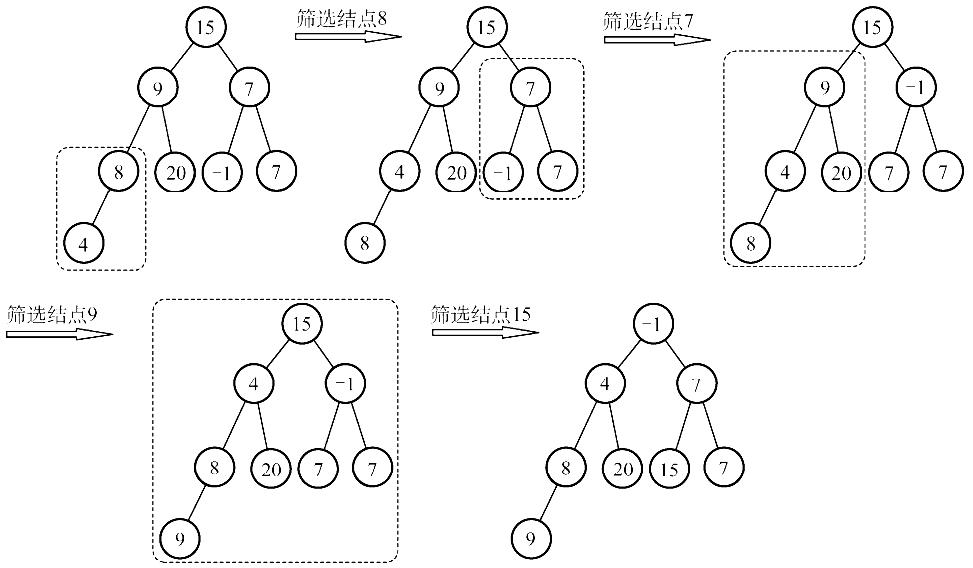
简单选择排序的比较次数始终为 $n(n-1)/2$，与序列状态无关。

09. D

堆是顺序存储的完全二叉树，因此其高度小于或等于结点数相同的二叉排序树，选项 A 正确。选项 B 显然正确。根据小根堆的定义，其根结点到任意叶结点的路径构成从小到大的序列，选项 C 正确。堆的各层结点之间没有大小要求，因此层序遍历不能保证得到有序序列，选项 D 错误。

10．C

从⌊*n*/2⌋～1 依次筛选堆的过程如下图所示，显然选择选项 C。



11．D

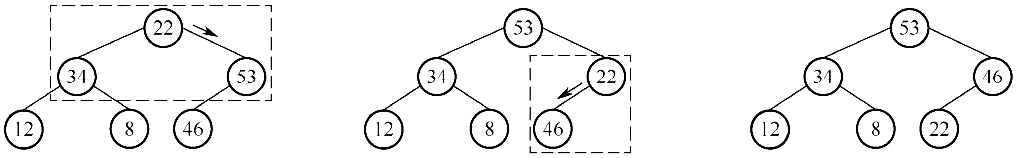
筛选法初始建堆为{8, 17, 23, 52, 25, 72, 68, 71, 60}，输出 8 后重建的堆为{17, 25, 23, 52, 60, 72, 68, 71}，输出 17 后重建的堆为{23, 25, 68, 52, 60, 72, 71}。

12．B

初始序列是一棵顺序存储的完全二叉树，然后根据大根堆的要求，按照从下到上、从右到左的顺序进行调整。98 和 77 比较，98 和 77 交换（交换 1 次）；14 和 35 比较，35 和 35 比较，不交换；98 和 55 比较，98 和 62 比较，98 和 62 交换（交换 1 次）；62 和 77 比较，77 和 62 交换（交换 1 次）；98 和 35 比较，98 和 48 比较，98 和 48 交换（交换 1 次）；77 和 55 比较，77 和 48 比较，77 和 48 交换（交换 1 次）；48 和 62 比较，62 和 48 交换（交换 1 次），共交换 6 次。

13．B

删除堆顶 62 后，将堆尾 22 放入堆顶，然后自上而下调整。首先 34 与 53 比较（第一次比较），较大者 53 与根 22 比较（第二次比较），53 被换至堆顶；22 只有一个孩子，直接与其左孩子 46 比较（第 3 次比较），22 与 46 交换，至此大根堆调整结束，具体过程如下图所示。

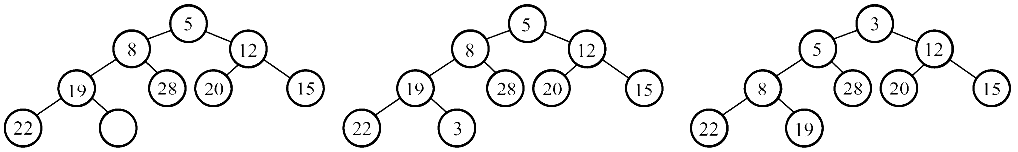


14．D

红黑树和二叉查找树的中序序列是有序序列，从根结点到任意叶结点的路径不能保证是有序的。哈夫曼树是根据权值按一定规则构造的树，和关键字次序无关。若是小根堆，则从根结点到任意叶结点的路径是升序序列；若是大根堆，则从根结点到叶结点的路径是降序序列。

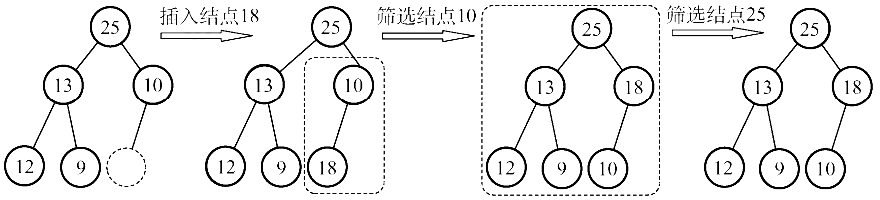
15．A

插入关键字 3 后，堆的变化过程如下图所示。



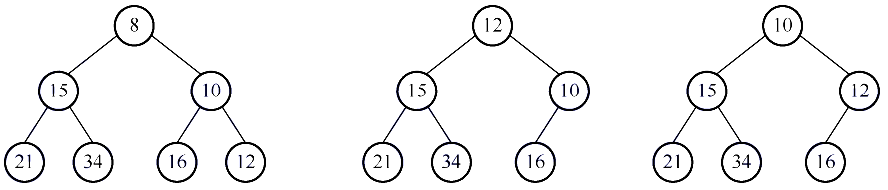
16. B

首先 18 与 10 比较，交换；18 与 25 比较，不交换。共比较 2 次，调整过程如下图所示。



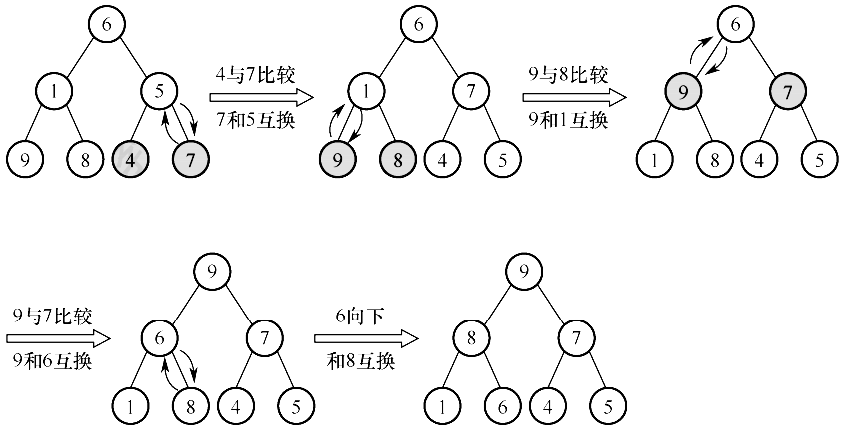
17. C

删除 8 后，将 12 移动到堆顶，第一次是 15 和 10 比较，第二次是 10 和 12 比较并交换，第三次还需比较 12 和 16，所以比较次数为 3。



18. A

要熟练掌握建堆和调整堆的方法，从序列末尾开始向前遍历，变换过程如下图所示。

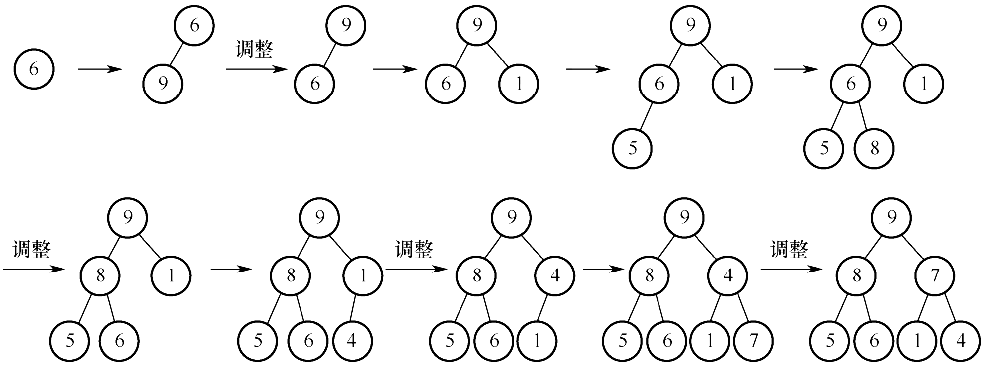


19. C

简单概念题。堆是一棵完全树，采用一维数组存储，选项 I 正确，选项 II 正确。大根堆只要求根结点值大于左右孩子值，并不要求左右孩子值有序，选项 III 错误。堆的定义是递归的，所以其左右子树也是大根堆，所以堆的次大值一定是其左孩子或右孩子，选项 IV 正确。

20. B

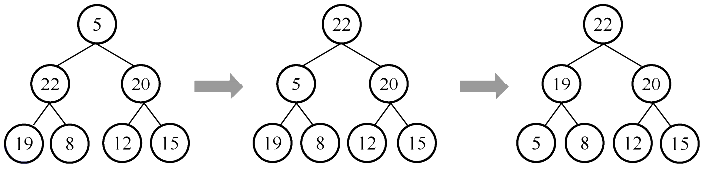
要熟练掌握调整堆的方法，建堆的过程如下图所示，所以答案选择选项 B。



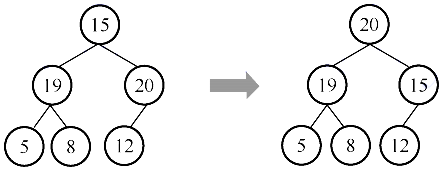
注意，给定序列依次插入空堆的结果与给定序列直接调整成堆的结果是不同的，最终得到的堆的形式也不同。若对序列 6, 9, 1, 5, 8, 4, 7 直接调整成堆，则会误选选项 C。

21．B

该序列已调整成大根堆，接下来进行第一次删除操作：删除 28 后，将 5 放入堆顶；然后自上而下调整。22 和 20 比较，较大者 22 与 5 比较，交换；19 和 8 比较，较大者 19 与 5 比较，交换。



接下来进行第二次删除操作：删除 22 后，将 15 放入堆顶；然后自上而下调整。19 和 20 比较，较大者 20 与 15 比较，交换；15 直接与其仅有的左孩子 12 比较，不交换。



因此，进行两次删除操作后，得到的新堆是 20, 19, 15, 5, 8, 12。

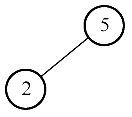
二、综合应用题

01.【解答】

以小根堆为例，堆的特点是双亲结点的关键字必然小于或等于该孩子结点的关键字，而两个孩子结点的关键字没有次序规定。在二叉排序树中，每个双亲结点的关键字均大于左子树结点的关键字，均小于右子树结点的关键字，也就是说，每个双亲结点的左、右孩子的关键字有次序关系。这样，当对两种树执行中序遍历后，二叉排序树会得到一个有序的序列，而堆则不一定能得到一个有序的序列。

02.【解答】

大根堆要求根结点的关键字值既大于或等于左子女的关键字值，又大于或等于右子女的关键字值。二叉排序树要求根结点的关键字值大于左子女的关键字值，同时小于右子女的关键字值。两者的交集是：根结点的关键字值大于左子女的关键字值。这意味着它是一棵左斜单支树，但大根堆要求是完全二叉树，因此最后得到的只能是如下图所示的两个结点的二叉树。



读者也可能会注意到，当只有一个结点时，显然是满足题意的，但我们不举一个结点的例子是为了体现出排序树与大根堆的区别。

03.【解答】

在基于比较的排序算法中，插入排序、快速排序和归并排序只有在将元素全部排完序后，才能得到前 k 个最小的元素序列，算法的效率不高。

冒泡排序、堆排序和简单选择排序可以，因为它们在每一趟中都可以确定一个最小的元素。采用堆排序最合适，对于 n 个元素的序列，建立初始堆的时间不超过 $4n$，取得第 k 个最小元素之前的排序序列所花的时间为 $k\log_2 n$，总时间为 $4n + k\log_2 n$；冒泡和简单选择排序完成此功能所花的时间为 kn，当 $k \geqslant 5$ 时，通过比较可以得出堆排序最优。

注　意

求前 k 个最小元素的顺序排列可采用的排序算法有冒泡排序、堆排序和简单选择排序。

04.【解析】

1）除最后一个分支结点外，其余每个分支结点都有 4 个孩子，所以该树是完全 4 叉树。插入元素 65 后，再删除堆顶元素的树形分别如下图 1 和图 2 所示。

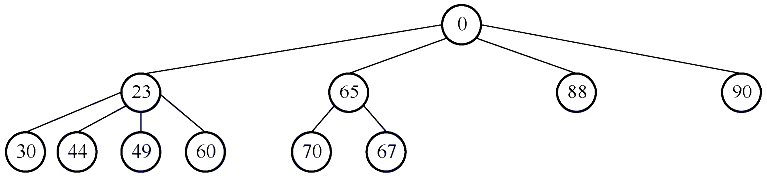


图 1　插入 65 后的树形

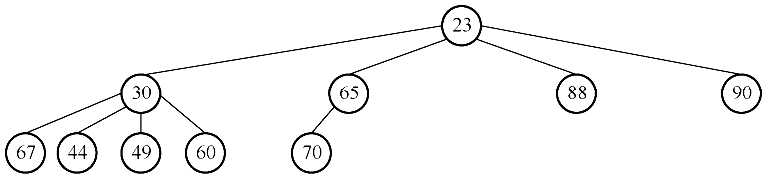


图 2　删除堆顶元素后的树形

2）父结点的编号为 $(k-1)/4$，第 i 个孩子的编号为 $4\times k + i$，$i = 1, 2, 3, 4$。

3）与二叉堆类似，插入和删除操作都有向上、向下调整的过程，操作时间都与树的高度有关。因此，插入和删除操作的时间复杂度都为 $O(\log_m n)$，其中 n 为元素个数。

05.【解答】

算法的思想是：每趟在原始链表中摘下关键字最大的结点，把它插入结果链表的最前端。在原始链表中摘下的关键字越来越小，在结果链表前端插入的关键字也越来越小，因此最后形成的结果链表中的结点将按关键字非递减的顺序有序链接。

单链表的定义如第 2 章所述，假设它不带表头结点。

```
void selectSort(LinkedList& L){
//对不带表头结点的单链表 L 执行简单选择排序
    LinkNode *h=L,*p,*q,*r,*s;
    L=NULL;
    while(h!=NULL){                                    //持续扫描原链表
        p=s=h;q=r=NULL;
        //指针 s 和 r 记忆最大结点和其前驱；p 为工作指针，q 为其前驱
```

```
            while(p!=NULL){                          //扫描原链表寻找最大结点 s
                if(p->data>s->data){s=p;r=q;}        //找到更大的，记忆它和它的前驱
                q=p;p=p->link;                       //继续寻找
            }
            if(s==h)
                h=h->link;                           //最大结点在原链表前端
            else
                r->link=s->link;                     //最大结点在原链表表内
            s->link=L;L=s;                           //结点 s 插入结果链前端
        }
    }
```

06.【解答】

将顺序表 L[1…n]视为一个完全二叉树，扫描所有分支结点，遇到孩子结点的关键字小于根结点的关键字时返回 false，扫描完后返回 true。算法的实现如下：

```
bool IsMinHeap(ElemType A[],int len){
    if(len%2==0){                    //len 为偶数,有一个单分支结点
        if(A[len/2]>A[len])          //判断单分支结点
            return  false;
        for(i=len/2-1;i>=1;i--)//判断所有双分支结点
            if(A[i]>A[2*i]||A[i]>A[2*i+1])
                return  false;
    }
    else{                            //len 为奇数时，没有单分支结点
        for(i=len/2;i>=1;i--)        //判断所有双分支结点
            if(A[i]>A[2*i]||A[i]>A[2*i+1])
                return  false;
    }
    return  true;
}
```

07.【解答】

1）题目要求整个队列所占用的空间不变，入队操作和出队操作的时间复杂度始终保持为 $O(\log_2 n)$，则应采用顺序存储结构，即用数组实现大根堆。

2）优先队列的数据结构定义如下：

```
typedef struct{
    PriorityQueueElement heap[MAX_SIZE];    //用数组实现堆
    int size;                               //当前堆中元素的数量
}PriorityQueue
```

3）入队操作：

```
void enqueue(PriorityQueue *pq,int value,int priority){
    if(pq->size>=MAX_SIZE){
        队列已满，无法入队;
        return;
    }
    将新元素添加到堆的末尾;
    向上调整堆;
}
```

出队操作：

```
PriorityQueueElement dequeue(PriorityQueue *pq){
    if(pq->size==0){
        队列为空时直接退出;
    }
    获取堆顶元素（优先级最高的元素）;
    将堆的最后一个元素放到堆顶;
```

```
        向下调整堆;
        返回出队元素;
    }
```

08.【解答】

1）算法思想。

【方法1】定义含10个元素的数组A，初始时元素值均为该数组类型能表示的最大数MAX。

```
for  M中的每个元素s
    if (s<A[9])  丢弃A[9]并将s按升序插入A;
```

当数据全部扫描完毕，数组A[0]～A[9]保存的就是最小的10个数。

【方法2】定义含10个元素的大根堆H，元素值均为该堆元素类型能表示的最大数MAX。

```
for  M中的每个元素s
    if (s<H的堆顶元素)  删除堆顶元素并将s插入H;
```

当数据全部扫描完毕，堆H中保存的就是最小的10个数。

2）算法平均情况下的时间复杂度是$O(n)$，空间复杂度是$O(1)$。

8.5 归并排序、基数排序和计数排序

8.5.1 归并排序

命题追踪 ▶ 二路归并操作的功能（2022）

归并排序与上述基于交换、选择等排序的思想不一样，*归并*的含义是将两个或两个以上的有序表合并成一个新的有序表。假定待排序表含有n个记录，则可将其视为n个有序的子表，每个子表的长度为1，然后两两归并，得到$\lceil n/2 \rceil$个长度为2或1的有序表；继续两两归并……如此重复，直到合并成一个长度为n的有序表为止，这种排序算法称为*二路归并排序*。

图8.9所示为二路归并排序的一个例子，经过三趟归并后合并成了有序序列。

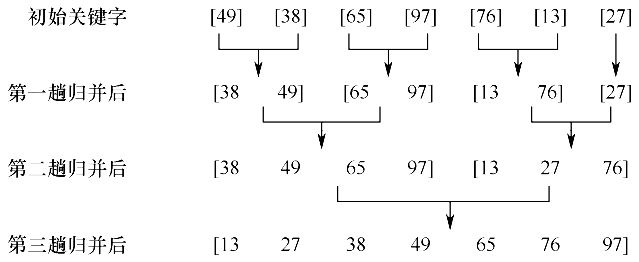


图8.9 二路归并排序示例

命题追踪 ▶ （算法题）归并排序思想的应用（2011）

Merge()的功能是将前后相邻的两个有序表归并为一个有序表。设两段有序表A[low…mid]、A[mid+1…high]存放在同一顺序表中的相邻位置，先将它们复制到辅助数组B中。每次从B的两段中取出一个记录进行关键字的比较，将较小者放入A中，当B中有一段的下标超出其对应的表长（该段的所有元素都已复制到A中）时，将另一段的剩余部分直接复制到A中。算法如下：

```
ElemType *B=(ElemType *)malloc((n+1)*sizeof(ElemType)); //辅助数组B
void Merge(ElemType A[],int low,int mid,int high){
```

```
//表 A 的两段 A[low…mid]和 A[mid+1…high]各自有序，将它们合并成一个有序表
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k];                          //将 A 中所有元素复制到 B 中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])                      //比较 B 的两个段中的元素
            A[k]=B[i++];                    //将较小值复制到 A 中
        else
            A[k]=B[j++];
    }
    while(i<=mid)    A[k++]=B[i++];  //若第一个表未检测完，复制
    while(j<=high)   A[k++]=B[j++];  //若第二个表未检测完，复制
}
```

注　意

在上面的代码中，最后两个 while 循环只有一个会执行。

一趟归并排序的操作是，调用$\lceil n/2h \rceil$次算法 merge()，将 L[1…n]中前后相邻且长度为 h 的有序段进行两两归并，得到前后相邻、长度为 $2h$ 的有序段，整个归并排序需要进行$\lceil \log_2 n \rceil$趟。

递归形式的二路归并排序算法是基于分治的，其过程如下。

分解：将含有 n 个元素的待排序表分成各含 $n/2$ 个元素的子表，采用二路归并排序算法对两个子表递归地进行排序。

合并：合并两个已排序的子表得到排序结果。

```
void MergeSort(ElemType A[],int low,int high){
    if(low<high){
        int mid=(low+high)/2;          //从中间划分两个子序列
        MergeSort(A,low,mid);          //对左侧子序列进行递归排序
        MergeSort(A,mid+1,high);       //对右侧子序列进行递归排序
        Merge(A,low,mid,high);         //归并
    }
}
```

命题追踪　归并排序和插入排序的对比（2017）

命题追踪　二路归并比较次数的分析（2024）

二路归并排序算法的性能分析如下：

空间效率：Merge()操作中，辅助空间刚好为 n 个单元，因此算法的空间复杂度为 $O(n)$。

时间效率：每趟归并的时间复杂度为 $O(n)$，共需进行$\lceil \log_2 n \rceil$趟归并，因此算法的时间复杂度为 $O(n\log_2 n)$。

稳定性：Merge()操作不会改变相同关键字记录的相对次序，因此二路归并排序算法是一种稳定的排序算法。

适用性：归并排序适用于顺序存储和链式存储的线性表。

注　意

一般而言，对于 N 个元素进行 k 路归并排序时，排序的趟数 m 满足 $k^m=N$，从而 $m=\log_k N$，又考虑到 m 为整数，因此 $m=\lceil \log_k N \rceil$。这和前面的二路归并排序算法是一致的。

8.5.2　基数排序

基数排序是一种很特别的排序算法，它不基于比较和移动进行排序，而基于关键字各位的大

小进行排序。基数排序是一种借助多关键字排序的思想对单逻辑关键字进行排序的方法。

假设长度为 n 的线性表中每个结点 a_j 的关键字由 d 元组（$k_j^{d-1}, k_j^{d-2}, \cdots, k_j^1, k_j^0$）组成，满足 $0 \leqslant k_j^i \leqslant r-1$（$0 \leqslant j < n, 0 \leqslant i \leqslant d-1$）。其中 k_j^{d-1} 为最高位关键字，k_j^0 为最低位关键字。

为实现多关键字排序，通常有两种方法：第一种是最高位优先（MSD）法，按关键字位权重递减依次逐层划分成若干更小的子序列，最后将所有子序列依次连接成一个有序序列；第二种是最低位优先（LSD）法，按关键字位权重递增依次进行排序，最后形成一个有序序列。

下面描述以 r 为基数的最低位优先基数排序的过程，在排序过程中，使用 r 个队列 $Q_0, Q_1, \cdots, Q_{r-1}$。基数排序的过程如下：

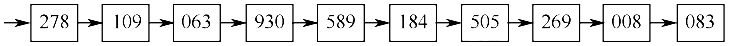
对 $i = 0, 1, \cdots, d-1$，依次做一次分配和收集（其实是一次稳定的排序过程）。

① 分配：开始时，把 $Q_0, Q_1, \cdots, Q_{r-1}$ 各个队列置成空队列，然后依次考察线性表中的每个结点 a_j（$j = 0, 1, \cdots, n-1$），若 a_j 的关键字 $k_j^i = k$，就把 a_j 放进 Q_k 队列中。

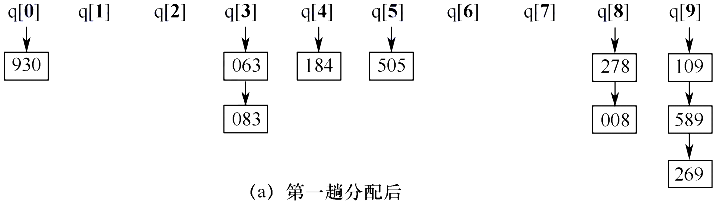
② 收集：把 $Q_0, Q_1, \cdots, Q_{r-1}$ 各个队列中的结点依次首尾相接，得到新的结点序列，从而组成新的线性表。

命题追踪 ▶ 基数排序的中间过程的分析（2013、2021）

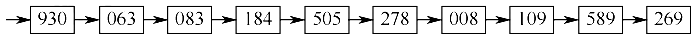
通常采用链式基数排序，假设对如下 10 个记录进行排序：



每个关键字是 1000 以下的正整数，基数 r = 10，在排序过程中需借助 10 个链队列，每个关键字由 3 位子关键字构成——$K^1K^2K^3$，分别代表百位、十位和个位，一共进行三趟“分配”和“收集”操作。第一趟分配用最低位子关键字 K^3 进行，将所有最低位子关键字（个位）相等的记录分配到同一个队列，如图 8.10(a)所示，然后进行收集操作。第一趟收集后的结果如图 8.10(b)所示。



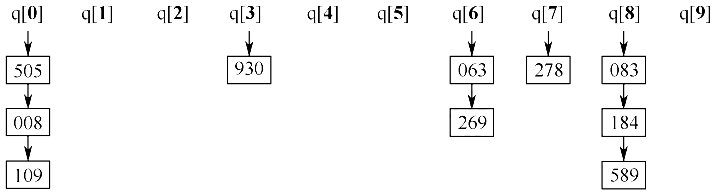
(a) 第一趟分配后



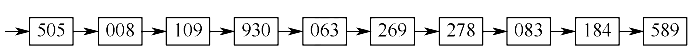
(b) 第一趟收集后

图 8.10　第一趟链式基数排序操作

第二趟分配用次低位子关键字 K^2 进行，将所有次低位子关键字（十位）相等的记录分配到同一个队列，如图 8.11(a)所示。第二趟收集后的结果如图 8.11(b)所示。



(a) 第二趟分配后



(b) 第二趟收集后

图 8.11　第二趟链式基数排序操作

第三趟分配用最高位子关键字 K^1 进行，将所有最高位子关键字（百位）相等的记录分配到同一个队列，如图 8.12(a)所示，第三趟收集后的结果如图 8.12(b)所示，至此整个排序结束。

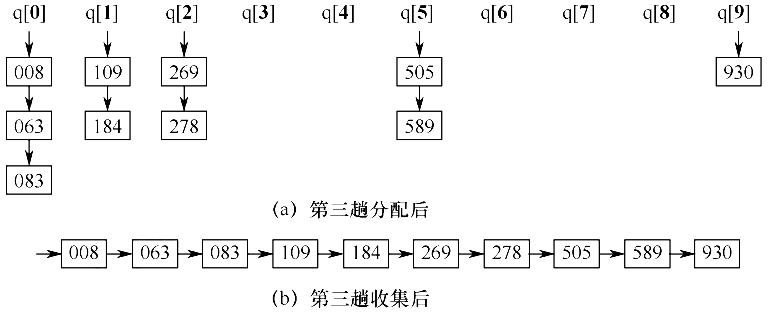


图 8.12　第三趟链式基数排序操作

基数排序算法的性能分析如下。

空间效率：一趟排序需要的辅助存储空间为 r（r 个队列：r 个队头指针和 r 个队尾指针），但以后的排序中会重复使用这些队列，所以基数排序的空间复杂度为 $O(r)$。

命题追踪 ▶ 元素的移动次数与序列初态无关的排序算法（2015）

时间效率：基数排序需要进行 d 趟“分配”和”收集”操作。一趟分配需要遍历所有关键字，时间复杂度为 $O(n)$；一趟收集需要合并 r 个队列，时间复杂度为 $O(r)$。因此基数排序的时间复杂度为 $O(d(n+r))$，它与序列的初始状态无关。

稳定性：每一趟分配和收集都是从前往后进行的，不会交换相同关键字的相对位置，因此基数排序是一种稳定的排序算法。

适用性：基数排序适用于顺序存储和链式存储的线性表。

*8.5.3　计数排序

命题追踪 ▶ （算法题）计数排序思想的应用（2013、2015、2018）

计数排序也是一种不基于比较的排序算法。计数排序的思想是：对每个待排序元素 x，统计小于 x 的元素个数，利用该信息就可确定 x 的最终位置。例如，若有 8 个元素小于 x，则 x 就排在第 9 号位置上。当有几个元素相同时，该排序方案还需做一定的优化。

注　意

计数排序并不在统考大纲的范围内，但其排序思想在历年真题中多次涉及。

在计数排序算法的实现中，假设输入是一个数组 A[n]，序列长度为 n，我们还需要两个数组：B[n]存放输出的排序序列，C[k]存储计数值。用输入数组 A 中的元素作为数组 C 的下标（索引），而该元素出现的次数存储在该元素作为下标的数组 C 中。算法如下：

命题追踪 ▶ 计数排序相关的思想和代码的分析（2021）

```
void CountSort(ElemType A[],ElemType B[],int n,int k){
    int i,C[k];
    for(i=0;i<k;i++)
        C[i]=0;                    //初始化计数数组
    for(i=0;i<n;i++)               //遍历输入数组，统计每个元素出现的次数
        C[A[i]]++;                 //C[A[i]]保存的是等于A[i]的元素个数
```

```
    for(i=1;i<k;i++)
        C[i]=C[i]+C[i-1];      //C[x]保存的是小于或等于x的元素个数
    for(i=n-1;i>=0;i--){       //从后往前遍历输入数组
        B[C[A[i]]-1]=A[i];     //将元素A[i]放在输出数组B[]的正确位置上
        C[A[i]]=C[A[i]]-1;
    }
}
```

第一个 for 循环执行完后，数组 C 的值初始化为 0。第二个 for 循环遍历输入数组 A，若一个输入元素的值为 x，则将 C[x]值加 1，该 for 循环执行完后，C[x]中保存的是等于 x 的元素个数。第三个 for 循环通过累加计算后，C[x]中保存的是小于或等于 x 的元素个数。第四个 for 循环从后往前遍历数组 A，把每个元素 A[i]放入它在输出数组 B 的正确位置上。若数组 A 中不存在相同的元素，则 C[A[i]]-1 就是 A[i]在数组 B 中的最终位置，这是因为共有 C[A[i]]个元素小于或等于 A[i]。若数组 A 中存在相同的元素，将每个元素 A[i]放入数组 B[]后，都要将 C[A[i]]减 1，这样，当遇到下一个等于 A[i]的输入元素（若存在）时，该元素就可放在数组 B 中 A[i]的前一个位置上。

假设输入数组A[]={2,4,3,0,2,3}，第二个for循环执行完后，辅助数组C的情况如图8.13(a)所示；第三个 for 循环执行完后，辅助数组 C 的情况如图 8.13(b)所示。图 8.13(c)至图 8.13(h)分别是第四个 for 循环每迭代一次后，输出数组 B 和辅助数组 C 的情况。

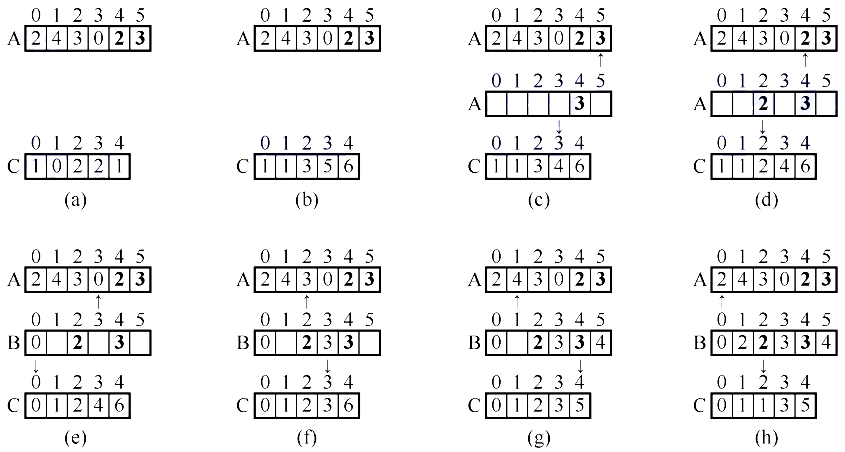


图 8.13　计数排序的过程

由上面的过程可知，计数排序的原理是：数组的索引（下标）是递增有序的，通过将序列中的元素作为辅助数组的索引，其个数作为值放入辅助数组，遍历辅助数组来排序。

计数排序算法的性能分析如下。

空间效率：计数排序是一种用空间换时间的做法。输出数组的长度为 n；辅助的计数数组的长度为 k，空间复杂度为 $O(n+k)$。若不把输出数组视为辅助空间，则空间复杂度为 $O(k)$。

时间效率：上述代码的第 1 个和第 3 个 for 循环所花的时间为 $O(k)$，第 2 个和第 4 个 for 循环所花的时间为 $O(n)$，总时间复杂度为 $O(n+k)$。因此，当 $k=O(n)$时，计数排序的时间复杂度为 $O(n)$；但当 $k>O(n\log_2 n)$时，其效率反而不如一些基于比较的排序（如快速排序、堆排序等）。

稳定性：上述代码的第 4 个 for 循环从后往前遍历输入数组，相同元素在输出数组中的相对位置不会改变，因此计数排序是一种稳定的排序算法。

适用性：计数排序更适用于顺序存储的线性表。计数排序适用于序列中的元素是整数且元素范围（0～$k-1$）不能太大，否则会造成辅助空间的浪费。

8.5.4　本节试题精选

一、单项选择题

01. 以下排序算法中，（　）在一趟结束后不一定能选出一个元素放在其最终位置上。

A. 简单选择排序　B. 冒泡排序　C. 归并排序　D. 堆排序

02. 以下排序算法中，（　）不需要进行关键字的比较。

A. 快速排序　B. 归并排序　C. 基数排序　D. 堆排序

03. 在下列排序算法中，平均情况下空间复杂度为 $O(n)$的是（　），最坏情况下空间复杂度为 $O(n)$的是（　）。

I. 希尔排序　II. 堆排序　III. 冒泡排序

IV. 归并排序　V. 快速排序　VI. 基数排序

A. I、IV、VI　B. II、V　C. IV、V　D. IV

04. 下列排序算法中，排序过程中比较次数的数量级与序列初始状态无关的是（　）。

A. 归并排序　B. 插入排序　C. 快速排序　D. 冒泡排序

05. 二路归并排序中，归并趟数的数量级是（　）。

A. $O(n)$　B. $O(\log_2 n)$　C. $O(n\log_2 n)$　D. $O(n^2)$

06. 若对 27 个元素只进行三趟多路归并排序，则选取的归并路数最少为（　）。

A. 2　B. 3　C. 4　D. 5

07. 将两个各有 N 个元素的有序表合并成一个有序表，最少的比较次数是（　），最多的比较次数是（　）。

A. N　B. $2N-1$　C. $2N$　D. $N-1$

08. 用归并排序算法对序列{1, 2, 6, 4, 5, 3, 8, 7}进行排序，共需要进行（　）次比较。

A. 12　B. 13　C. 14　D. 15

09. 一组经过第一趟二路归并排序后的记录的关键字为{25, 50, 15, 35, 80, 85, 20, 40, 36, 70}，其中包含 5 个长度为 2 的有序表，用二路归并排序算法对该序列进行第二趟归并后的结果为（　）。

A. 15, 25, 35, 50, 80, 20, 85, 40, 70, 36　B. 15, 25, 35, 50, 20, 40, 80, 85, 36, 70

C. 15, 25, 50, 35, 80, 85, 20, 36, 40, 70　D. 15, 25, 35, 50, 80, 20, 36, 40, 70, 85

10. 若将中国人按照生日（不考虑年份，只考虑月、日）来排序，则使用下列排序算法时，最快的是（　）。

A. 归并排序　B. 希尔排序　C. 快速排序　D. 基数排序

11. 设线性表中每个元素有两个数据项 k_1 和 k_2，现对线性表按以下规则进行排序：先看数据项 k_1，k_1 值小的元素在前，大的元素在后；在 k_1 值相同的情况下，再看 k_2，k_2 值小的元素在前，大的元素在后。满足这种要求的排序算法是（　）。

A. 先按 k_1 进行直接插入排序，再按 k_2 进行简单选择排序

B. 先按 k_2 进行直接插入排序，再按 k_1 进行简单选择排序

C. 先按 k_1 进行简单选择排序，再按 k_2 进行直接插入排序

D. 先按 k_2 进行简单选择排序，再按 k_1 进行直接插入排序

12. 对{05, 46, 13, 55, 94, 17, 42}进行基数排序，一趟排序的结果是（　）。

A. 05, 46, 13, 55, 94, 17, 42　B. 05, 13, 17, 42, 46, 55, 94

C. 42, 13, 94, 05, 55, 46, 17　D. 05, 13, 46, 55, 17, 42, 94

13. 有 n 个十进制整数进行基数排序，其中最大的整数为5位，则基数排序过程中临时建立的队列个数是（ ）。

A. n　　B. 2　　C. 5　　D. 10

14. 下列各种排序算法中，（ ）需要的附加存储空间最大。

A. 快速排序　　B. 堆排序　　C. 归并排序　　D. 插入排序

15.【2013统考真题】已知两个长度分别为 m 和 n 的升序链表，若将它们合并为长度为 $m+n$ 的一个降序链表，则最坏情况下的时间复杂度是（ ）。

A. $O(n)$　　B. $O(mn)$　　C. $O(\min(m,n))$　　D. $O(\max(m,n))$

16.【2013统考真题】对给定的关键字序列110, 119, 007, 911, 114, 120, 122进行基数排序，第二趟分配、收集后得到的关键字序列是（ ）。

A. 007, 110, 119, 114, 911, 120, 122　　B. 007, 110, 119, 114, 911, 122, 120

C. 007, 110, 911, 114, 119, 120, 122　　D. 110, 120, 911, 122, 114, 007, 119

17.【2015统考真题】下列排序算法中，元素的移动次数与关键字的初始状态无关的是（ ）。

A. 直接插入排序　　B. 冒泡排序　　C. 基数排序　　D. 快速排序

18.【2021 统考真题】设数组 S[]={93,946,372,9,146,151,301,485,236,327,43,892}，采用最低位优先（LSD）基数排序将S排列成升序序列。第一趟分配、收集后，元素372之前、之后紧邻的元素分别是（ ）。

A. 43, 892　　B. 236, 301　　C. 301, 892　　D. 485, 301

19.【2022统考真题】使用二路归并排序对含 n 个元素的数组M进行排序时，二路归并排序的功能是（ ）。

A. 将两个有序表合并为一个新的有序表

B. 将M划分为两部分，两部分的元素个数大致相等

C. 将M划分为 n 个部分，每个部分中仅含有一个元素

D. 将M划分为两部分，一部分元素的值均小于另一部分元素的值

20.【2024统考真题】现有由关键字组成的3个有序序列(3, 5)、(7, 9)和(6)，若按从左至右的次序选择有序序列进行二路归并排序，则关键字之间的总比较次数是（ ）。

A. 3　　B. 4　　C. 5　　D. 6

二、综合应用题

01. 已知序列{503, 87, 512, 61, 908, 170, 897, 275, 653, 462}，采用非递归的二路归并排序算法对该序列做升序排序时需要几趟排序？给出每一趟的结果。

02. 设待排序的关键字序列为{12, 2, 16, 30, 28, 10, 16^*, 20, 6, 18}，试写出使用最低位优先（LSD）基数排序算法每趟排序后的结果。

03.【2021统考真题】已知某排序算法如下：

```
void cmpCountSort(int a[],int b[],int n){
    int i,j,*count;
    count=(int *)malloc(sizeof(int)*n);
                          //C++语言：count=new int[n];
    for(i=0;i<n;i++)    count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])  count[j]++;
            else           count[i]++;
    for(i=0;i<n;i++)     b[count[i]]= a[i];
    free(count);       //C++语言：delete count;
}
```

请回答下列问题。

1）若有 int a[]={25,-10,25,10,11,19},b[6];，则调用 cmpCountSort(a,b,6)后数组 b 中的内容是什么？

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少？

3）该算法是稳定的吗？若是，阐述理由；否则，修改为稳定排序算法。

8.5.5　答案与解析

一、单项选择题

01．C

我们知道插入排序不能保证在一趟排序结束后一定有元素放在最终位置上。事实上，归并排序也不能保证。例如，序列{6, 5, 7, 8, 2, 1, 4, 3}进行一趟二路归并排序（从小到大）后为{5, 6, 7, 8, 1, 2, 3, 4}，显然它们都未被放在最终位置上。

02．C

基数排序是基于关键字各位的大小进行排序的，而不是基于关键字的比较进行的。

03．D、C

归并排序在平均情况和最坏情况下的空间复杂度都是 $O(n)$，快速排序只在最坏情况下才是 $O(n)$，平均情况是 $O(\log_2 n)$。因此，归并排序是本章所有排序算法中占用辅助空间最多的。

04．A

前面已讲过选择排序的比较次数与序列初始状态无关，归并排序的比较次数的数量级也与序列的初始状态无关。读者应能从算法的原理方面来考虑为什么和初始状态无关。

05．B

N 个元素进行 k 路归并排序的趟数 m 满足 $k^m=N$，即 $m=\lceil \log_k N \rceil$，本题中为 $\lceil \log_2 n \rceil$。

06．B

N 个元素进行 k 路归并排序的趟数 m 满足 $k^m=N$，这里要求的是 k，代入可得 $k=3$。

07．A、B

注意，当一个表中的最小元素比另一个表中的最大元素还大时，比较的次数是最少的，仅比较 N 次；而当两个表中的元素依次间隔地比较时，即 $a_1<b_1<a_2<b_2<\cdots<a_n<b_n$ 时，比较的次数是最多的，为 $2N-1$ 次。

建议读者对此举一反三：若将本题中的两个有序表的长度分别设为 M 和 N，则最多（或最少）的比较次数是多少？时间复杂度又是多少？

08．C

第一趟归并后{1, 2}, {4, 6}, {3, 5}, {7, 8}，共比较 4 次；第二趟归并后{1, 2, 4, 6}, {3, 5, 7, 8}，共比较 4 次；第三趟归并后{1, 2, 3, 4, 5, 6, 7, 8}，共比较 6 次。三趟归并共需进行 14 次比较。

09．B

这里采用二路归并排序算法，且是第二趟排序，因此每 4 个元素放在一起归并，可将序列划分为{25, 50, 15, 35}, {80, 85, 20, 40}和{36, 70}，分别对它们进行排序后有{15, 25, 35, 50}, {20, 40, 80, 85}和{36, 70}。

10．D

按照所有中国人的生日排序，一方面 N 是非常大的，另一方面关键字所含的排序码数为 2，且一个排序码的基数为 12，另一个排序码的基数为 31，都是较小的常数值，因此采用基数排序可以在 $O(N)$内完成排序过程。

11．D

本题思路来自基数排序的 LSD，首先应确定 k_1、k_2 的排序顺序，若先排 k_1 再排 k_2，则排序结果不符合题意，排除选项 A 和 C。再考虑排序的稳定性，当 k_2 排好序后，再对 k_1 排序，若对 k_1 排序采用的方法是不稳定的，则对于 k_1 相同而 k_2 不同的元素可能会改变相对次序，从而不一定能满足题设要求。直接插入排序是稳定的，而简单选择排序是不稳定的，只能选择选项 D。

12．C

基数排序有 MSD 和 LSD 两种，基数排序是稳定的。对于选项 A，不符合 LSD 和 MSD；对于选项 B，符合 MSD，但关键字 42、46 的相对位置发生了变化；对于选项 D，不符合 LSD 和 MSD。

13．D

基数排序中建立的队列个数等于进制数。

14．C

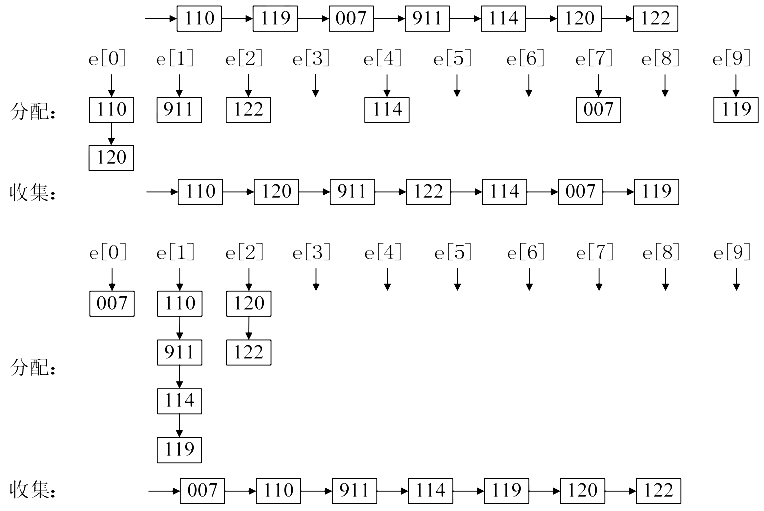
快速排序的平均空间复杂度是 $O(\log_2 n)$，归并排序的空间复杂度是 $O(n)$，其他都是 $O(1)$。

15．D

考虑两个升序链表合并，两两比较表中元素，每比较一次，确定一个元素的链接位置（取较小元素，头插法）。当一个链表比较结束后，将另一个链表的剩余元素插入即可。最坏的情况是两个链表中的元素依次进行比较，因为 $2\max(m, n) \geqslant m+n$，所以时间复杂度为 $O(\max(m, n))$。此外，每次比较把两个链表中的较小结点插入新链表的表头，直到一个链表为空，因为原链表是升序排列的，要求合并后为降序排列的，因此还要把另一个链表剩下的结点一一插入新链表的表头，不论是最好情况还是最坏情况，都需要遍历两个链表中的所有结点。

16．C

基数排序的第一趟排序是按照个位数字的大小来进行的，第二趟排序是按照十位数字的大小来进行的，排序的过程如下图所示。



17．C

基数排序的元素移动次数与序列初态无关，而其他三种排序算法都与序列初态有关。

18．C

基数排序是一种稳定的排序算法。因为采用最低位优先（LSD）的基数排序，即第一趟对个位进行分配和收集操作，所以第一趟分配和收集后的结果是{151, 301, 372, 892, 93, 43, 485, 946,

146, 236, 327, 9}，元素 372 之前、之后紧邻的元素分别是 301 和 892。

19．A。

送分题。本书对归并的定义原话是“归并的含义是将两个或两个以上的有序表合并成一个新的有序表”，而二路归并是将两个有序表合并为一个新的有序表。

20．C

从左至右对这三个有序序列进行二路归并排序的过程如下：

①合并(3, 5)和(7, 9)：比较 3 和 7（1 次），选择 3；比较 5 和 7（1 次），选择 5；剩余元素 7 和 9 直接加入。②合并(3, 5, 7, 9)和(6)：比较 3 和 6（1 次），选择 3；比较 5 和 6（1 次），选择 5；比较 7 和 6（1 次），选择 6；剩余元素 7 和 9 直接加入。总共比较 5 次。

二、综合应用题

01．【解答】

$n=10$，需要排序的趟数 $=\lceil \log_2 10 \rceil = 4$，各趟的排序结果如下：

初始序列：503, 87, 512, 61, 908, 170, 897, 275, 653, 462

第一趟：87, 503, 61, 512, 170, 908, 275, 897, 462, 653（长度为 2）

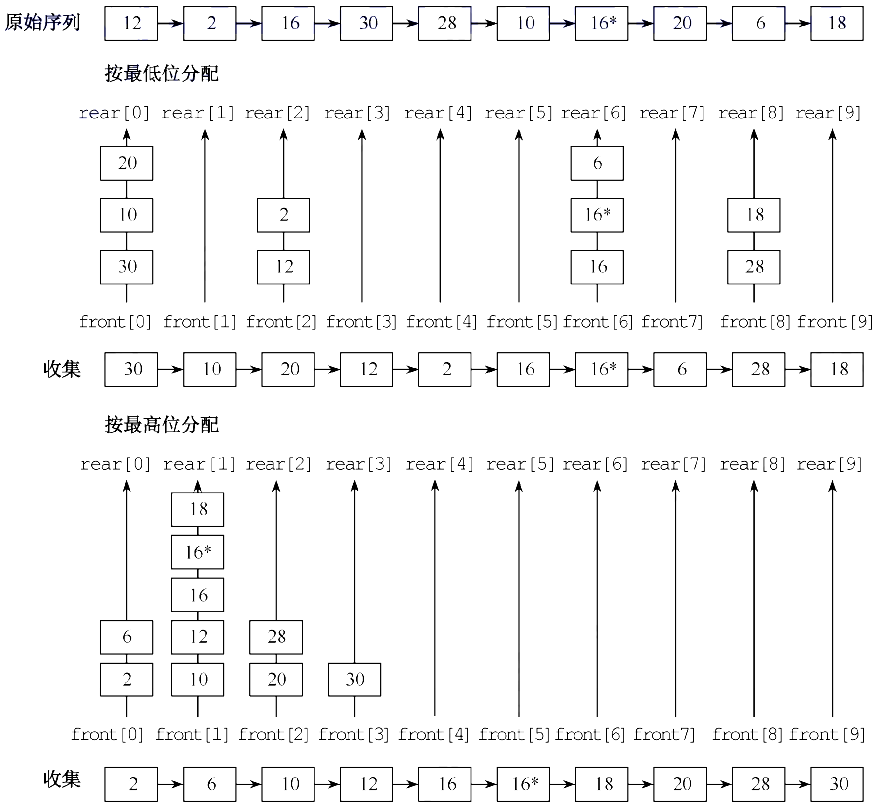
第二趟：61, 87, 503, 512, 170, 275, 897, 908, 462, 653（长度为 4）

第三趟：61, 87, 170, 275, 503, 512, 897, 908, 462, 653（长度为 8）

第四趟：61, 87, 170, 275, 462, 503, 512, 653, 897, 908（长度为 10）

02．【解答】

使用链式队列的基数排序的排序过程如下图所示。



需要通过 2 次分配和收集完成排序。

03.【解答】

cmpCountSort 算法基于计数排序的思想，对序列进行排序。cmpCountSort 算法遍历数组中的元素，count 数组记录比对应待排序数组元素下标大的元素个数，例如，count[1]=3 的意思是数组 a 中有三个元素比 a[1]小，即 a[1]是第四大元素，a[1]的正确位置应是 b[3]。

1）排序结果为 b[6]={-10,10,11,19,25,25}。

2）由代码 for(i=0;i<n-1;i++)和 for(j=i+1;j<n;j++)可知，在循环过程中，每个元素都与它后面的所有元素比较一次（所有元素都两两比较一次），比较次数之和为 $(n-1)+(n-2)+\cdots+1$，所以总比较次数是 $n(n-1)/2$。

3）不是。需要将程序中的 if 语句修改如下：

```
if(a[i]<=a[j]) count[j]++;
else count[i]++;
```

若不加等号，两个相等的元素比较时，前面元素的 count 值会加 1，则导致原序列中靠前的元素在排序后的序列中处于靠后的位置。

8.6 各种内部排序算法的比较及应用

8.6.1 内部排序算法的比较

前面讨论的排序算法很多，对各种排序算法的比较是考研常考的内容。一般基于五个因素进行对比：时间复杂度、空间复杂度、稳定性、适用性和过程特征。

命题追踪 ▶ 各种排序算法的特点、比较和适用场景（2017、2020、2022）

从时间复杂度看：简单选择排序、直接插入排序和冒泡排序平均情况下的时间复杂度都为 $O(n^2)$，且实现过程也较为简单，但直接插入排序和冒泡排序最好情况下的时间复杂度可以达到 $O(n)$，而简单选择排序则与序列的初始状态无关。希尔排序作为插入排序的拓展，对较大规模的数据都可以达到很高的效率，但目前未得出其精确的渐近时间。堆排序利用了一种称为堆的数据结构，可以在线性时间内完成建堆，且在 $O(n\log_2 n)$内完成排序过程。快速排序基于分治的思想，虽然最坏情况下的时间复杂度会达到 $O(n^2)$，但快速排序的平均性能可以达到 $O(n\log_2 n)$，在实际应用中常常优于其他排序算法。归并排序同样基于分治的思想，但其分割子序列与初始序列的排列无关，因此它的最好、最坏和平均时间复杂度均为 $O(n\log_2 n)$。

从空间复杂度看：简单选择排序、插入排序、冒泡排序、希尔排序和堆排序都仅需借助常数个辅助空间。快速排序需要借助一个递归工作栈，平均大小为 $O(\log_2 n)$，当然在最坏情况下可能会增长到 $O(n)$。二路归并排序在合并操作中需要借助较多的辅助空间用于元素复制，大小为 $O(n)$，虽然有方法能克服这个缺点，但其代价是算法会很复杂而且时间复杂度会增加。

命题追踪 ▶ 排序算法的稳定性判断及改进（2021、2023）

从稳定性看：插入排序、冒泡排序、归并排序和基数排序是稳定的排序算法，而简单选择排序、快速排序、希尔排序和堆排序都是不稳定的排序算法。平均时间复杂度为 $O(n\log_2 n)$的稳定排序算法只有归并排序，对于不稳定的排序算法，只需举出一个不稳定的实例即可。对于排序算法的稳定性，读者应能从算法本身的原理上去理解，而不应拘泥于死记硬背。

命题追踪 ▶ 更适合采用顺序存储的排序算法（2017）

从适用性看：折半插入排序、希尔排序、快速排序和堆排序适用于顺序存储。直接插入排序、

冒泡排序、简单选择排序、归并排序和基数排序既适用于顺序存储，又适用于链式存储。

命题追踪 ▶ 根据排序的中间过程判断所采用的排序算法（2009、2010）

命题追踪 ▶ 每趟排序后都至少能确定一个元素的最终位置的排序算法（2012）

从过程特征看：采用不同的排序算法，在一趟或几趟处理后的排序结果通常是不同的，考研题中经常出现给出一个待排序的初始序列和已部分排序的序列，问其采用何种排序算法。这就要对各类排序算法的过程特征十分熟悉，如冒泡排序、简单选择排序和堆排序在每趟处理后都能产生当前的最大值或最小值，而快速排序一趟处理至少能确定一个元素的最终位置等。

表 8.1 列出了各种排序算法的时空复杂度和稳定性情况，其中空间复杂度仅列举了平均情况的复杂度，因为希尔排序的时间复杂度依赖于增量函数，所以无法准确给出其时间复杂度。

表 8.1　各种排序算法的性质

算法种类	时间复杂度			空间复杂度	是否稳定
	最好情况	平均情况	最坏情况		
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否
希尔排序				$O(1)$	否
快速排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	否
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	否
二路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	是
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是

8.6.2　内部排序算法的应用

通常情况，对排序算法的比较和应用应考虑以下情况。

命题追踪 ▶ 选取排序算法时需要考虑的因素（2019）

1．选取排序算法需要考虑的因素

1）待排序的元素个数 n。

2）待排序的元素的初始状态。

3）关键字的结构及其分布情况。

4）稳定性的要求。

5）存储结构及辅助空间的大小限制等。

2．排序算法小结

1）若 n 较小，可采用直接插入排序或简单选择排序。直接插入排序所需的记录移动次数较简单选择排序的多，因此当记录本身信息量较大时，用简单选择排序较好。

2）若 n 较大，应采用时间复杂度为 $O(n\log_2 n)$的排序算法：快速排序、堆排序或归并排序。当待排序的关键字随机分布时，快速排序被认为是目前基于比较的内部排序算法中最好的算法。堆排序所需的辅助空间少于快速排序，且不会出现快速排序可能的最坏情况，这两种排序都是不稳定的。若要求稳定且时间复杂度为 $O(n\log_2 n)$，可选用归并排序。

3）若文件的初始状态已按关键字基本有序，则选用直接插入或冒泡排序为宜。

4）在基于比较的排序算法中，每次比较两个关键字的大小之后，仅出现两种可能的转移，

因此可以用一棵二叉树来描述比较判定过程，由此可以证明：当文件的 n 个关键字随机分布时，任何借助于“比较”的排序算法，至少需要 $O(n\log_2 n)$的时间。

5）若 n 很大，记录的关键字位数较少且可以分解时，采用基数排序较好。

6）当记录本身信息量较大时，为避免耗费大量时间移动记录，可用链表作为存储结构。

8.6.3 本节试题精选

一、单项选择题

01. 若要求排序是稳定的，且关键字为实数，则在下列排序算法中应选（ ）。

A. 直接插入排序 B. 选择排序 C. 基数排序 D. 快速排序

02. 以下排序算法中时间复杂度为 $O(n\log_2 n)$且稳定的是（ ）。

A. 堆排序 B. 快速排序 C. 归并排序 D. 直接插入排序

03. 设被排序的结点序列共有 n 个结点，在该序列中的结点已十分接近有序的情况下，用直接插入排序、归并排序和快速排序对其进行排序，这些算法的时间复杂度应为（ ）。

A. $O(n), O(n), O(n)$ B. $O(n), O(n\log_2 n), O(n\log_2 n)$

C. $O(n), O(n\log_2 n), O(n^2)$ D. $O(n^2), O(n\log_2 n), O(n^2)$

04. 下列排序算法中属于稳定排序的是（①），平均时间复杂度为 $O(n\log_2 n)$的是（②），在最好的情况下，时间复杂度可以达到线性时间的有（③）。（注：多选题）

I. 冒泡排序 II. 堆排序 III. 选择排序 IV. 直接插入排序

V. 希尔排序 VI. 归并排序 VII. 快速排序

05. 就排序算法所用的辅助空间而言，堆排序、快速排序和归并排序的关系是（ ）。

A. 堆排序<快速排序<归并排序 B. 堆排序<归并排序<快速排序

C. 堆排序>归并排序>快速排序 D. 堆排序>快速排序>归并排序

06. 排序趟数与序列的初始状态无关的排序算法是（ ）。

I. 快速排序 II. 简单选择排序 III. 冒泡排序 IV. 基数排序

A. I、III B. II、IV C. I、II、IV D. I、IV

07. 排序趟数与序列的初始状态有关的排序算法是（ ）。

A. 直接插入排序 B. 二路归并排序 C. 快速排序 D. 堆排序

08. 对 n 个元素进行排序，其排序趟数肯定为 $n-1$ 趟的排序算法是（ ）。

A. 直接插入排序和快速排序 B. 冒泡排序和快速排序

C. 简单选择排序和直接插入排序 D. 简单选择排序和冒泡排序

09. 若序列的初始状态为{1, 2, 3, 4, 5, 10, 6, 7, 8, 9}，要想使得排序过程中的元素比较次数最少，则应该采用（ ）方法。

A. 插入排序 B. 选择排序 C. 希尔排序 D. 冒泡排序

10. 对于元素个数相同的不同初始序列，总比较次数相同的排序算法是（ ）。

A. 折半插入排序和简单选择排序 B. 基数排序和归并排序

C. 冒泡排序和快速排序 D. 堆排序

11. 一般情况下，以下查找效率最低的数据结构是（ ）。

A. 有序顺序表 B. 二叉排序树 C. 堆 D. 平衡二叉树

12. 一台计算机具有多核CPU，可以同时执行相互独立的任务。归并排序的各个归并段可以并行执行，在下列排序算法中，不可以并行执行的有（ ）。

I. 基数排序 II. 快速排序 III. 冒泡排序 IV. 堆排序

A. Ⅰ、III　　B. Ⅰ、II　　C. Ⅰ、III、IV　　D. II、IV

13.【2009 统考真题】若数据元素序列{11, 12, 13, 7, 8, 9, 23, 4, 5}是采用下列排序算法之一得到的第二趟排序后的结果，则该排序算法只能是（　）。

A. 冒泡排序　　B. 插入排序　　C. 选择排序　　D. 二路归并排序

14.【2010 统考真题】对一组数据(2, 12, 16, 88, 5, 10)进行排序，若前 3 趟排序结果如下：

第一趟排序结果：2, 12, 16, 5, 10, 88

第二趟排序结果：2, 12, 5, 10, 16, 88

第三趟排序结果：2, 5, 10, 12, 16, 88

则采用的排序算法可能是（　）。

A. 冒泡排序　　B. 希尔排序　　C. 归并排序　　D. 基数排序

15.【2012 统考真题】在内部排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一趟排序。下列排序算法中，每趟排序结束都至少能够确定一个元素最终位置的方法是（　）。

I. 简单选择排序　II. 希尔排序　III. 快速排序

IV. 堆排序　　V. 二路归并排序

A. 仅 I、III、IV　　B. 仅 I、III、V　　C. 仅 II、III、IV　　D. 仅 III、IV、V

16.【2017 统考真题】在内部排序时，若选择了归并排序而未选择插入排序，则可能的理由是（　）。

I. 归并排序的程序代码更短　　II. 归并排序的占用空间更少

III. 归并排序的运行效率更高

A. 仅 II　　B. 仅 III　　C. 仅 I、II　　D. 仅 I、III

17.【2017 统考真题】下列排序算法中，若将顺序存储更换为链式存储，则算法的时间效率会降低的是（　）。

I. 插入排序　II. 选择排序　III. 冒泡排序　IV. 希尔排序　V. 堆排序

A. 仅 I、II　　B. 仅 II、III　　C. 仅 III、IV　　D. 仅 IV、V

18.【2019 统考真题】选择一个排序算法时，除算法的时空效率外，下列因素中，还需要考虑的是（　）。

I. 数据的规模　II. 数据的存储方式　III. 算法的稳定性　IV. 数据的初始状态

A. 仅 III　　B. 仅 I、II　　C. 仅 II、III、IV　　D. I、II、III、IV

19.【2020 统考真题】对大部分元素已有序的数组排序时，直接插入排序比简单选择排序效率更高，其原因是（　）。

I. 直接插入排序过程中元素之间的比较次数更少

II. 直接插入排序过程中所需的辅助空间更少

III. 直接插入排序过程中元素的移动次数更少

A. 仅 I　　B. 仅 III　　C. 仅 I、II　　D. I、II 和 III

20.【2022 统考真题】对数据进行排序时，若采用直接插入排序而不采用快速排序，则可能的原因是（　）

I. 大部分元素已有序　　II. 待排序元素数量很少

III. 要求空间复杂度为 $O(1)$　　IV. 要求排序算法是稳定的

A. 仅 I、II　　B. 仅 III、IV　　C. 仅 I、II、IV　　D. I、II、III、IV

21.【2023 统考真题】下列排序算法中，不稳定的是（　）。

I. 希尔排序　II. 归并排序　III. 快速排序　IV. 堆排序　V. 基数排序

A. 仅I、II　B. 仅II、V　C. 仅I、III、IV　D. 仅III、IV、V

二、综合应用题

01. 设关键字序列为{3, 7, 6, 9, 7, 1, 4, 5, 20}，对其进行排序的最小交换次数是多少？

02. 设顺序表用数组A[]表示，表中元素存储在数组下标1～$m+n$的范围内，前m个元素递增有序，后n个元素递增有序，设计一个算法，使得整个顺序表有序。

1）给出算法的基本设计思想。

2）根据设计思想，采用C/C++描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度与空间复杂度。

03. 设有一个数组中存放了一个无序的关键序列$K_1, K_2, \cdots, K_n$。现要求将K_n放在将元素排序后的正确位置上，试编写实现该功能的算法，要求比较关键字的次数不超过n。

8.6.4 答案与解析

一、单项选择题

01. A

采用排除法。题目要求是稳定排序，因此排除选项B和D，又基数排序不能对`float`和`double`类型的实数进行排序，因此排除选项C。

02. C

堆排序和快速排序不是稳定排序算法，而直接插入排序算法的时间复杂度为$O(n^2)$。

03. C

各种排序算法的时间和空间复杂度、稳定性等见表8.1。

04. ① I、IV、VI　② II、VI、VII　③ I、IV

读者应能从算法的原理上理解算法的稳定性情况。堆排序和归并排序在最坏情况下的时间复杂度与最好情况下的时间复杂度是同一数量级的，都是$O(n\log_2 n)$。

05. A

堆排序的空间复杂度为$O(1)$，因此快速排序的空间复杂度在最坏情况下为$O(n)$，平均空间复杂度为$O(\log_2 n)$，归并排序的空间复杂度为$O(n)$。

06. B

冒泡排序的趟数为1～$n-1$，和序列初态有关。简单选择排序每趟都选出一个最小（或最大）的元素，排序趟数固定为$n-1$。基数排序每趟都要进行分配和收集，排序趟数固定为d。快速排序的趟数和枢轴的选取有关，即和划分是否对称有关，当划分的两个区域分别包含0个元素和$n-1$个元素时，这种最大限度的不对称性发生在序列初态基本有序时。

07. C

直接插入排序的趟数始终为$n-1$，而与序列的初始状态无关。二路归并排序的趟数始终为$\lceil\log_2 n\rceil$，而与序列的初始状态无关。堆排序每趟选出一个最大元素或最小元素，然后调整堆，初始建好堆后，需要$n-1$趟输出和调整堆的操作，而与序列的初始状态无关。

08. C

简单选择排序和直接插入排序的趟数始终为$n-1$。冒泡排序的趟数为1～$n-1$，快速排序的趟数为$\log_2 n$～$n-1$，具体取决于序列的初始状态（快速排序还取决于划分方法）。

09. A

选择排序和序列初态无关，直接排除。初始序列基本有序时，插入排序比较次数较少。本题

中，插入排序仅需比较 $n-1+4$ 次，而希尔排序和冒泡排序的比较次数均远大于此。

10．A

简单选择排序的总比较次数显然是确定的。折半插入排序每趟的比较次数都为 $O(\log_2 m)$（m 为当前已排好序的子序列的长度），因此总比较次数也是确定的。基数排序不是基于比较的排序算法。其他几种排序算法的比较次数显然和序列的初始状态有关。

11．C

堆是用于排序的，在查找时它是无序的，所以效率没有其他查找结构的高。

12．A

基数排序每趟需要利用前一趟已排好的序列，无法并行执行。快速排序每趟划分的子序列互不影响，可以并行执行。冒泡排序每趟对未排序的所有元素进行一趟处理，无法并行执行。堆排序可以并行执行，因为根结点的左右子树构成的子堆在执行过程中是互不影响的。

13．B

每趟冒泡和选择排序后，总会有一个元素被放置在最终位置上。显然，这里{11, 12}和{4, 5}所处的位置并不是最终位置，因此不可能是冒泡和选择排序。二路归并算法经过第二趟后应该是每 4 个元素有序的，但{11, 12, 13, 7}并非有序，因此也不可能是二路归并排序。

14．A

题中给出的排序过程，每一趟都是从前往后依次比较使最大值沉底，符合冒泡排序的特点。分别用其他 3 种排序算法尝试，归并排序第一趟后的结果为(2, 12, 16, 88, 5, 10)，基数排序第一趟后的结果为(10, 2, 12, 5, 16, 88)，希尔排序显然不符合。

15．A

对于选项 I，简单选择排序每次选择未排序序列中的最小元素放入其最终位置。对于选项 II，希尔排序每次对划分的子表进行排序，得到局部有序的结果，所以不能保证每趟结束都能确定一个元素的最终位置。对于选项 III，快速排序每趟结束后都将枢轴元素放到最终位置。对于选项 IV，堆排序属于选择排序，每次都将大根堆的根结点与表尾结点交换，确定其最终位置。对于选项 V，二路归并排序每趟对子表进行两两归并，从而得到若干局部有序的结果，但无法确定最终位置。

16．B

归并排序的代码比插入排序的代码更为复杂，前者的空间复杂度是 $O(n)$，后者是 $O(1)$。但是前者的时间复杂度是 $O(n\log_2 n)$，后者是 $O(n^2)$。

17．D

在顺序存储的条件下，插入排序、选择排序、冒泡排序的时间复杂度都是 $O(n^2)$，更换为链式存储后的时间复杂度还是 $O(n^2)$。希尔排序和堆排序都利用了顺序存储的随机访问特性，而链式存储不支持这种性质，所以时间复杂度会增加。

18．D

当数据规模较小时可选择复杂度为 $O(n^2)$ 的简单排序算法，当数据规模较大时应选择复杂度为 $O(n\log_2 n)$ 的排序算法，当数据规模大到内存无法放下时需选择外部排序算法，选项 I 正确。数据的存储方式主要分为顺序存储和链式存储，有些排序算法（如堆排序）只能用于顺序存储方式，选项 II 正确。若对数据稳定性有要求，则不能选择不稳定的排序算法，选项 III 显然正确。当数据初始基本有序时，直接插入排序的效率最高，冒泡排序和直接插入排序的时间复杂度都是 $O(n)$，而归并排序的时间复杂度依旧是 $O(n\log_2 n)$，选项 IV 正确。

19．A

考虑比较极端的情况，对于有序数组，直接插入排序的比较次数为 $n-1$，简单选择排序的比

较次数始终为$1+2+\cdots+n-1=n(n-1)/2$，选项 I 正确。两种排序算法的辅助空间都是 $O(1)$，无差别，选项 II 错误。初始有序时，移动次数均为 0；对于通常情况，直接插入排序每趟插入都需要依次向后挪位，而简单选择排序只需与找到的最小元素交换位置，后者的移动次数少很多，选项 III 错误。

20. D

直接插入排序和快速排序的特点如下表所示。

	适合初始序列情况	适合元素数量	空间复杂度	稳定性
直接插入排序	大部分元素有序	较少	$O(1)$	稳定
快速排序	基本无序	较多	$O(\log_2 n)$	不稳定

可见，选项 I、II、III、IV 都是采用直接插入排序而不采用快速排序的可能原因。

21. C

稳定的内部排序算法：插入排序、冒泡排序、归并排序和基数排序。不稳定的内部排序算法：简单选择排序、快速排序、希尔排序和堆排序。

二、综合应用题

01.【解答】

直接插入排序的交换次数更多，因此应当采用简单选择排序。

初始序列： 3, 7, 6, 9, 7, 1, 4, 5, 20

第一次：1, 7, 6, 9, 7, 3, 4, 5, 20　交换 1, 3

第二次：1, 3, 6, 9, 7, 7, 4, 5, 20　交换 3, 7

第三次：1, 3, 4, 9, 7, 7, 6, 5, 20　交换 4, 6

第四次：1, 3, 4, 5, 7, 7, 6, 9, 20　交换 5, 9

第五次：1, 3, 4, 5, 6, 7, 7, 9, 20　交换 6, 7

所以最小交换次数为 5（注意这里求的是交换次数，而不是移动次数或比较次数）。

02.【解答】

1）算法的基本设计思想如下：将数组 A[1…m+n] 视为一个已经过 m 趟插入排序的表，则从 $m+1$ 趟开始，将后 n 个元素依次插入前面的有序表中。

2）算法的实现如下：

```
void Insert_Sort(ElemType A[],int m,int n){
    int i,j;
    for(i=m+1;i<=m+n;i++){              //依次将 A[m+1…m+n]插入有序表
        A[0]=A[i];                      //复制为哨兵
        for(j=i-1;A[j]>A[0];j--)        //从后往前插入
            A[j+1]=A[j];                //元素后移
        A[j+1]=A[0];                    //插入
    }
}
```

3）时间复杂度由 m 和 n 共同决定，从上面的算法不难看出，在最坏情况下元素的比较次数为 $O(mn)$，而元素移动的次数为 $O(mn)$，所以时间复杂度为 $O(mn)$。

因为算法只用到了常数个辅助空间，所以空间复杂度为 $O(1)$。

此外，本题也可采用归并排序，将 A[1…m] 和 A[m+1…m+n] 视为两个待归并的有序子序列，算法的时间复杂度为 $O(m+n)$，空间复杂度为 $O(m+n)$。

03.【解答】

基本思想：以 K_n 为枢轴进行一趟快速排序。将快速排序算法改为以最后一个元素为枢轴，

先从前往后，再从后往前。算法的代码如下：

```
int  Partition(ElemType K[],int n){
//交换序列 K[1..n]中的记录，使枢轴到位，并返回其所在位置
    int i=1,j=n;              //设置两个交替变量初值分别为 1 和 n
    ElemType pivot=K[j];      //枢轴
    while(i<j){               //循环跳出条件
        while(i<j&&K[i]<=pivot)
            i++;              //从前往后找比枢轴大的元素
        if(i<j)
            K[j]=K[i];        //移动到右端
        while(i<j&&K[j]>=pivot)
            j--;              //从后往前找比枢轴小的元素
        if(i<j)
            K[i]=K[j];        //移动到左端
    } //while
    K[i]=pivot;               //枢轴存放在最终位置
    return  i;                //返回存放枢轴的位置
}
```

8.7　外部排序

外部排序可能会考查相关概念、方法和排序过程，外部排序的算法比较复杂，不会在算法设计上进行考查。本节的主要内容有：

① 外部排序指的是大文件的排序，即待排序的记录存储在外存中，待排序的文件无法一次性装入内存，需要在内存和外存之间进行多次数据交换，以达到排序整个文件的目的。

② 为减少平衡归并中外存读/写次数所采取的方法：增大归并路数和减少归并段个数。

③ 利用败者树增大归并路数。

④ 利用置换-选择排序增大归并段长度来减少归并段个数。

⑤ 由长度不等的归并段进行多路平衡归并，需要构造最佳归并树。

8.7.1　外部排序的基本概念

前面介绍过的排序算法都是在内存中进行的（称为*内部排序*）。而在许多应用中，经常需要对大文件进行排序，因为文件中的记录很多，无法将整个文件复制进内存中进行排序。因此，需要将待排序的记录存储在外存上，排序时再把数据一部分一部分地调入内存进行排序，在排序过程中需要多次进行内存和外存之间的交换。这种排序算法就称为*外部排序*。

8.7.2　外部排序的方法

文件通常是按块存储在磁盘上的，操作系统也是按块对磁盘上的信息进行读/写的。因为磁盘读/写的机械动作所需的时间远远超过在内存中进行运算的时间（相比而言可以忽略不计），因此在外部排序过程中的时间代价主要考虑访问磁盘的次数，即 I/O 次数。

命题追踪　▶　对大文件排序时使用的排序算法（2016）

外部排序通常采用归并排序算法。它包括两个阶段：①根据内存缓冲区大小，将外存上的文件分成若干长度为 ℓ 的子文件，依次读入内存并利用内部排序算法对它们进行排序，并将排序后得到的有序子文件重新写回外存，称这些有序子文件为*归并段*或*顺串*；②对这些归并段进行逐趟

归并，使归并段（有序子文件）逐渐由小到大，直至得到整个有序文件为止。

例如，一个含有 2000 个记录的文件，每个磁盘块可容纳 125 个记录，首先通过 8 次内部排序得到 8 个初始归并段 R1～R8，每段都含 250 条记录。然后对该文件做如图 8.15 所示的两两归并，直至得到一个有序文件。可以把内存工作区等分为三个缓冲区，如图 8.14 所示，其中的两个为输入缓冲区，一个为输出缓冲区。首先，从两个输入归并段 R1 和 R2 中分别读入一个块，放在输入缓冲区 1 和输入缓冲区 2 中。然后，在内存中进行二路归并，归并后的对象顺序存放在输出缓冲区中。若输出缓冲区中对象存满，则将其顺序写到输出归并段（R1′）中，再清空输出缓冲区，继续存放归并后的对象。若某个输入缓冲区中的对象取空，则从对应的输入归并段中再读取下一块，继续参加归并。如此继续，直到两个输入归并段中的对象全部读入内存并都归并完成为止。当 R1 和 R2 归并完后，再归并 R3 和 R4、R5 和 R6、最后归并 R7 和 R8，这是一趟归并。再把上趟的结果 R1′和 R2′、R3′和 R4′两两归并，这又是一趟归并。最后把 R1″和 R2″两个归并段归并，得到最终的有序文件，一共进行了 3 趟归并。

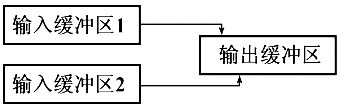


图 8.14　二路归并

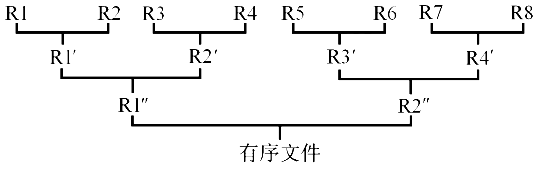


图 8.15　二路归并的排序过程

在外部排序中实现两两归并时，不可能将两个有序段及归并结果段同时存放在内存中，因此需要不停地将数据读出、写入磁盘，而这会耗费大量的时间。一般情况下：

外部排序的总时间 = 内部排序的时间 + 外存信息读/写的时间 + 内部归并的时间

显然，外存信息读/写的时间远大于内部排序和内部归并的时间，因此应着力减少 I/O 次数。外存信息的读/写是以“磁盘块”为单位的，因此可知每趟归并需进行 16 次读和 16 次写，3 趟归并加上内部排序时所需进行的读/写，使得总共需进行 32×3 + 32 = 128 次读/写。

若改用 4 路归并排序，则只需 2 趟归并，外部排序时的总读/写次数便减至 32×2 + 32 = 96。因此，增大归并路数，可减少归并趟数，进而减少总的磁盘 I/O 次数，如图 8.16 所示。

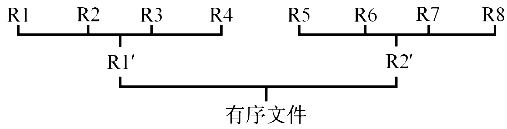


图 8.16　4 路归并的排序过程

一般地，对 r 个初始归并段，做 k 路归并（每趟将 k 个或 k 个以下的有序子文件归并成一个有序子文件）。第一趟可将 r 个初始归并段归并为$\lceil r/k \rceil$个归并段，以后每趟归并将 m 个归并段归并成$\lceil m/k \rceil$个归并段，直至最后形成一个大的归并段为止。树的高度$-1 = \lceil \log_k r \rceil =$ 归并趟数 S。可见，只要增大归并路数 k，或减少初始归并段个数 r，都能减少归并趟数 S，进而减少读/写磁盘的次数，达到提高外部排序速度的目的。

8.7.3　多路平衡归并与败者树

增加归并路数 k 能减少归并趟数 S，进而减少 I/O 次数。然而，增加归并路数 k 时，内部归并的时间将增加。做内部归并时，在 k 个元素中选择关键字最小的元素需要 $k-1$ 次比较。

每趟归并 n 个元素需要做 $(n-1)(k-1)$ 次比较，S 趟归并总共需要的比较次数为[①]

$$S(n-1)(k-1)=\lceil \log_k r\rceil (n-1)(k-1)=\lceil \log_2 r\rceil (n-1)(k-1)/\lceil \log_2 k\rceil$$

式中，$(k-1)/\lceil \log_2 k\rceil$ 随 k 增长而增长，因此内部归并时间亦随 k 的增长而增长。这将抵消因增大 k 而减少外存访问次数所得到的效益。因此，不能使用普通的内部归并排序算法。

为了使内部归并不受 k 的增大的影响，引入了败者树。败者树是树形选择排序的一种变体，可视为一棵完全二叉树。k 个叶结点分别存放 k 个归并段在归并过程中当前参加比较的元素，内部结点用来记忆左右子树中的“失败者”，而让胜利者往上继续进行比较，一直到根结点。若比较两个数，大的为失败者、小的为胜利者，则根结点指向的数为最小数。

命题追踪 ▶ 败者树的实现原理（2024）

如图 8.17(a)所示，b3 与 b4 比较，b4 是败者，将段号 4 写入父结点 ls[4]。b1 与 b2 比较，b2 是败者，将段号 2 写入 ls[3]。b3 与 b4 的胜者 b3 与 b0 比较，b0 是败者，将段号 0 写入 ls[2]。最后两个胜者 b3 与 b1 比较，b1 是败者，将段号 1 写入 ls[1]。而将胜者 b3 的段号 3 写入 ls[0]。此时，根结点 ls[0]所指的段的关键字最小。对于 k 路归并，初始构造败者树需要 $k-1$ 次比较。b3 中的 6 输出后，将下一关键字填入 b3，继续比较。

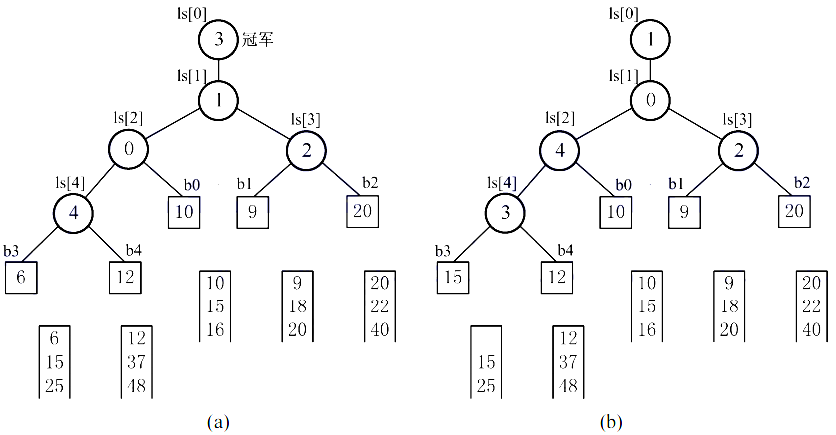


图 8.17　实现 5 路归并的败者树

因为 k 路归并的败者树深度为 $\lceil \log_2 k\rceil+1$，所以从 k 个记录中选择最小关键字，仅需进行 $\lceil \log_2 k\rceil$ 次比较。因此总的比较次数约为

$$S(n-1)\lceil \log_2 k\rceil=\lceil \log_k r\rceil (n-1)\lceil \log_2 k\rceil=(n-1)\lceil \log_2 r\rceil$$

可见，使用败者树后，内部归并的比较次数与 k 无关了。因此，只要内存空间允许，增大归并路数 k 将有效地减少归并树的高度，从而减少 I/O 次数，提高外部排序的速度。

值得说明的是，归并路数 k 并不是越大越好。归并路数 k 增大时，相应地需要增加输入缓冲区的个数。若可供使用的内存空间不变，势必要减少每个输入缓冲区的容量，使得内存、外存交换数据的次数增大。当 k 值过大时，虽然归并趟数会减少，但读/写外存的次数仍会增加。

8.7.4　置换-选择排序（生成初始归并段）

从 8.7.2 节的讨论可知，减少初始归并段个数 r 也可以减少归并趟数 S。若总的记录个数为 n，每个归并段的长度为 ℓ，则归并段的个数 $r=\lceil n/\ell\rceil$。采用内部排序算法得到的各个初始归并段长

① 本节中出现的关于比较次数的公式只是为了帮助读者理解相关原理，无须死记硬背。

度都相同（除最后一段外），它依赖于内部排序时可用内存工作区的大小。因此，必须探索新的方法，用来产生更长的初始归并段，这就是本节要介绍的置换-选择算法。

命题追踪 ▶ 置换-选择排序生成初始归并段的实例（2023）

设初始待排文件为 FI，初始归并段输出文件为 FO，内存工作区为 WA，FO 和 WA 的初始状态为空，WA 可容纳 w 个记录。置换-选择算法的步骤如下：

1）从 FI 输入 w 个记录到工作区 WA。

2）从 WA 中选出其中关键字取最小值的记录，记为 MINIMAX 记录。

3）将 MINIMAX 记录输出到 FO 中去。

4）若 FI 不空，则从 FI 输入下一个记录到 WA 中。

5）从 WA 中所有关键字比 MINIMAX 记录的关键字大的记录中选出最小关键字记录，作为新的 MINIMAX 记录。

6）重复 3）～5），直至在 WA 中选不出新的 MINIMAX 记录为止，由此得到一个初始归并段，输出一个归并段的结束标志到 FO 中去。

7）重复 2）～6），直至 WA 为空。由此得到全部初始归并段。

设待排文件 FI = {17, 21, 05, 44, 10, 12, 56, 32, 29}，WA 容量为 3，排序过程如表 8.2 所示。

表 8.2 置换-选择排序过程示例

输出文件 FO	工作区 WA	输入文件 FI
—	—	17, 21, 05, 44, 10, 12, 56, 32, 29
—	17 21 **05**	44, 10, 12, 56, 32, 29
05	**17** 21 44	10, 12, 56, 32, 29
05 17	10 **21** 44	12, 56, 32, 29
05 17 21	10 12 **44**	56, 32, 29
05 17 21 44	10 12 **56**	32, 29
05 17 21 44 56	10 12 32	29
05 17 21 44 56 #	**10** 12 32	29
10	29 **12** 32	—
10 12	**29** 32	—
10 12 29	**32**	—
10 12 29 32	—	—
10 12 29 32 #	—	—

上述算法，在 WA 中选择 MINIMAX 记录的过程需利用败者树来实现。

8.7.5 最佳归并树

文件经过置换-选择排序后，得到的是长度不等的初始归并段。下面讨论如何组织长度不等的初始归并段的归并顺序，使得 I/O 次数最少。假设由置换-选择排序得到 9 个初始归并段，其长度（记录数）依次为 9, 30, 12, 18, 3, 17, 2, 6, 24。现做 3 路平衡归并，其归并树如图 8.18 所示。

在图 8.18 中，各叶结点表示一个初始归并段，上面的权值表示该归并段的长度，叶结点到根的路径长度表示其参加归并的趟数，各非叶结点代表归并成的新归并段，根结点表示最终生成的归并段。树的带权路径长度 WPL 为归并过程中的总读记录数，所以 I/O 次数= 2×WPL = 484。

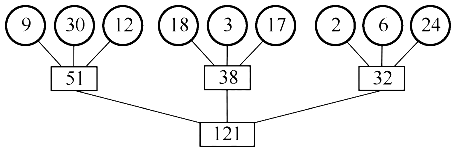


图 8.18　3 路平衡归并的归并树

命题追踪 ▶ 构造三叉哈夫曼树及相关的分析和计算（2013）

显然，归并方案不同，所得归并树不同，树的带权路径长度（I/O 次数）亦不同。为了优化归并树的 WPL，可以将哈夫曼树的思想推广到 m 叉树的情形，在归并树中，让记录数少的初始归并段最先归并，记录数多的初始归并段最晚归并，就可以建立总的 I/O 次数最少的最佳归并树。上述 9 个初始归并段可构造成一棵如图 8.19 所示的归并树，按此树进行归并，仅需对外存进行 446 次读/写，这棵归并树称为最佳归并树。

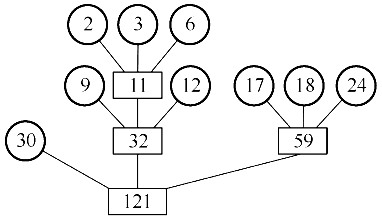


图 8.19　3 路平衡归并的最佳归并树

图 8.19 中的哈夫曼树是一棵严格 3 叉树，即树中只有度为 3 或 0 的结点。若只有 8 个初始归并段，如上例中少了一个长度为 30 的归并段。若在设计归并方案时，缺额的归并段留在最后，即除最后一次做二路归并外，其他各次归并仍是 3 路归并，此归并方案的 I/O 次数为 386。显然，这不是最佳方案。正确的做法是：若初始归并段不足以构成一棵严格 k 叉树（也称正则 k 叉树）时，则需添加长度为 0 的“虚段”，按照哈夫曼树的原则，权为 0 的叶子应离树根最远。因此，最佳归并树应如图 8.20 所示，此时的 I/O 次数仅为 326。

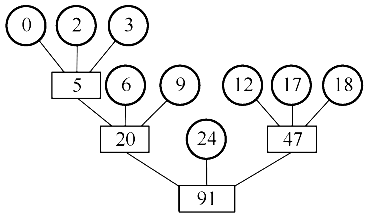


图 8.20　8 个归并段的最佳归并树

如何判定添加虚段的数目？

设度为 0 的结点有 n_0 个，度为 k 的结点有 n_k 个，归并树的结点总数为 n，则有：

- $n = n_k + n_0$　　（总结点数 = 度为 k 的结点数 + 度为 0 的结点数）
- $n = kn_k + 1$　　（总结点数 = 所有结点的度数之和 + 1）

因此，对严格 k 叉树有 $n_0 = (k-1)n_k + 1$，由此可得 $n_k = (n_0 - 1)/(k-1)$。

- 若 $(n_0 - 1)\%(k-1) = 0$（%为取模运算），则说明这 n_0 个叶结点（初始归并段）正好可以构造 k 叉归并树。此时，内结点有 n_k 个。
- 若 $(n_0 - 1)\%(k-1) = u \neq 0$，则说明对于这 n_0 个叶结点，其中有 u 个多余，不能包含在 k 叉归并树中。为构造包含所有 n_0 个初始归并段的 k 叉归并树，应在原有 n_k 个内结点的

基础上再增加1个内结点。它在归并树中代替了一个叶结点的位置，被代替的叶结点加上刚才多出的 u 个叶结点，即再加上 $k-u-1$ 个空归并段，就可以建立归并树。

命题追踪 ▶ 实现最佳归并时需补充的虚段数量的分析（2019）

以图8.20为例，用8个归并段构成3叉树，$(n_0-1)\%(k-1)=(8-1)\%(3-1)=1$，说明7个归并段刚好可以构成一棵严格3叉树（假设把以5为根的树视为一个叶子）。为此，将叶子5变成一个内结点，再添加 $3-1-1=1$ 个空归并段，就可以构成一棵严格3叉树。

8.7.6 本节试题精选

一、单项选择题

01. 外部排序和内部排序的主要区别是（ ）。

A. 内部排序的数据量小，而外部排序的数据量大

B. 内部排序不涉及内、外存数据交换，而外部排序涉及内、外存数据交换

C. 内部排序的速度快，而外部排序的速度慢

D. 内部排序所需的内存小，而外部排序所需的内存大

02. 下列关于外部排序的说法中，正确的是（ ）。

A. 置换选择排序得到的初始归并段的长度一定相等

B. 内外存交换数据的时间只占总排序时间的一小部分

C. 败者树是一棵完全二叉树

D. 外部排序不涉及对文件的读/写操作

03. 多路平衡归并的作用是（ ）。

A. 减少归并趟数　　B. 减少初始归并段的个数

C. 便于实现败者树　　D. 以上都对

04. 设在磁盘上存放有375000个记录，做5路平衡归并排序，内存工作区能容纳600个记录，为把所有记录排好序，需要做（ ）趟归并排序。

A. 3　　B. 4　　C. 5　　D. 6

05. 在下列关于外部排序过程输入/输出缓冲区作用的叙述中，不正确的是（ ）。

A. 暂存输入/输出记录　　B. 内部归并的工作区

C. 产生初始归并段的工作区　　D. 传送用户界面的消息

06. 在做 m 路平衡归并排序的过程中，为实现输入/内部归并/输出的并行处理，需要设置（①）个输入缓冲区和（②）个输出缓冲区。

① A. 2　　B. m　　C. $2m-1$　　D. $2m$

② A. 2　　B. m　　C. $2m-1$　　D. $2m$

07. 若只需3趟排序就可完成64个元素的多路归并排序，则选取的归并路数最少是（ ）。

A. 2　　B. 3　　C. 4　　D. 5

08. 置换-选择排序的作用是（ ）。

A. 用于生成外部排序的初始归并段

B. 完成将一个磁盘文件排序成有序文件的有效的外部排序算法

C. 生成的初始归并段的长度是内存工作区的2倍

D. 对外部排序中输入/归并/输出的并行处理

09. 一个无序文件的 n 个记录采用置换选择排序产生 m 个有序段，则 m 和 n 的关系是（ ）。

A. m 与 n 成正比　　B. $m=\log_2 n$　　C. m 与 n 成反比　　D. 以上都不对

10. 在由 k 路归并构建的败者树中选取一个关键字最小的记录，则所需时间为（　）。

A. $O(1)$　　B. $O(k)$　　C. $O(\log_2 k)$　　D. 以上都不对

11. 下列关于小顶堆和败者树的说法中，正确的是（　）。

I. 败者树从下往上维护，每上一层，只需和失败结点比较 1 次

II. 败者树的每次维护，必定要从叶结点一直走到根结点，不可能从中间停止

III. 堆从上往下维护，每下一层，若其左右孩子均不为空，则需比较 2 次

IV. 堆的每次维护，必定要从根结点一直走到叶结点，不可能从中间停止

A. I、III　　B. II、III　　C. Ⅰ、II、III　　D. I、III、IV

12. 最佳归并树在外部排序中的作用是（　）。

A. 完成 m 路归并排序　　B. 设计 m 路归并排序的优化方案

C. 产生初始归并段　　D. 与锦标赛树的作用类似

13. 在由 m 个初始归并段构建的 k 阶最佳归并树中，不需要补充虚段，则度为 k 的结点个数是（　）。

A. $(m-1)/k$　　B. m/k　　C. $(m-1)/(k-1)$　　D. 无法确定

14. 【2013 统考真题】已知三叉树 T 中 6 个叶结点的权分别是 2, 3, 4, 5, 6, 7，T 的带权（外部）路径长度最小是（　）。

A. 27　　B. 46　　C. 54　　D. 56

15. 【2016 统考真题】对 10TB 的数据文件进行排序，应使用的方法是（　）。

A. 希尔排序　　B. 堆排序　　C. 快速排序　　D. 归并排序

16. 【2019 统考真题】设外存上有 120 个初始归并段，进行 12 路归并时，为实现最佳归并，需要补充的虚段个数是（　）。

A. 1　　B. 2　　C. 3　　D. 4

17. 【2024 统考真题】在外排序中，利用败者树对初始为升序的归并段进行多路归并，败者树中记录“冠军”的结点保存的是（　）。

A. 最大关键字　　B. 最小关键字

C. 最大关键字所在的归并段号　　D. 最小关键字所在的归并段号

二、综合应用题

01. 若某个文件经内部排序得到 80 个初始归并段，试问：

1）若使用多路平衡归并执行 3 趟完成排序，则应取得的归并路数至少应为多少？

2）若操作系统要求一个程序同时可用的输入/输出文件的总数不超过 15 个，则按多路归并至少需要几趟可以完成排序？若限定这个趟数，可取的最低路数是多少？

02. 假设文件有 4500 个记录，在磁盘上每个块可放 75 个记录。计算机中用于排序的内存区可容纳 450 个记录。试问：

1）可以建立多少个初始归并段？每个初始归并段有多少记录？存放于多少个块中？

2）应采用几路归并？请写出归并过程及每趟需要读/写磁盘的块数。

03. 设初始归并段为(10, 15, 31), (9, 20), (22, 34, 37), (6, 15, 42), (12, 37), (84, 95)。试利用败者树进行 m 路归并，手工执行选择最小的 5 个关键字的过程。

04. 给出 12 个初始归并段，其长度分别为 30, 44, 8, 6, 3, 20, 60, 18, 9, 62, 68, 85。现要做 4 路外归并排序，试画出表示归并过程的最佳归并树，并计算该归并树的带权路径长度 WPL。

05. 【2023 统考真题】对含有 n（$n>0$）个记录的文件进行外部排序，采用置换-选择排序生成初始归并段时需要使用一个工作区，工作区中能保存 m 个记录。请回答：

1）若文件中含有 19 个记录，其关键字依次是 51, 94, 37, 92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100，则当 $m=4$ 时，可生成几个初始归并段？各是什么？

2）对任意的 m（$n \gg m > 0$），生成的第一个初始归并段的长度最大值和最小值分别是多少？

8.7.7 答案与解析

一、单项选择题

01. B

外部排序和内部排序最主要的区别是是否涉及内存、外存的数据交换。

02. C

置换选择排序得到的是长度不一定相等的归并段，选项 A 错误。外部排序的主要时间消耗在内外存之间的数据交换上，选项 B 错误。败者树是一棵完全二叉树，选项 C 正确。外部排序包括两个阶段：生成初始归并段和对初始归并段进行归并，这两个阶段都涉及对文件的读/写操作，选项 D 错误。

03. A

多路平衡归并的目的是减少归并趟数，因为当 m 个初始归并段采用 k 路平衡归并时，所需趟数 $s=\lceil \log_k m \rceil$，若不采用多路平衡归并，则其归并趟数大于 s。

04. B

初始归并段的个数 $r=375000/600=625$，因此，归并趟数 $S=\lceil \log_m r \rceil=\lceil \log_5 625 \rceil=4$。第一趟把 625 个归并段归并成 625/5 = 125 个；第二趟把 125 个归并段归并成 125/5 = 25 个；第三趟把 25 个归并段归并成 25/5 = 5 个；第四趟把 5 个归并段归并成 5/5 = 1 个。

05. D

在外部排序过程中输入/输出缓冲区就是排序的内存工作区，例如做 m 路平衡归并需要 m 个输入缓冲区和 1 个输出缓冲区，用以存放参加归并的和归并完成的记录。在产生初始归并段时也可用作内部排序的工作区。它没有传送用户界面的消息的任务。

06. D、A

相比普通的 m 路归并：需增加一个输出缓冲区，当一个输出缓冲区满时，输出一个缓冲区的同时归并程序可向另一个输出缓冲区填充数据，这就实现了内部归并和输出的并行。需增加 m 个输入缓冲区，当 m 个输入缓冲区正在运行时，外部可向新增的 m 个缓冲区写入数据，这就实现了输入和内部归并的并行。综上，需设置 2 个输出缓冲区，$2m$ 个输入缓冲区。

07. C

归并趟数=$\lceil \log_k n \rceil$，其中 k 表示归并的路数，n 表示元素个数，当 $k=4$、$n=64$ 时，归并趟数恰好等于 3，因此选取的归并路数至少是 4。

08. A

置换-选择排序是外部排序中生成初始归并段的方法，用此方法得到的初始归并段的长度是不等长的，其长度平均是传统等长初始归并段的 2 倍，从而使得初始归并段数减少到原来的近二分之一。但是，置换-选择排序不是一种完整的生成有序文件的外部排序算法。

09. D

设内存工作区 $w=1$，则文件{1, 2, 3, 4, 5}产生 1 个有序段，而文件{5, 4, 3, 2, 1}产生 5 个有序段，因此 m 与待排文件、内存工作区大小 w 和 n 都有关，但不是选项 A、B、C 描述的直接关系。

10. C

在败者树中选取最小关键字的时间复杂度取决于败者树的高度，所需时间为 $O(\log_2 k)$。

11．C

选项 I 正确，是败者树的性质。在败者树的维护过程中，会让胜利者一直调整到根结点，选项 II 正确。以小根堆为例，每次调整时，先比较下一层的两个元素（1 次），找出较小值，然后比较当前元素和下一层的较小元素（1 次），以决定是否向下交换位置，选项 III 正确。堆在维护时，可能会在中间某层停止（若此处无须调整），而不一定要走到叶结点，选项 IV 错误。

12．B

最佳归并树在外部排序中的作用是设计 m 路归并排序的优化方案，仿照构造哈夫曼树的方法，以初始归并段的长度为权值，构造具有最小带权路径长度的 m 叉哈夫曼树，可以有效地减少归并过程中的读/写记录数，加快外部排序的速度。

13．C

k 阶最佳归并树中只有度为 0 和 k 的结点。设结点总数为 n，度为 0 的结点数为 n_0，度为 k 的结点数为 n_k，则 $n_0=m$、$n-1=k\times n_k$，$n=n_0+n_k$，因此 $k\times n_k=m+n_k-1$，求得 $n_k=(m-1)/(k-1)$。

14．B

题中的三叉树为使 WPL 最小，必须构造三叉哈夫曼树，应满足$(n_0-1)\%(3-1)=0$ 的条件，因此需添加 1 个权值为 0 的虚叶结点，说明 7 个叶结点刚好可构成一棵严格的三叉树。按照哈夫曼树的原则，权为 0 的叶结点应离树根最远，构造三叉哈夫曼树的过程如下：

① 合并权值最小的三个结点 0, 2, 3，得到新结点的权值 =5，剩下 5, 4, 5, 6, 7。

② 合并权值最小的三个结点 4, 5, 5，得到新结点的权值 =14，剩下 14, 6, 7。

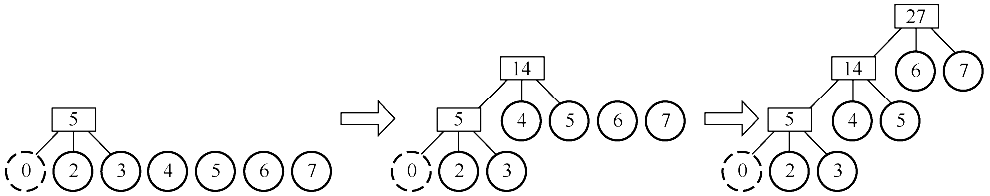
③ 合并权值最小的三个结点 6, 7, 14，得到新结点的权值 =27，仅有 27，建树完成。

$$\mathrm{WPL}=\sum_1^n \text{权值}W_{\text{叶结点}i}\times\text{深度}D_{\text{叶结点}i}=(2+3)\times3+(4+5)\times2+(6+7)\times1=46$$

或

$$\mathrm{WPL}=\sum_1^m \text{权值}W_{\text{分支结点}j}=27+14+5=46$$

每个分支结点的权值都累加了其下面所有分支结点的权值，因此采用第二种方法更方便。



15．D

外部排序指的是大文件的排序，即待排序的记录存储在外存中，待排序的文件无法一次性装入内存，需要在内存和外存之间进行多次数据交换，以达到排序整个文件的目的。外部排序通常采用归并排序算法。选项 A、B、C 都是内部排序的方法。

16．B

在 12 路归并树中只存在度为 0 和度为 12 的结点，设度为 0 的结点数、度为 12 的结点数和要补充的结点数分别为 n_0、n_{12} 和 $n_{补}$，则有 $n_0=120+n_{补}$，$n_0=(12-1)n_{12}+1$，可得 $n_{12}=(120-1+n_{补})/(12-1)$。因为结点数 n_{12} 为整数，所以 $n_{补}$ 是使上式整除的最小整数，求得 $n_{补}=2$。

此外，题中为实现最佳归并，应满足 12 叉哈夫曼树，$n=120$，$m=12$，不满足$(n-1)\%(m-1)=0$ 的条件，因此需要添加两个权值为 0 的叶结点，使得 $n=122$，才能满足条件。

17．D

利用败者树归并升序归并段，每次需要得到当前的最小关键字，因此记录“冠军”的结点保存的只能是最小关键字或最小关键字所在的归并段号。通过败者树找出最小关键字后，还需要找

到该关键字所在的归并段，并移动段内元素，以继续比较下一个元素，下面分别进行讨论：①假设记录“冠军”的结点保存的是最小关键字所在的归并段号，则能直接得到最小关键字及其所在的段，时间复杂度低。②假设记录“冠军”的结点保存的是最小关键字，则查找其所在的段需要检索所有段的首元素是否与该最小关键字相等，时间复杂度高。为了提高效率，记录“冠军”的结点保存的是最小关键字所在的归并段号，而不是最小关键字。

二、综合应用题

01.【解答】

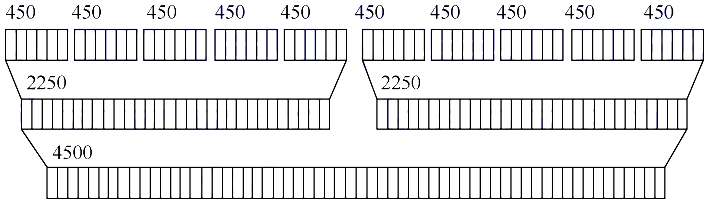
1）设归并路数为 m，初始归并段个数 $r=80$，根据归并趟数计算公式 $S=\lceil \log_m r \rceil=\lceil \log_m 80 \rceil=3$，得 $\log_m 80 \leqslant 3$，$m^3 \geqslant 80$。由此解得 $m \geqslant 5$，即应取的归并路数至少为 5。

2）设多路归并的归并路数为 m，需要 m 个输入缓冲区和 1 个输出缓冲区。一个缓冲区对应一个文件，有 $m+1=15$，因此 $m=14$，可做 14 路归并。由 $S=\lceil \log_m r \rceil=\lceil \log_{14} 80 \rceil=2$，即至少需要 2 趟归并可完成排序。若限定趟数为 2，由 $S=\lceil \log_m 80 \rceil=2$，有 $80 \leqslant m^2$，可取的最低路数为 9。即要在 2 趟内完成排序，进行 9 路归并排序即可。

02.【解答】

1）文件有 4500 个记录，用于排序的内存区可容纳 450 个记录，可建立的初始归并段有 4500/450 = 10 个。每个初始归并段中有 450 个记录，存于 450/75 = 6 个块中。

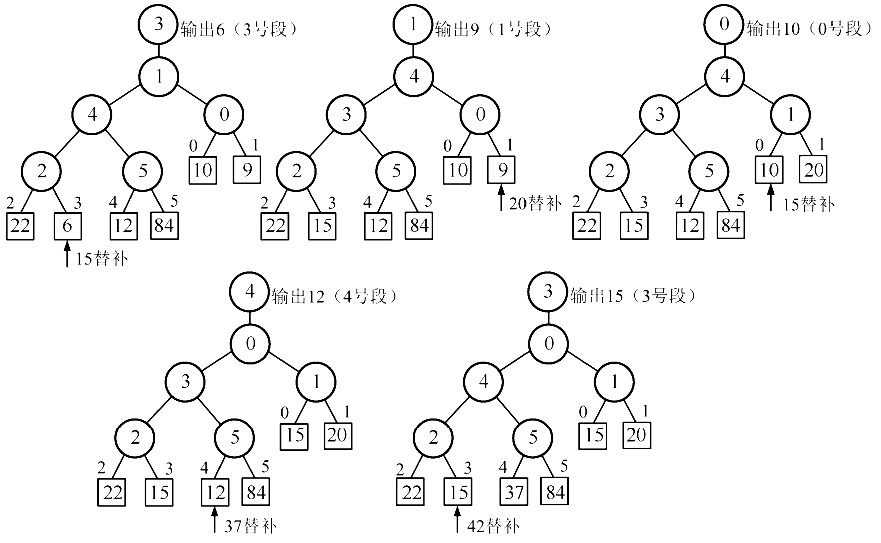
2）内存区可容纳 6 个块，可建立 6 个缓冲区，其中 5 个缓冲区用于输入，1 个缓冲区用于输出，因此可采用 5 路归并，归并过程如下图所示。



共做了 2 趟归并，每趟需要读 60 块、写 60 块。

03.【解答】

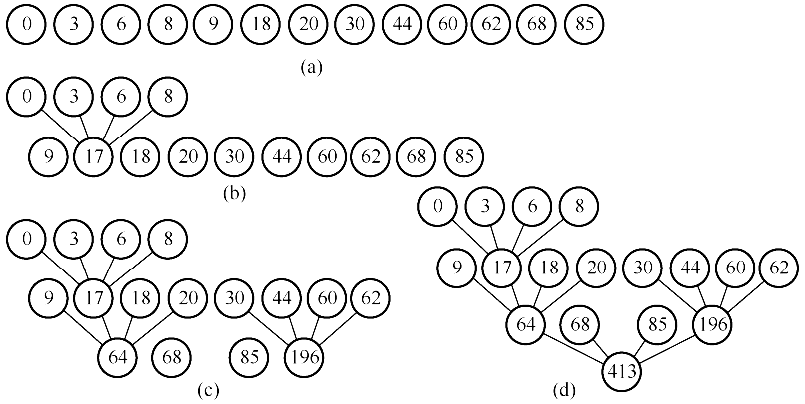
做 6 路归并排序，选择最小的 5 个关键字的败者树如下图所示。



04.【解答】

设初始归并段个数 $n=12$，外归并路数 $k=4$，计算 $(n-1)\%(k-1)=11\%3=2\neq 0$，说明不能做完全的 4 路归并，因为多出了两个初始归并段，必须添加 $k-2-1=1$ 个长度为 0 的空归并段，才能构成严格的 4 路归并树，即每次归并都有 k 个归并段参加归并。

此时，归并树的内结点应有 $(n-1+1)/(k-1)=12/3=4$ 个，如下图所示。



WPL = (3 + 6 + 8)×3 + (9 + 18 + 20 + 30 + 44 + 60 + 62)×2 + (68 + 85)×1 = 51 + 486 + 153 = 690。

05.【解答】

1）当文件中的 n 个记录升序排列时，只生成一个归并段，长度达到最大，为 n。若初始工作区内的 m 个元素都大于输入文件中剩下的所有记录，则第一个归并段的长度就为 m，此时为第一个归并段长度最小的情况。排序过程如下表所示。

输出文件 FO	工作区 WA	输入文件 FI
—	—	51, 94, 37, 92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
—	51, 94, 37, 92	14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37	51, 94, 14, 92	63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51	94, 14, 92, 63	15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63	15, 94, 14, 92	99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92	15, 94, 14, 99	48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92, 94	15, 14, 99, 48	56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92, 94, 99	15, 14, 56, 48	23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92, 94, 99#	15, 14, 56, 48	23, 60, 31, 17, 43, 8, 90, 166, 100
14	15, 23, 56, 48	60, 31, 17, 43, 8, 90, 166, 100
14, 15	60, 23, 56, 48	31, 17, 43, 8, 90, 166, 100
14, 15, 23	60, 31, 56, 48	17, 43, 8, 90, 166, 100
14, 15, 23, 31	60, 17, 56, 48	43, 8, 90, 166, 100
14, 15, 23, 31, 48	60, 17, 56, 43	8, 90, 166, 100
14, 15, 23, 31, 48, 56	60, 17, 8, 43	90, 166, 100
14, 15, 23, 31, 48, 56, 60	90, 17, 8, 43	166, 100
14, 15, 23, 31, 48, 56, 60, 90	166, 17, 8, 43	100
14, 15, 23, 31, 48, 56, 60, 90, 166	100, 17, 8, 43	—
14, 15, 23, 31, 48, 56, 60, 90, 166#	100, 17, 8, 43	—
8	100, 17, 43	—
8, 17	100, 43	—
8, 17, 43	100	—
8, 17, 43, 100	—	—
8, 17, 43, 100#	—	—

生成三个初始归并段，分别是37, 51, 63, 92, 94, 99；14, 15, 23, 31, 48, 56, 60, 90, 166；8, 17, 43, 100。

2）最大值为n，最小值为m。

归纳总结

下面对本章所介绍的排序算法进行一次系统的比较和复习。

1. 直接插入排序、冒泡排序和简单选择排序是基本的排序算法，它们主要用于元素个数n不是很大（$n<10000$）的情形。

它们的平均时间复杂度均为$O(n^2)$，实现也都非常简单。直接插入排序对于规模很小的元素序列（$n\leqslant25$）非常有效。它的时间复杂度与待排序元素序列的初始排列有关。在最好情况下，直接插入排序只需要$n-1$次比较操作就可以完成，且不需要交换操作。在平均情况下和最差情况下，直接插入排序的比较和交换操作都是$O(n^2)$。冒泡排序在最好情况下只需要一趟排序过程就可以完成，此时也只需要$n-1$次比较操作，不需要交换操作。简单选择排序的关键字比较次数与待排序元素序列的初始排列无关，其比较次数总是$O(n^2)$，但元素移动次数则与待排序元素序列的初始排列有关，最好情况下数据不需要移动，最坏情况下元素移动次数不超过$3(n-1)$。

从空间复杂度来看，这三种基本的排序算法除一个辅助元素外，都不需要其他额外空间。从稳定性来看，直接插入排序和冒泡排序都是稳定的，但简单选择排序不是。

2. 对于中等规模的元素序列（$n\leqslant1000$），希尔排序是一种很好的选择。

在希尔排序中，开始时增量较大，分量较多，每个组内的记录数较少，因而记录的比较和移动次数较少，且移动距离较远；到后来步长越来越小（最后一步为1），分组越少，每个组内的记录数越多，但同时记录次序也越来越接近有序，因而记录的比较和移动次数也都比较少。从理论上和实验上都已证明，在希尔排序中，记录的总比较次数和总移动次数比直接插入排序时少得多，特别是当n越大时效果越明显。而且，希尔排序代码简单，基本上不需要什么额外内存，但希尔排序是一种不稳定的排序算法。 公众号：小兔网盘 免费分享无水印PDF

3. 对于元素个数n很大的情况，可以采用快速排序、堆排序、归并排序或基数排序，其中快速排序和堆排序都是不稳定的，而归并排序和基数排序是稳定的排序算法。

快速排序是最通用的高效内部排序算法，特别是它的划分思想经常在很多算法设计题中出现。平均情况下它的时间复杂度为$O(n\log_2n)$，一般情况下所需要的额外空间也是$O(\log_2n)$。但是快速排序在有些情况下也可能会退化（如元素序列已经有序时），时间复杂度会增加到$O(n^2)$，空间复杂度也会增加到$O(n)$。但我们可以通过“三者取中”法来避免最坏情况的发生。

堆排序也是一种高效的内部排序算法，它的时间复杂度是$O(n\log_2n)$，而且没有什么最坏情况会导致堆排序的运行明显变慢，并且堆排序基本上不需要额外的空间。但堆排序不大可能提供比快速排序更好的平均性能。

归并排序也是一个重要的高效排序算法，它的一个重要特性是性能与输入元素序列无关，时间复杂度总是$O(n\log_2n)$。归并排序的主要缺点是需要$O(n)$的额外存储空间。

基数排序是一种相对特殊的排序算法，这类算法不仅是对元素序列的关键字进行比较，更重要的是它们对关键字的不同位部分进行处理和比较。虽然基数排序具有线性增长的时间复杂度，但由于在常规编程环境中，基数排序的线性时间开销实际上并不比快速排序的时间开销小很多，并且由于基数排序基于的关键字抽取算法受到操作系统和排序元素的影响，其适应性远不如普通

的进行比较和交换操作的排序算法。因此，在实际工作中，常规的高效排序算法如快速排序的应用要比基数排序广泛得多。基数排序需要的额外存储空间包括和待排序元素序列规模相同的存储空间及与基数数目相等的一系列桶（一般用队列实现）。

4. 混合使用。

我们可以混合使用不同的排序算法，这也是得到普遍应用的一种算法改进方法，例如，可以将直接插入排序集成到归并排序的算法中。这种混合算法能够充分发挥不同算法各自的优势，从而在整体上得到更好的性能。

思维拓展

下面是一道看起来很吓人的题目：对 n 个整数进行排序，要求时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。

提　示

设待排序整数的范围为 0～65535，设定一个数组 `int count[65535]`并初始化为 0，则所需空间与 n 无关，为 $O(1)$。扫描一遍待排序列 `X[]`，`count[X[i]]++`，时间复杂度为 $O(n)$；再扫描一次 `count[]`，当 `count[i]>0` 时，输出 `count[i]`个 `i`，排序完毕所需的时间复杂度也为 $O(n)$；所以总时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。另外，读者可能会问假如有负整数怎么办，这种情况下可以给所有整数都加上一个偏移量，使之都变成正整数，再使用上述方法。

参 考 文 献

[1] 严蔚敏，吴伟民．数据结构（C 语言版）[M]．北京：清华大学出版社，2018．

[2] 托马斯·科尔曼，查尔斯·雷瑟尔森，罗纳德·李维斯特，等．算法导论[M]．北京：机械工业出版社，2013．

[3] 李春葆，陈良臣，尹为民，等．数据结构教程（C++语言描述）[M]．北京：清华大学出版社，2009．

[4] 陈守孔，胡潇琨，李玲．算法与数据结构考研试题精析[M]．北京：机械工业出版社，2007．

[5] 夏清国．数据结构考研教案[M]．西安：西北工业大学出版社，2006．

[6] 全国考研计算机大纲配套教材专家委员会．全国硕士研究生入学统一考试计算机专业基础综合考试大纲解析[M]．北京：高等教育出版社，2023．

[7] 李春葆，尹为民，陈良臣．数据结构联考辅导教程[M]．北京：清华大学出版社，2010．

4.9元包邮@5分钱/每张

微信刷题小程序:酷酷刷题

免费刷肖1000徐涛腿姐等18位名师题库
支持错题导出、添加笔记、电脑端刷题...